



Frameworkx Standard

Information Framework (SID)

Physical Resource Business Entities

Information Framework Suite

GB922 Physical Resource

Release 16.0.1

September 2016

Latest Update: Frameworkx Release 16	TM Forum Approved Suitable for Conformance
Version 16.0.2	IPR Mode: RAND

Notice

Copyright © TM Forum 2016. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the [TM FORUM IPR Policy](#), must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an "AS IS" basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

TM FORUM invites any TM FORUM Member or any other party that believes it has patent claims that would necessarily be infringed by implementations of this TM Forum Standards Final Deliverable, to notify the TM FORUM Team Administrator and provide an indication of its willingness to grant patent licenses to such patent claims in a manner consistent with the IPR Mode of the TM FORUM Collaboration Project Team that produced this deliverable.

The TM FORUM invites any party to contact the TM FORUM Team Administrator if it is aware of a claim of ownership of any patent claims that would necessarily be infringed by implementations of this TM FORUM Standards Final Deliverable by a patent holder that is not willing to provide a license to such patent claims in a manner consistent with the IPR Mode of the TM FORUM Collaboration Project Team that produced this TM FORUM Standards Final Deliverable. TM FORUM may include such claims on its website, but disclaims any obligation to do so.

TM FORUM takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this TM FORUM Standards Final Deliverable or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any effort to identify any such rights. Information on TM FORUM's procedures with respect to rights in any document or deliverable produced by a TM FORUM Collaboration Project Team can be found on the TM FORUM website. Copies of claims of rights made available for publication and any assurances of licenses to be made available, or the result of an attempt made to obtain a general license or permission for the use of such proprietary rights by implementers or users of this TM FORUM Standards Final Deliverable, can be obtained from



the TM FORUM Team Administrator. TM FORUM makes no representation that any information or list of intellectual property rights will at any time be complete, or that any claims in such list are, in fact, Essential Claims.

Direct inquiries to the TM Forum office:

240 Headquarters Plaza,
East Tower – 10th Floor,
Morristown, NJ 07960 USA
Tel No. +1 973 944 5100
Fax No. +1 973 944 5110
TM Forum Web Page: www.tmforum.org

Table of Contents

Notice.....	2
Table of Contents	4
Table of Figures	5
1. Business Entities	7
1.1. Physical Resource Domain	7
Introduction.....	7
Overview of “Physical Resource”	9
Use Cases.....	10
A Starting Point – Supporting Basic Inventory Concepts	12
Equipment Requirements In More Detail	14
Device Roles	26
1.2. PhysicalResource Business Entities in More Detail.....	28
Filling in the Details	28
PhysicalResource	29
PhysicalDevice in More Detail	33
Hardware in More Detail.....	34
PhysicalConnector.....	35
ManagedHardware	36
PhysicalPort	38
PhysicalContainer	40
PhysicalRoles in More Detail	42
EquipmentHolder in More Detail	44
Adapter and Holder Roles	50
Equipment in More Detail	52
Types of Cards.....	54
AuxiliaryComponents	55
PhysicalComponents.....	56
1.3. Basic Business Entities That Interact with PhysicalResources	57
Party and PartyRole	57
Location	60
Capacity and Redundancy	62
Product	63
1.4. References	68
1.5. Business Entity Definitions	69
2. Administrative Appendix.....	198
2.1. About this document	198
2.2. Document History	198
Version History.....	198
Release History.....	200
2.3. Acknowledgments.....	200

Table of Figures

Figure PR.01 - eTOM Level 0 Concepts and Domains	8
Figure PR.02 - Level One of the Resource Domain of the SID Framework	9
Figure PR.03. Different Aspects of a Device	16
Figure PR.04 - Relating the Physical and Logical Aspects of a Device to Each Other	17
Figure PR.05 - The Makeup of a Device	20
Figure PR.06 - The Concept of “Holding” Hardware	22
Figure PR.07 - Our Simple Example Router	24
Figure PR.09 - The Concept of a DeviceRole	27
Figure PR.10 - Attributes of a PhysicalResource	29
Figure PR.11 - The Relationship Between PhysicalResource and PhysicalResourceSpecification	30
Figure PR.12 - Specifying the PhysicalDevice and Hardware Classes in More Detail	33
Figure PR.13 - Hardware in More Detail	34
Figure PR.14 - Physical Connector	35
Figure PR.15 - ManagedHardware in More Detail	36
Figure PR.16 - PhysicalPort in More Detail	38
Figure PR.17 - PhysicalContainer in More Detail	40
Figure PR.18 - PhysicalDeviceRole and HardwareRole	42
Figure PR.19 - Examples of PhysicalDeviceRoles	43
Figure PR.20 - Specifying Different Types of EquipmentHolders	44
Figure PR.21 - HolderAtomic in More Detail	46
Figure PR.22 - HolderComposite in More Detail	48
Figure PR.23 - AdapterRoles and HolderRoles	50
Figure PR.24 - Equipment in More Detail	52
Figure PR.25 - Relating PhysicalPorts to Cards and Chassis	53
Figure PR.26 - Types of Cards	54
Figure PR.27 - Exemplary Subclasses of AuxiliaryComponent	55
Figure PR.28 - Example Subclasses of PhysicalComponent	56
Figure PR.29 - Overview of Party and PartyRole	58
Figure PR.30 - Representing Administering and Owning a Resource	59
Figure PR.31 - Representing the Physical Location of Physical Objects	61
Figure PR.32 - Simplified Product Model	63

Figure PR.33 - Relationship Between ProductSpec, ResourceSpec and ServiceSpec	64
Figure PR.34 - Relationship Between Product, Service and Resource	66
Figure PR- 35. PhysicalResource Business Entity Model	72
Figure PR- 36. PhysicalResourceSpecification Business Entity Model	75
Figure PR- 37. PhysicalResourceRole Business Entity Model	78
Figure PR- 38. PhysicalHolderRole Business Entity Model	81
Figure PR- 39. Hardware Business Entity Model	88
Figure PR- 40. ManagedHardware Business Entity Model	93
Figure PR- 41. PhysicalContainer Business Entity Model	96
Figure PR- 42. PhysicalDevice Business Entity Model	99
Figure PR- 43. PhysicalConnector Business Entity Model	109
Figure PR- 44. PhysicalPort Business Entity Model	113
Figure PR- 45. Equipment Business Entity Model	120
Figure PR- 46. Card Business Entity Model	126
Figure PR- 47. EquipmentHolder Business Entity Model	143
Figure PR- 48. HolderAtomic Business Entity Model	146
Figure PR- 49. Slot Business Entity Model	150
Figure PR- 50. HolderComposite Business Entity Model	154
Figure PR- 51. SecureHolder Business Entity Model	158
Figure PR- 52. Rack and Chassis Business Entity Model	161
Figure PR- 53. AuxiliaryComponent Business Entity Model	168
Figure PR- 54. CoolingDevice and Fan Business Entity Model	171
Figure PR- 55. PowerSupply Business Entity Model	178
Figure PR- 56. PhysicalComponent Business Entity Model	181
Figure PR- 57. Chip, ASIC, and MemoryComponent Business Entity Model	187



1. Business Entities

1.1. Physical Resource Domain

Introduction

One of the goals of the SID model is to enable a seamless transition from business concepts and definitions to other domains, such as networking. This addendum covers the definition of physical resources.

In looking at "Physical Resource" for the SID model, the modelers used the notion of physical devices and equipment, as defined in the Resource domain of the eTOM [eTOM], as the basis for defining the appropriate business entities. This domain is concerned with the management of equipment. While this Addendum concentrates on using network entities as examples of its concepts, it should be noted that these concepts are generic in nature and all but the network-specific classes apply to other physical entities, such as laptops, servers, and printers. Physical resources can also represent things that are sold, leased, and so forth, like a phone and SIM card, and don't in general have any dependency to network entities. However, this version concentrates on network entities, and will use network entities as examples, in order to better illustrate a complete worked example and to explain some of the more subtle parts of the model.

Network entities are described at the physical network level. The eTOM provides a process model that defines functionality to manage the physical inventory, plan and control physical changes to the network, and provide mechanisms to (for example) measure performance, notify alarms / events and cater for routine administration. Clearly, most of these functions are logical concepts, but equally clearly, appropriate entities need to be available in the physical model to enable it to support these functions. While the initial physical model is aimed at describing networking resources, it contains generic elements that will enable it to be used to represent other types of physical entities as well.

The eTOM [eTOM] implies this separation in its Level 0 Domain map, shown below.

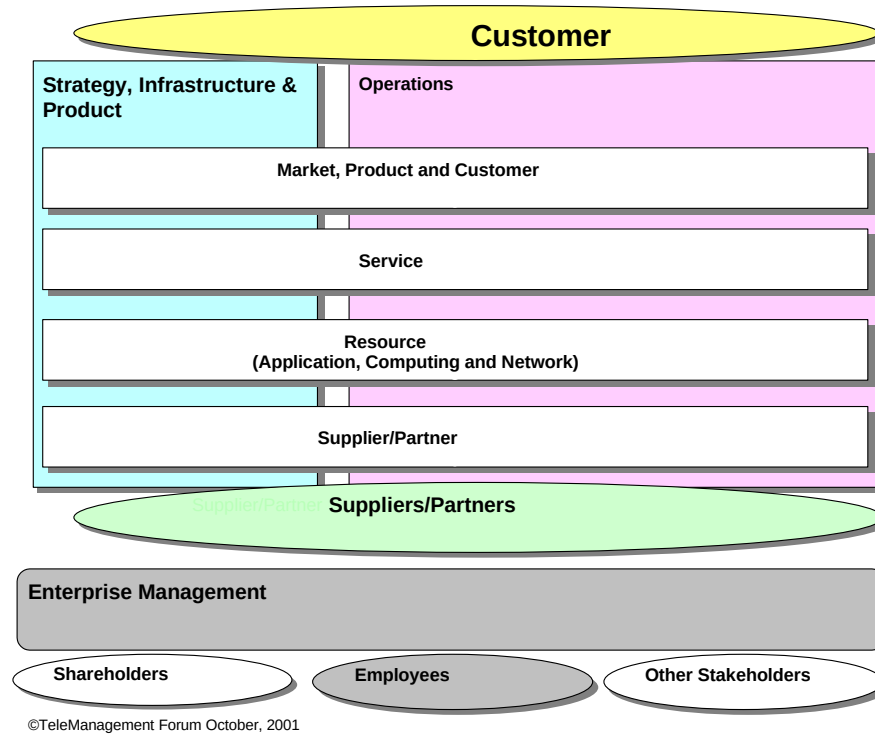


Figure PR.01 - eTOM Level 0 Concepts and Domains

The reader is encouraged to refer to the eTOM for an explanation of the various business processes that are defined to support the concepts in the above domains.

The SID uses the SID Framework, as defined in [GB922], to link the eTOM to the SID. Level 1 of the Resource Domain of the SID Framework is shown in Figure PR.02 below. Please note that this is subject to change over time.

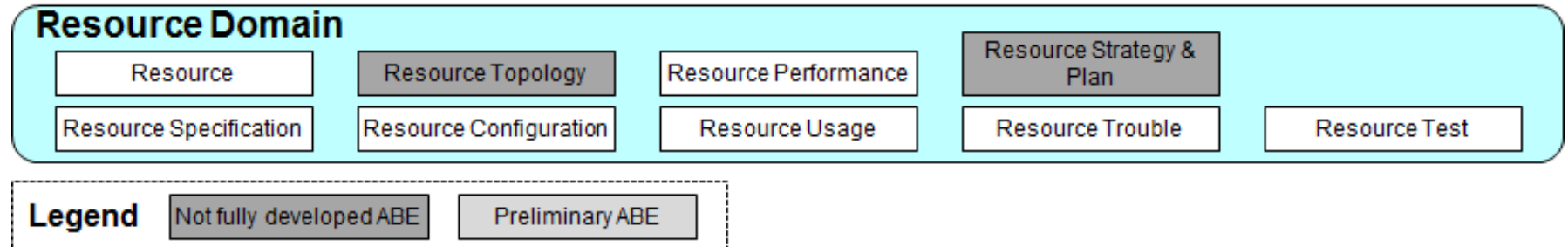


Figure PR.02 - Level One of the Resource Domain of the SID Framework

This Addendum will concentrate on the Resource and Resource Specification ABEs, and set the stage for further work on other ABEs shown in Figure PR.02 in subsequent phases of the SID.

The key concepts looked at were the support of inventory and capacity management functions, as well as support for planning equipment installation and maintenance. Rudimentary ties to related processes defined in the eTOM, as well as relation to other major SID entities (e.g., Product, Party, and others) are also included. This model represents physical resource entities from a business-oriented point-of-view, and uses the best of standard modeling patterns for this area to build in extensibility. For example, patterns are used to capture common relationships and occurrences of physical connections and structures, thereby making the model inherently extensible.

Three important examples of patterns that will be developed in this document are:

- Composite pattern, which provides a common structure for stand-alone and aggregate objects of a given type.
- Entity-EntitySpecification pattern, which separates the invariant characteristics and behavior of an object from its changeable characteristics and behavior.
- Role Object pattern, which uses the notion of roles to extend the use and application of an object.

Overview of “Physical Resource”

This section outlines the approach taken to build the SID physical resource model. The approach used in this document is to build up pieces of the model gradually due to the complexity of the domain and resulting model.

An important goal of the SID effort is to reuse standards (as opposed to reinventing the same concepts) wherever possible. There are several important sources of information for these concepts. The most important of these are [M.3100], [DEN], and [CIM]. Since this model is a federated model that draws from different standards, we need a starting point for resolving the naming conflicts that inevitably arise. We



have elected to start with [M.3100]. This has a number of ramifications, the most important of which is that we will use names directly from [M.3100], or names that are compatible with the names found in [M.3100], wherever possible. The final section of this document contains a detailed data dictionary that defines where classes, attributes, relationships (e.g., associations, aggregations and compositions) presented in this model were derived. Note that this section has a special “synonyms / aliases” section to provide easy correlation to these external documents.

Consistent terminology in a federated model is critical. This terminology, as the model itself, will be introduced in stages. This enables more complex ideas to build on simpler concepts, thereby enabling both an easier as well as a more thorough understanding of the Physical Resource domain.

Regarding terminology, it is important to be able to relate this view (i.e., the “business” view) of the Physical Resource model to other views (e.g., the “system” and the “implementation” views) of these same concepts. By doing this, we achieve the important goal of being able to connect the business requirements to the system representation to how the system is implemented. Thus, we will establish at the outset two important mappings. First, we will map business concepts to system concepts, and organize the system view in a way that facilitates various implementations. (Note that the reverse mapping is also possible – the one chosen is based on the viewpoint of the modeler.)

Second, we will establish a mapping of terminology. For example, business people might think of a “Device” and types of Devices, such as Routers and Switches. The eTOM [eTOM] refers to the term “resource” in its most general sense as something that can represent physical equipment as well as other types of entities.

(Note: we define “Device” as a synonym for Equipment and PhysicalResource. “Device” implicitly means the hardware characteristics of a “Network Device”, such as a router or firewall. Furthermore, a guiding assumption of all SID Addenda is that we will only model objects that we want or need to manage. While a bolt is a useful and needed object, you won’t find an object corresponding to a bolt in this Addendum)

We will over the course of this document define different types of Equipment. As we will see, we need to differentiate between entities that are Equipment and entities that hold, or contain, Equipment. Therefore, we will also develop a suitable class hierarchy to enable these and other concepts to be extended.

Use Cases

The primary use case that drove the design of this model is physical inventory. In this use case, it is important to be able to represent how a particular physical device (equipment) is constructed. For example, for a specific router, we are interested in the number and type of line cards installed in that router, which slots they occupy; whether it has a backup power supply, and other important features that, taken together, describe completely the physical characteristics of this device. We are also interested in being able to describe any spares that exist for the different components of the device being modeled.

There are many different uses of physical inventory as described above. For example, one important use is to be able to understand how different components in a system are physically connected together. Another example is to relate the physical infrastructure and topology to the logical topology of the system.



Another closely related use case is the ability to describe the composition and structure of a physical device as a Bill of Materials. This use case demands a complete listing of all physical components that make up the device, as well as descriptions of where they are located, so that the equipment may be constructed correctly.

There are many other use cases that require a detailed or partial knowledge of physical inventory. These will be explored in future versions of this and other Addenda to GB922.



A Starting Point – Supporting Basic Inventory Concepts

Let's start by categorizing the key entities that need to be modeled:

- Equipment, such as Routers and Switches (for now, this document is focused on chassis-based devices)
- Equipment components, such as line cards
- Equipment “containers”, such as racks and chassis
- Location of the Equipment (e.g., a Router is in this rack in this wiring closet...)
- Location of a physical item within an Equipment (e.g., ability to distinguish between different physical ports on different cards of a Router)
- Important auxiliary Equipment (e.g., cables, connectors, and power supplies) that are necessary for the correct operation of the Equipment

The idea of Equipment containers that can contain Equipment is modeled using three concepts. Fundamental to this is the need to have separate classes to model Equipment and containers of Equipment. We accomplish this by defining two separate classes, called Equipment and EquipmentHolder. While this models the different type of object categories that we need, what we haven't yet done is model how Equipment is contained in an EquipmentHolder. This is done using the EquipmentInHolder aggregation.

It is important to know and keep separate the different states that Equipment can exist in. For example, consider the LineCard mentioned above. The function of the LineCard is (for example) to provide routing and forwarding of packets. This means that a LineCard cannot exist in the ether – it must interact with other elements (e.g., the other components of a Router, of which the LineCard is but one component) and therefore must be contained by a type of EquipmentHolder. However, LineCards can exist outside of being installed in an EquipmentHolder – as spares or as part of a kit that is yet to be built in inventory storage, for example. This model is concerned solely with modeling the physical composition of Network Devices. Thus, it will view Equipment and EquipmentHolders in this light, and concentrate on developing a model of physical equipment that satisfies the use cases mentioned earlier. While identifying a spare LineCard is of course very important, that identification is *not* done as a formal part of this model. However, it can be easily constructed as an extension of this model (or as part of a new model) using this model and the Location model (and possibly others as needed). Note that the Location model (Addendum 1L) can be used to provide a physical description of where an object is.

Back to the EquipmentInHolder relationship, the multiplicity of this relationship is 0 or 1 on the aggregate side, and 0 or more on the component side. The “0” on the aggregate side means that this aggregation is an optional relationship and does not need to be instantiated. This makes sense, because it is certainly possible to have a rack (a type of EquipmentHolder, as we will see) and a Router (a type of Equipment) that are both sitting in an inventory shelf prior to installation. In this case, the router is not yet installed into the Rack, and therefore this aggregation does not yet exist. However, if the EquipmentHolder is being used (the “1” part of its cardinality) then it may contain 0 or more Equipment. The cardinality of 0 or more is important, because it reflects the process of first installing the EquipmentHolder and then populating it with one or more Equipment.



(Note that we could have implemented this as 0..* on both sides of the relationship. The motivation for this, of course, is to be able to use the model to represent time variances (past, present and future states). Remember, however, that this represents a “current state” model. Time variance can be accommodated by providing multiple instances of this state model, one each to represent different time periods. The other problem with 0..* on the EquipmentHolder side of the relationship is that this means that any Equipment can be put in any EquipmentHolder, which is clearly wrong. We can constrain the relationship using EquipmentSpecifications, which will be described later in this document.)

When Equipment, such as our LineCard above, are not installed in an EquipmentHolder, then the EquipmentInHolder relationship is not instantiated, and the Location model takes over, driving the description of where the LineCard is currently located. When the LineCard is installed in an EquipmentHolder, then the position of the LineCard relative to its containing EquipmentHolder is determined by this model. The Location model is still useful, and for example provides the physical location of the Equipment that the LineCard is a part of. However, it does not provide the physical location of the LineCard within its containing EquipmentHolder.

We notice that the multiplicity of EquipmentHolder, Equipment, and EquipmentComponent on the HolderLocatedAt, EquipmentLocatedAt, and ComponentLocatedAt associations, respectively, is 0..n. This reflects the fact that each of these objects can exist at multiple locations. The cardinality at the Location end for each of these three associations is 0..1, meaning that if this (optional) association is instantiated, it specifies a particular Location that an EquipmentHolder, Equipment, or EquipmentComponent is located at. Again, it is important to remember that if the respective cardinalities of these associations was “1” instead of “0..1”, then that would mean that this entity *must* be instantiated. That is why these cardinalities all have a “0” component (to indicate their optional behavior).

The above is a lot of detail. It was felt that one example was needed to orient the reader. Note that for brevity, this version of this Addendum will not provide the amount of detail on each relationship and object as was just provided above. However, similar thinking processes were used to construct these other relationships and objects.

Examples of EquipmentHolders include Racks and Chassis. Each of these can have other components attached to them (e.g., fans, cable ducts, cards, and so forth). We'll represent this set of entities by the generic name “EquipmentComponent” for now – the real class hierarchy is more complicated and will be developed later in this Addendum. Given this type of Equipment, we see that it can be attached to EquipmentHolders as well as Equipment. Thus, we need two additional compositions – ComponentInHolder and ComponentInEquipment – to represent these relationships. Since EquipmentComponent is the entity that is being associated with another entity, both of these relationships define the cardinality on the EquipmentComponent side to be *. Similar to the above reasoning, both of these relationships define the cardinality on the EquipmentHolder and Equipment side to be 0..1.



Note that other parts have also been simplified. For example, in reality, EquipmentHolders can of course contain Equipment as well as other EquipmentHolders – think of the case where a Chassis is contained inside of a Rack. Both a Chassis and a Rack are EquipmentHolders. Details such as these were deliberately left off in order to keep the diagram at this introductory stage of this Addendum as simple as possible. These details will be added into the final model later in this document.

The final part of this first preliminary model is to realize that every object has a physical location. Some applications do care about the exact physical location of every physical object, while others are content to abstract this into a physical location for the overall Equipment and perhaps physical references to key components of the Equipment (e.g., this physical Port is on this physical Card located in this Slot). Therefore, we've identified three generic associations – HolderLocatedAt, EquipmentLocatedAt, and ComponentLocatedAt – that represent this capability. We'll consolidate this as we develop the class hierarchy throughout the remainder of this section.

Equipment Requirements In More Detail

Equipment is at the core of the overall SID model. As such, there are two distinct requirements that Equipment must meet. First, it must be easy to differentiate who owns and administers the Equipment. Otherwise, the Equipment will not be properly accounted for, let alone managed. Second, it is important that customers as well as Service Providers be able to express their needs clearly in order to determine which Equipment can support their requirements.

In the SID, the Party Model (defined in Addendum 1P) is used to describe people as well as organizations. Ownership is represented by a Party playing a particular role. Thus, we will use an association to relate an instance of the PartyRole class to an instance of the Equipment class.

However, this brings us up against a conceptual problem: how do we classify Equipment? Remember that the business person wants to use concepts such as Router, Switch, Firewall in addition to different types of cards, chips, and auxiliary components, such as fans, cables, and power supplies. Clearly, we need to start working on a hierarchy that supports the concept of different types of Equipment.

The objects that we've just described lend themselves to classification using three different categories:

- "Devices", such as Routers, Switches and Firewalls, that perform one or more end-user functions
- "Device Components", such as LineCards and chips, that fit into the Device and are managed components; these objects are directly needed by the Device to perform its primary end-user function(s)
- "Auxiliary Components", such as power supplies, fans and cables; that are required for proper operation of the Device but have a function that is different than the primary end-user function(s) of the Device.

In the above, a "Device" is a managed entity that exists in a single physical structure, or "box". Clearly, a managed entity such as a Router can be a very complicated device, having its functionality supplied by multiple LineCards that it contains. However, it is still important to be able to refer to either the "Device" as a whole or to individual components of the Device (e.g., a PhysicalPort of a particular LineCard in the Device).



Device Components and Auxiliary Components don't form a complete function on their own – rather, their purpose is to support the function(s) performed by their containing Device. However, the functionality of Device Components is directly related to the primary purpose of the device, whereas the functionality of the Auxiliary Component isn't. For example, a LineCard is a type of DeviceComponent because it supplies routing and forwarding functions, which are related to why one uses a router and not some other type of device. A power supply is an AuxiliaryComponent because while power is clearly required by a router, power is not one of the salient characteristics that differentiate a router from other types of Devices.

The SIM tends to define Devices, as described above, as logical components. However, from an inventory point-of-view, a Device usually corresponds to a single physical “box” that is located somewhere. (Alternatively, a Device could correspond to multiple physical boxes. This gets interesting, because oftentimes such a Device appears as a single logical entity, even though it consists of multiple physical entities. An example is a “stackable” switch, which consists of multiple physical switches that can be connected so that they appear to be a single Device to the network). The challenge is to be able to assign one or more locations to the physical box(es) that correspond to the Device.

By providing an entity to represent the physical manifestation of the Device, we can not only track changes to its location, but we can also directly relate the physical manifestation of a router with its logical components (we need the System View of the Physical Resource model to do this)..

An example will help to make this clearer. Consider a Router. People think of a Router as a Device in its own right. However, a Router is in actuality a very complex object, having many components and concepts that exist as managed objects.

Some of these objects are shown conceptually in Figure PR.03. The reader will notice that all but Hardware and AuxiliaryComponent (which represents the “Device Components” and “Auxiliary Components” categories, respectively, from the above description) are logical in nature. The focus of this Addendum is, of course, on the physical aspects of a Device. However, we must be cognizant of the other major entities, so that we have the correct set of classes defined to facilitate relating these logical concepts to the appropriate objects in the physical model. The logical concept of a “Router” is really the collection of the different aspects of a Router, some of which are shown below (note that “Device” in the figure below represents physical and logical components).

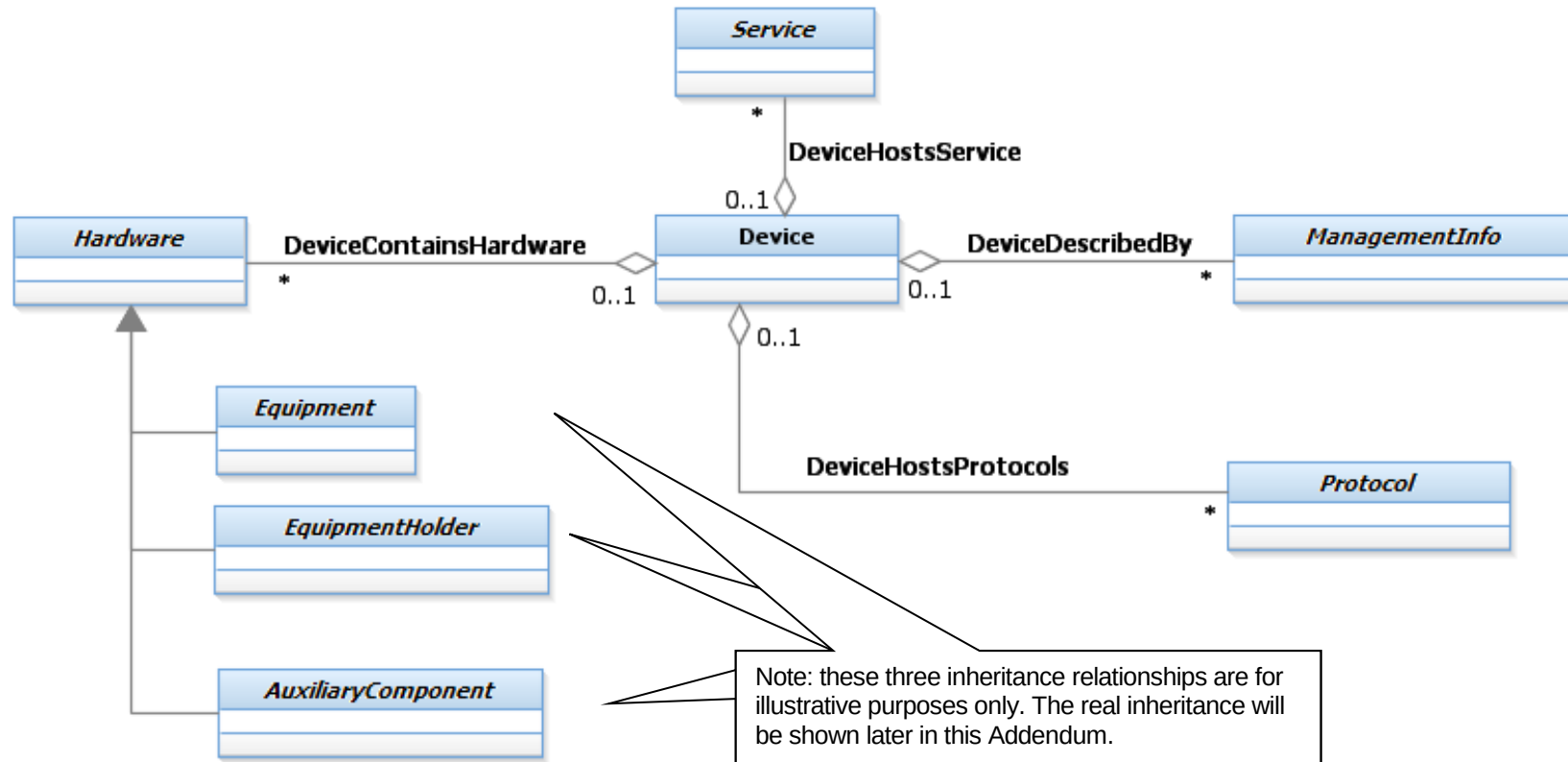


Figure PR.03. Different Aspects of a Device

In the above figure, the “Device” is the “center of the universe”. It is the entity that provides functionality. From the point-of-view of this Addendum, this functionality takes the form of being the physical basis for:

- hosting services (conceptually represented by the DeviceHostsService aggregation – this is developed more in this Addendum and in Addendum 4SO)
- hosting protocols (represented by the DeviceHostsProtocols aggregation)
- containing hardware, which is the main subject of this Addendum.

The first three aggregations represent that services, and protocols (which are all *logical* entities) all require a physical medium to run on. The fourth aggregation enables us to focus on the physical composition and characteristics of a device.



Devices also are the physical basis for hosting different types of management information, such as alarms, statistics, and so forth. This is represented by the DeviceDescribedBy aggregation. ManagementEntity is the superclass for representing entities that represent management information obtained in a managed environment. Specifically, in the process of managing an entity, information of various forms are created. ManagementEntity is the base class for defining and representing different types of management information.

If we remember that a “Device” is really just a type of Resource that can be managed, we can make it more extensible. This is done by defining two classes – PhysicalResource and LogicalResource – that show the inherent correlation between the physical and logical aspects of a given Resource. Then, we can represent particular physical and logical aspects of a resource as subclasses of either PhysicalResource or LogicalResource. This also enables us to define a more accurate and rigorous class structure for the various types of managed objects that are aggregated by a “Device”. This is illustrated in Figure PR.04 below.

Note that this enables the physical and logical description of managed entities to be decoupled from each other. This enables different subject matter experts to work on the areas that they are interested in. The description of PhysicalResources is the subject of this Addendum. The description of LogicalResources is the subject of Addendum 5LR.

Please see Addendum 4SO for an overview of Service entities, and how they interact with Resource entities.

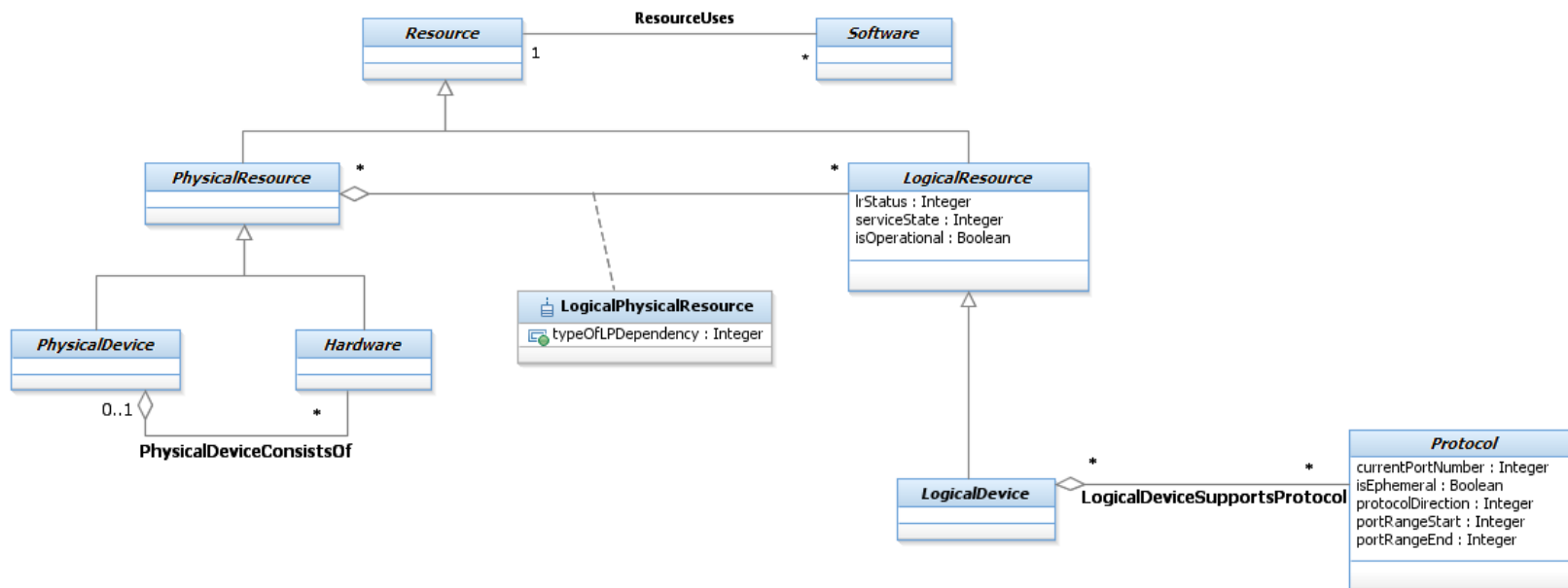


Figure PR.04 - Relating the Physical and Logical Aspects of a Device to Each Other



Note that PhysicalDevice is a subclass of PhysicalResource. PhysicalResource will be used to gather the common management characteristics of “Devices” (which are represented by the PhysicalDevice class) and components of a “Device” (which are represented by subclasses of the Hardware class). Similarly, the logical features seen in Figure PR.04 are grouped under a common base class, called LogicalResource. Note that LogicalResource is explained further in Addendum for Logical Resource, which is a separate Addendum. This Addendum also describes managed entities that are associated with a LogicalResource, such as Software and Protocol. Similarly, Service is in a separate Addendum (Addendum 4SO). Management information is not shown in Figure PR.04 above in order to keep the model as simple as possible for right now.

This structure enables us to focus on specific relationships between PhysicalResource and other entities and LogicalResource and other entities. It is imprecise to say that a Device hosts a Service (as we did in Figure PR.03) – there are in reality several relationships that describe this. A PhysicalResource supports LogicalResources, as indicated by the LogicalPhysicalResource aggregation in Figure PR.04 above. But the actual running of the Service is a logical concept, and hence is associated with a LogicalResource. The value of this Addendum is that it describes all physical entities that are used to host or support logical entities, and corresponds to viewing a Resource as consisting of physical and logical aspects that each needs to be managed.

At this point, we need to be cognizant of existing standards. M.3100 has defined the Equipment and EquipmentHolder classes, as previously mentioned. Unfortunately, it defined those classes mostly from a logical point-of-view, and failed to take into account important physical characteristics that we need. Therefore, we’re going to use the concept of “Hardware” to represent Equipment, EquipmentHolders and AuxiliaryComponents.

This will pay dividends in the future, and enable us to assign common physical attributes, such as weight, length, and others, to the Hardware class so that its subclasses can inherit them. Note that such aspects are missing from M.3100.

Other applicable standards include the original DEN specification, the DMTF’s CIM specification (which included some, but not all, of the DEN work) and the new DEN-ng specification, from which this specification is based. There are similarities and differences between the DEN and CIM specifications and this specification. These will be covered in detail in the Business Entity definitions in the last part of this Addendum. At this point, it is appropriate to bring up two important points. First, the original DEN specification defined a class called NetworkPackage that has the same function as the PhysicalDevice class shown above. This class was deleted when the original DEN specification was incorporated into the CIM specification. Thus, CIM has no concept of a PhysicalDevice. Second, both DEN and CIM have similar concepts to the Equipment and EquipmentHolder classes, though their class structures were different. These will serve to enhance the resulting class structure of this Addendum.

A Device, such as a Router, is in reality represented by a PhysicalDevice, which is made up of different Hardware entities. This enables different users having different use cases to represent the router as either a single physical entity, or as a rich collection of interconnected entities.



PhysicalDevice enables the router to be thought of as a single atomic object. This is necessary for a variety of different use cases. For example, many types of resource planning processes require a single object to represent one or more complex sets of objects. Even though each object in a group of objects can be individually manageable, the group of objects as a whole has meaning. This enables us to refer to the router as, for example, the “Internet Gateway Router”.

The individual line cards that are contained within the Router are represented (as we will see) by subclasses of Equipment. Equipment are particular types of Hardware objects, and are related to a particular PhysicalDevice using the consistsOf aggregation. The following will expand on these concepts.

It is important to be able to manage the individual components of a higher-level object. This is the purpose of the Hardware base class. Hardware consists of Equipment (e.g., a LineCard that performs routing), EquipmentHolders (e.g., a Chassis or some other managed entity whose purpose is to “hold” Equipment), and AuxiliaryComponents (physical components that are required by the “Device” to operate correctly, but whose individual purposes are orthogonal to the main purpose of the device). This is shown in Figure PR.05 below.

The difference between Equipment and AuxiliaryComponents are whether or not the physical object performs a function intrinsic to the main function of the Device. This is best understood by way of example. The main function of a Router is to route and forward packets. A PowerSupply is an AuxiliaryComponent, because while it is needed for the proper operation of the Router, it does not directly help in routing and forwarding packets. A LineCard (that provides routing functionality) is an Equipment component, because its purpose is to route and forward packets. Similar examples exist for different types of equipment, where their criteria may be different. For example, instead of whether it routes or forwards packets, the criterion “does it carry signal” may be useful to appropriately classify components.

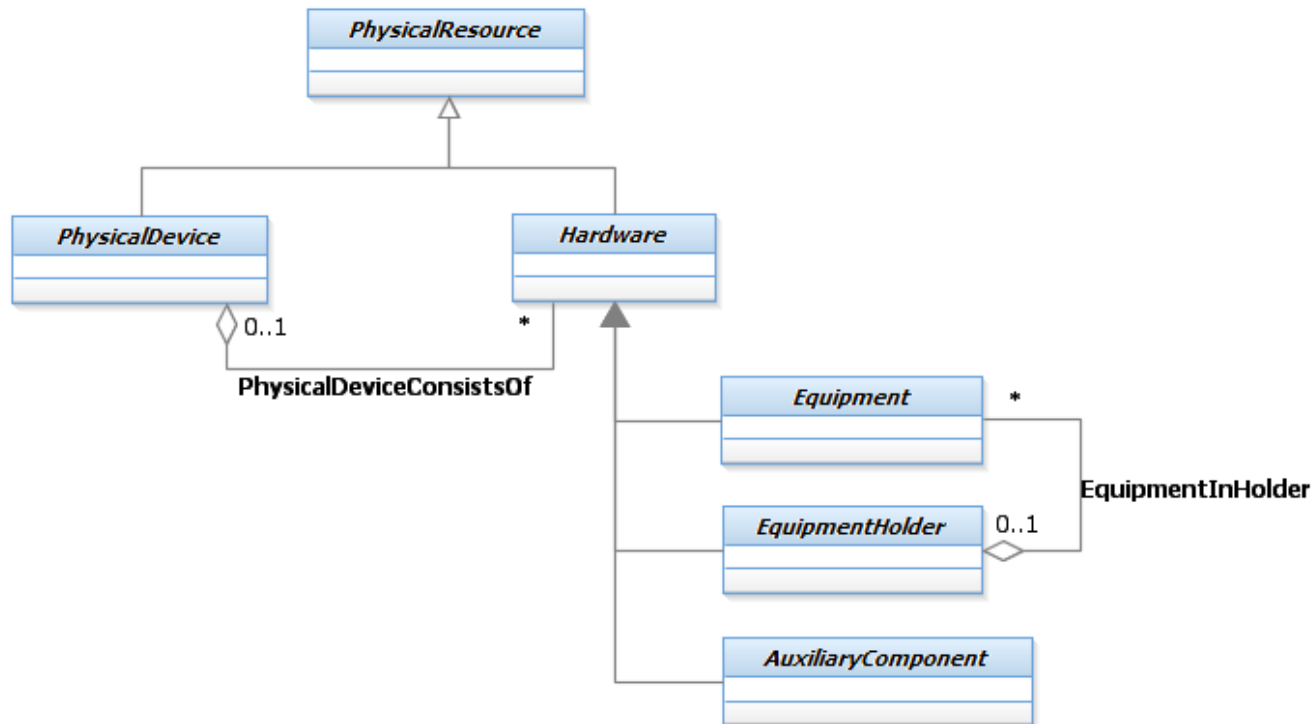


Figure PR.05 - The Makeup of a Device

Figure PR.05 shows that what people conceptually think of a Router or a Switch is in reality a set of managed entities. (Again, it is important to remember that while this Addendum uses network device examples for consistency, the principles and models described in this Addendum apply to other types of physical entities as well.) The physical entity that one can pick up and refer to as a Router or a Switch is represented in Figure PR.05 by an instance of the **PhysicalDevice** class, which is a specific type of **PhysicalResource**. The **PhysicalDevice** may consist of zero or more **Hardware** components, depending on the needs of the user. That is, it is perfectly reasonable for users of this model to use all or parts of it to suit their needs.



Note that the cardinality of the ConsistsOf aggregation is a “may”, not a “must”, as indicated by the 0..1 cardinality of the composite end and the 0..n on the parts end of the aggregation).

Hardware is an abstract base class for defining managed components that are part of a PhysicalDevice. At this level of abstraction, it consists of three principal classes - EquipmentHolder, Equipment, and AuxiliaryComponent. This enables us to satisfy one of our main requirements, in that we can refer to the Network Device as a whole (using the PhysicalDevice object), or specifically to the components of a Network Device (using entities that are subclasses of Equipment, EquipmentHolder, and AuxiliaryComponent). This latter ability is the basis for constructing Bill of Materials, Spare Inventory, and other applications that require a robust physical component model.

Note that AuxiliaryComponent, Equipment, and EquipmentHolder are all subclasses (i.e., children) of the Hardware entity. This enables these objects to inherit the attributes and relationships of their superclasses (i.e., parents). Thus, instead of having to define three relationships (one each between PhysicalDevice and Equipment, EquipmentHolder and AuxiliaryComponent), all we need to do is to define a single relationship between PhysicalDevice and Hardware. This relationship, ConsistsOf, is inherited by the three subclasses of Hardware, and enables each of these subclasses to be “contained in” (or, more precisely, be a part of) a device.

The above model also lays the groundwork for modeling the physical properties of different types of devices. Think again about a Router. A Router is what we refer to as a particular type of “Device”. A Switch is another type of Device. Thus, we have the notion that a “Device” can be a Router or a Switch. This is fortunate, because we don’t want to have to go through this entire procedure again for each new type of Device!

Almost all such Devices are Chassis-based Devices, meaning that the Router functionality is based around a chassis that holds Hardware. The AuxiliaryComponents (e.g., a PowerSupply) are necessary for the Router to operate properly, while the Hardware components supply functionality that directly contributes to the basic functions of the Router (i.e., routing and forwarding packets).

Note that there are many different types of Cards – LineCards, SystemCards, MemoryCards, and so forth. Not all of these Card types are Equipment from the Router’s point-of-view, because not all of them do routing and forwarding functions. A MemoryCard is a good example of this. We’ll deal with this added bit of complexity later in the model by more fully developing the difference between Equipment, EquipmentHolder, and AuxiliaryComponent.

Neither the LineCard nor the PowerSupply can simply exist in the ether, disconnected from the Router! Clearly, both need to be physically contained in the Chassis, which is associated with the Router. Now the question is, are they contained in the same EquipmentHolder, or are there different EquipmentHolders for different types of Equipment and AuxiliaryComponents?

The answer depends on the manufacturer as well as the type and complexity of the device. Some use a single Chassis as a “frame” into which other components plug into, and others use a set of nested EquipmentHolders. Therefore, we must provide the ability for both options to be modeled.

What’s an example of nested EquipmentHolders? For large Routers and Switches, it is quite common to see a Chassis with a number of Slots into which different types of Cards are plugged in. The Slot functions as a type of EquipmentHolder. Therefore, we can define a single relationship, HoldsHardware, to enable an EquipmentHolder to hold any type of Hardware. This is exactly analogous to our earlier use of the ContainsHardware relationship. This is shown in Figure PR.06 below.

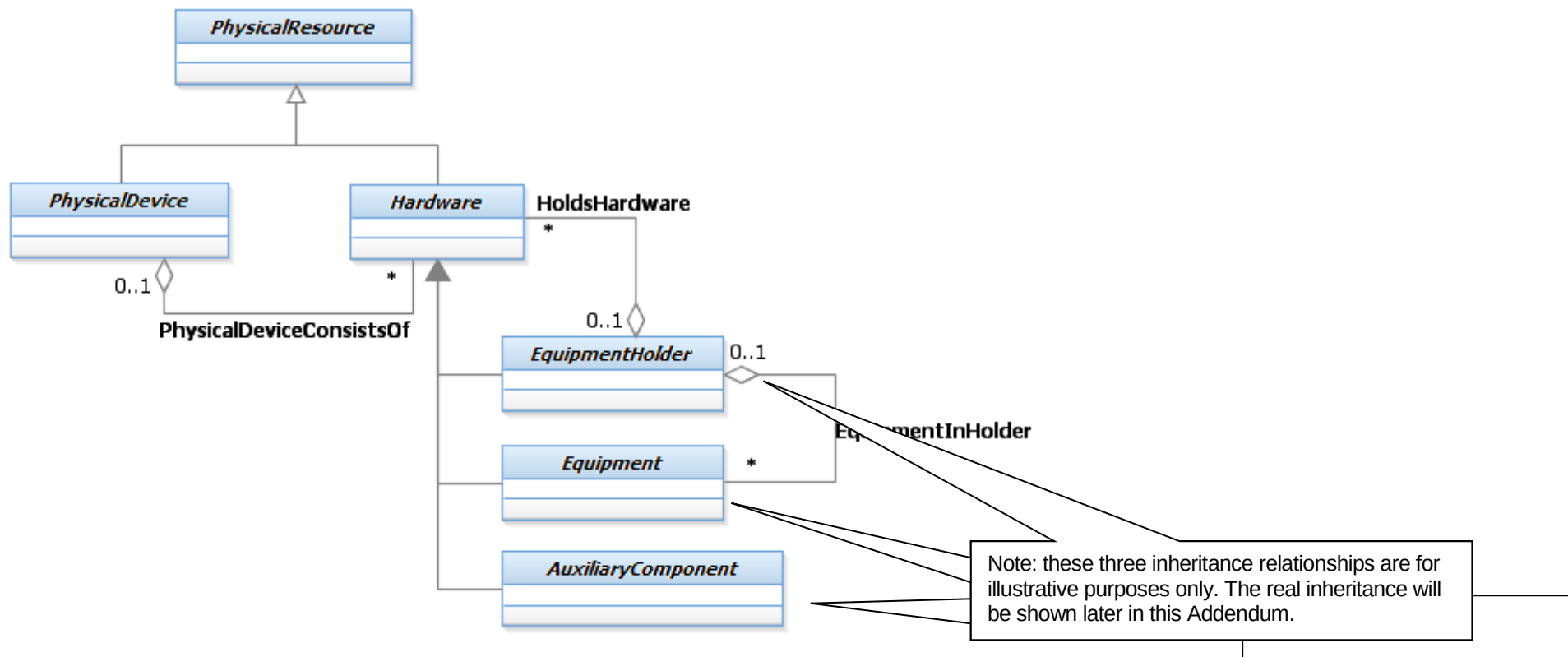


Figure PR.06 - The Concept of “Holding” Hardware

(Before we celebrate too early, we must also realize that certain Cards have connectors on them into which “DaughterCards” can attach. This, however, is too complicated for where we’re at in our model, so we’ll deal with that later in this document.)



Recapping, this is what we know:

- A Router is-a type of Device
- There are two types of EquipmentHolders, Slots and Chassis
- A PowerSupply is-a type of AuxiliaryComponent
- A LineCard is-a type of Card, which is-a type of Hardware (for a more concrete illustration, we are substituting different types of cards that are part of the “real” model that is being developed; a LineCard is a synonym for a NetworkCard, but Figure PR.07 shows other types of Cards as well)
- A NetworkCard fits into a Slot of a Chassis
- A PowerSupply fits directly into the Chassis
- The physical part of the Router consists of all of the above Equipment and EquipmentHolders

Thus, what we conceptually have is the following model of our simple Router (note that not all subclasses are shown; rather, this is a simple example to define the concepts that we have developed to date):

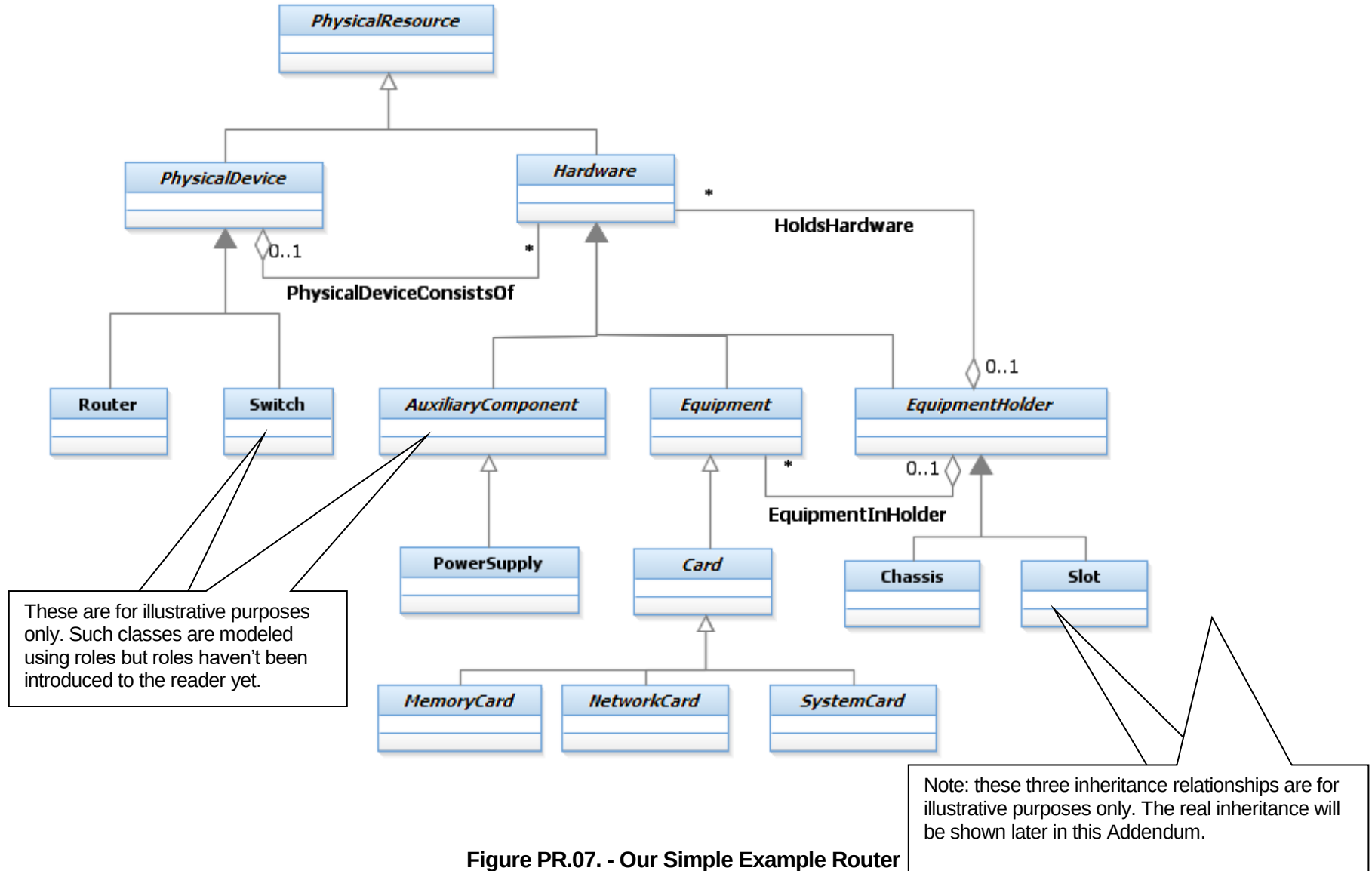


Figure PR.07. - Our Simple Example Router



The power of inheritance works well for us. Since PowerSupply and NetworkCard both ultimately inherit from Hardware, they also inherit the attributes and relationships that Hardware participates in. Thus, both a PowerSupply as well as a LineCard can be contained in an EquipmentHolder (not necessarily the same one) via the HoldsHardware aggregation. Similarly, both can be used to build a Router via the ConsistsOf aggregation.

As for the two EquipmentHolders, we've chosen to make them siblings. That is, they are two separate objects that are derived from a common superclass (EquipmentHolder). This is an example of the object-oriented concept of *specialization*. Specialization is used to take a single concept and add detail to it. The added detail is in the form of subclasses that inherit all of the attributes and relationships (and, in the System view, methods and constraints as well) of their parent superclasses, but add their own characteristics and behavior (in the form of attributes and relationships, as well as methods and constraints in the System view).

In this case, there is a fundamental difference between a Slot and a Chassis. However, since both of these classes are subclasses (i.e., types) of EquipmentHolder, we can simply reuse the HoldsHardware composition to enable a Chassis to contain Slots. Finally, the HoldsHardware aggregation can also be used to nest EquipmentHolders, as previously discussed.

We've also added some additional detail, showing that there are (for now) three different types of Cards – MemoryCards, NetworkCards, and SystemCards. A MemoryCard is a type of Card that is dedicated to providing additional memory for use by other components of a Resource. Such Cards can be used to expand memory, or provide different types of memory.

A NetworkCard is a type of Card that is dedicated to providing networking functions, such as routing and forwarding. Line cards and port adapter cards are examples of this type of card.

Finally, a SystemCard is a type of Card that is dedicated to providing system functions. Main processor boards and controller boards are examples of this type of Card.

Given this introduction, we will now introduce the concept of Roles. Roles are a fundamental means of providing an extensible model. An innovative feature is that it has physical as well as logical roles.



Device Roles

Before we dive deeper, let's clear up one thing that we previously hinted at. Figure PR.05 defined a Switch and a Router as types of Devices. We all know the basic difference between a Switch and a Router – the former forwards traffic, while the latter routes and forwards traffic. But what about the so-called “Layer 3” switches, which are Switches that have Routing capability?

Assume that our model actually did implement a router and a switch subclass of PhysicalDevice. One would then be tempted to invent a new subclass, called Layer3Switch, which subclasses a Switch and adds routing capabilities, to handle this new type of device. However, this is a poor solution, because now every time routing changes, we have to update the Router and the Layer3Switch class. Besides, this implies that a Layer3Switch can do everything that a full-blown Router can do, which is almost never the case. There are other problems too, but this is enough for now.

One may then be tempted to use multiple inheritance to solve this problem. However, this creates a new problem – one of extensibility. What if there is a “Layer4Switch” (unfortunately, some vendors do define such an animal!)? What if we want to differentiate between the type of routing done in a Router vs. those done in the Layer3Switch vs. those done in the Layer4Switch? What if there is a Router that has firewalling capabilities? The list is endless. Multiple inheritance cannot be used to solve all of these problems, and besides, many implementations don't support multiple inheritance. We don't want to compromise our ability to build a system view out of this business view!

Instead, a much more elegant and extensible solution is available – we can use the notion of roles. In fact, one of the strengths of a good model is to be able to reuse the same set of patterns over and over. Roles form a core part of the Party model, which (among other things) describes People, Organizations, and the functions (i.e., *roles*) that they play. This is documented in Addendum 1P (the Common Business Entity Addendum that defines Party and PartyRole).

Our approach is now simplified. Instead of trying to either define many subclasses or introduce multiple inheritance, we can instead define a set of roles that the device is meant to play. (It should be noted that this is another reason why the concept of a managed device is a good one – now, we can define a base concept of a managed device, and model its functionality by associating one or more roles to it as appropriate). This solves the mess of having the same generic function (such as routing) assigned to two different types of devices that implement that same generic function in different ways, producing different subsets of functionality.



Since the networking industry has shown to be ever-evolving, inheritance is not appropriate for capturing this behavior, as the behavior itself changes over time. In addition, by capturing the key functional concepts as roles, we can model present and future Devices as entities that can play many roles at one time.

Thus, by modeling DeviceRole as a separate concept from Device, we can represent these complex behaviors in an extensible manner.

Figure PR. shows a simple model introducing the concept of a DeviceRole:

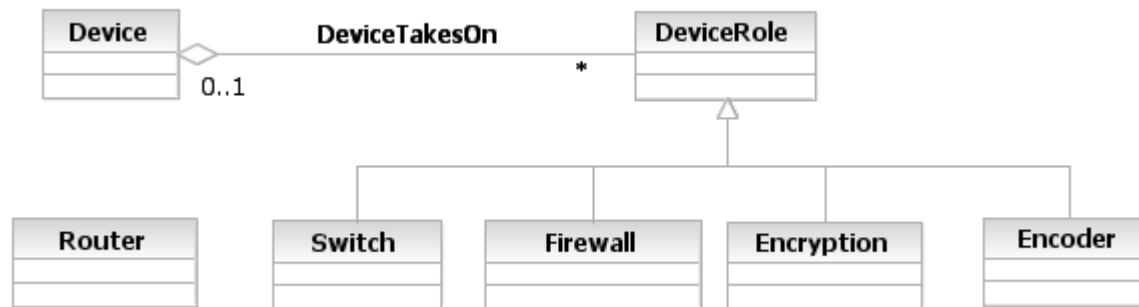


Figure PR.09 - The Concept of a DeviceRole

It should be noted that Figure PR. does not define all of the different types of roles that a Device can take on. This concept is clearly not a physical one, but rather a logical one, and will show up in other Addenda. It is brought up in this Addendum because this is a critical concept for modeling different types of devices, and the Physical Resource model must be able to interface with this concept. It also inspires us to see if there isn't a similar concept that can be used to abstract similar differences in the physical world. In fact, there is a similarity, and is defined in the section titled EquipmentHolder in More Detail later in this Addendum.



1.2. Physical Resource Business Entities in More Detail

The purpose of this section is to take the basic model presented in Figure PR.07 and expand it to add more detail.

Filling in the Details

We're now at the stage where we can add some detail. Given that we are building a Business Model, we will simply strive to identify certain essential attributes and relationships for PhysicalDevice as well as Hardware. We will not define methods, low-level attributes, association classes, and in general all of the detail that fully describe these entities. This detail will be provided in an accompanying System model. This model has already been built and is currently being refined. It will be formalized as part of a future phase of the SID work.

The following rules were used to define the set of attributes and relationships for this model:

- Keep to high-level, business-oriented concepts (e.g., while some applications might need to know specific details about a physical object, the purpose of this model is to simply identify the key concepts necessary for the purposes specified in our use cases)
- Note that these entities represent business concepts in a Service Provider that conforms to the eTOM process model [eTOM] (specifically, the Resource Development and Operations processes).
- This is intended as a "minimalist" model. Subtypes and attributes should be added as required to suit the needs of a particular application. The attributes shown should be considered as suggested, not required, unless they are specified as mandatory.
- Parent attributes are not repeated in their subclasses
- Only significant entities are shown in the "related business entities" cells
- Entities that are outside the scope of this model facet are shown with a white fill color.

PhysicalResource

We already introduced the notion of this entity as a superclass of our two main categories of physical objects: Device and Hardware. Hardware defined three types of objects – Equipment, EquipmentHolder, and AuxiliaryComponent. We abstracted this set of entities into a single entity, called PhysicalResource. This is defined as an abstract base class for describing different types of hardware that constitute a Product. It has two main purposes: (1) to collect common attributes and relationships for all hardware, and (2) to provide a convenient, single point where relationships with other managed objects can be defined.

Now, it's time to see if we can discover common attributes that these three types of entities have in common. We'll tie this into the concept of a PhysicalResourceSpecification in a minute.

The most obvious types of attributes that physical objects have in common are the set of attributes that define specific characteristics about a given physical object. These include who manufactured it and when, and what the model number of the object is. To this, inventory encourages us to add part number, SKU number, and serial number. This gives us the following preliminary picture of a PhysicalResource:

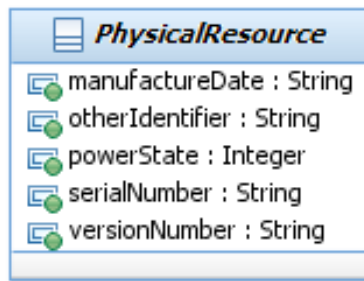


Figure PR.10 - Attributes of a PhysicalResource

These six attributes are inherited by all subclasses of PhysicalResource.

The key in making the above model extensible and reusable is to use the concept of a Specification. A Specification is a way of defining the invariant characteristics and behavior (attributes and relationships in the Business View, along with methods and constraints in the System View) of a managed entity.



Now, if we look at these attributes, we see that some of them lend themselves to being templated, while some don't. That is, we can use the concept of a PhysicalResourceSpecification to supply the ModelNumber, PartNumber, SKUNumber, and VendorName attributes, because these correspond to a generic specification describing a particular type of PhysicalResource. The other two attributes (ManufactureDate and SerialNumber) are specific to a particular instance of a PhysicalResource object, and therefore are not contained in the PhysicalResourceSpecification.

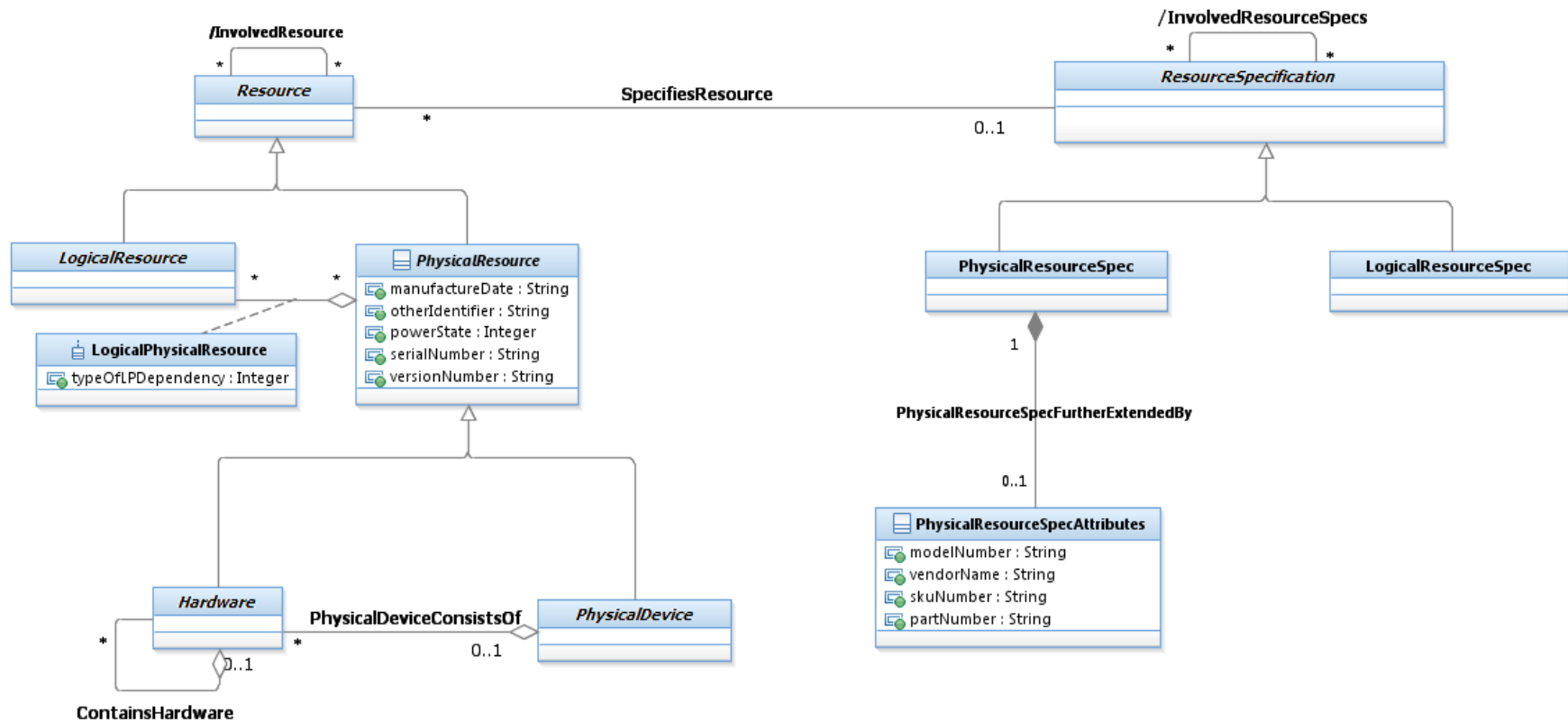


Figure PR.11 - The Relationship Between PhysicalResource and PhysicalResourceSpecification

We separate PhysicalResourceSpecs from LogicalResourceSpecs because they are fundamentally different in nature. In addition, they have different attributes, methods, constraints, and relationships in the System view.



Both PhysicalResourceSpecs and LogicalResourceSpecs use the composite pattern for extensibility. Only the PhysicalResourceSpec classes are shown in Figure PR. above for simplicity.

A PhysicalResourceSpecAtomic class is a concrete class for describing specific attributes, behavior, relationships, constraints, and semantics for building PhysicalResource objects. The purpose of this class is to be able to track PhysicalResourceSpecifications separately from other types of ResourceSpecifications. Note that this class inherits the SpecifiesResource aggregation, and therefore can be used with the corresponding PhysicalResource class.

The difference between this class and the PhysicalResourceSpecComposite class is that this class represents stand-alone PhysicalResourceSpecifications. The PhysicalResourceSpecComposite class represents a specification that is in reality made up of a set (usually a hierarchy) of PhysicalResourceSpecifications.

The composite pattern is completed by defining the HasPhysicalResourceSpecs aggregation. This aggregation defines the set of PhysicalResourceSpecs that are contained by this particular PhysicalResourceSpecComposite entity.

The SpecifiesResource aggregation defines the particular ResourceSpecification that is used to provide the invariant characteristics (i.e., attributes and relationships, as well as methods and constraints in the System View) of a given Resource. This enables multiple Resources that each use a common set of attributes, methods, constraints, and/or relationships to be related to a single invariant specification of those characteristics and behavior. This facilitates their updating.

The cardinality of the SpecifiesResource aggregation is 1 on the ResourceSpecification side and 0..n on the Resource side. This means that a ResourceSpecification can be written that isn't related to any specific Resources, but if a Resource is built, it must be derived from a ResourceSpecification. Furthermore, this is an aggregation because this is a whole-part relationship: the ResourceSpecification defines the invariant attributes and behavior of zero or more Resources, and each Resource derived from the ResourceSpecification uses all of these invariant attributes and behavior (and presumably adds its own instance-specific attributes and behavior).

Since this aggregation is defined at the Resource and ResourceSpecification level, each of the subclasses of Resource and ResourceSpecification inherit its use. This is why (for example) there isn't a relationship connecting a PhysicalResourceSpec to a PhysicalResource.



The LResSpecBindsToPResSpec association represents the binding between a set of LogicalResourceSpecifications and a set of PhysicalResourceSpecifications. The semantics of this binding are represented by the LogicalPhysicalResourceSpec association class.

A convenient way to view the Entity-EntitySpecification pattern is that EntitySpecifications represent templates for building an Entity, while Entities themselves represent instances of managed objects.

The various subclasses of the PhysicalResource class shown in earlier figures still inherit all of the attributes and relationships that were shown in, for example, Figure PR.07, because the SpecifiesResource association is used to add the attributes and relationships of a ResourceSpecification to a Resource. In fact, inheritance means that each subclass now inherits the ability to have its own PhysicalResourceSpecification (or subclass of PhysicalResourceSpecification). This enables any specific needs of one of the various subclasses of PhysicalResource to be met by subclassing the PhysicalResourceSpecification corresponding to that object.

We'll look at the definitions of each of these major entities in the following sections.

PhysicalDevice in More Detail

PhysicalDevice represents the completed physical assembly, including all components, of the network device. There are two important aspects to a PhysicalDevice – its physical components and the role(s) that these components play in the network.

A PhysicalDevice may itself be a set of devices. This can become quite complex and will be treated in the next version of this. Therefore, we will use the composite pattern to model PhysicalDevices within PhysicalDevices. As an aid to applications, a special attribute (deviceGroupID) is used to signify this.

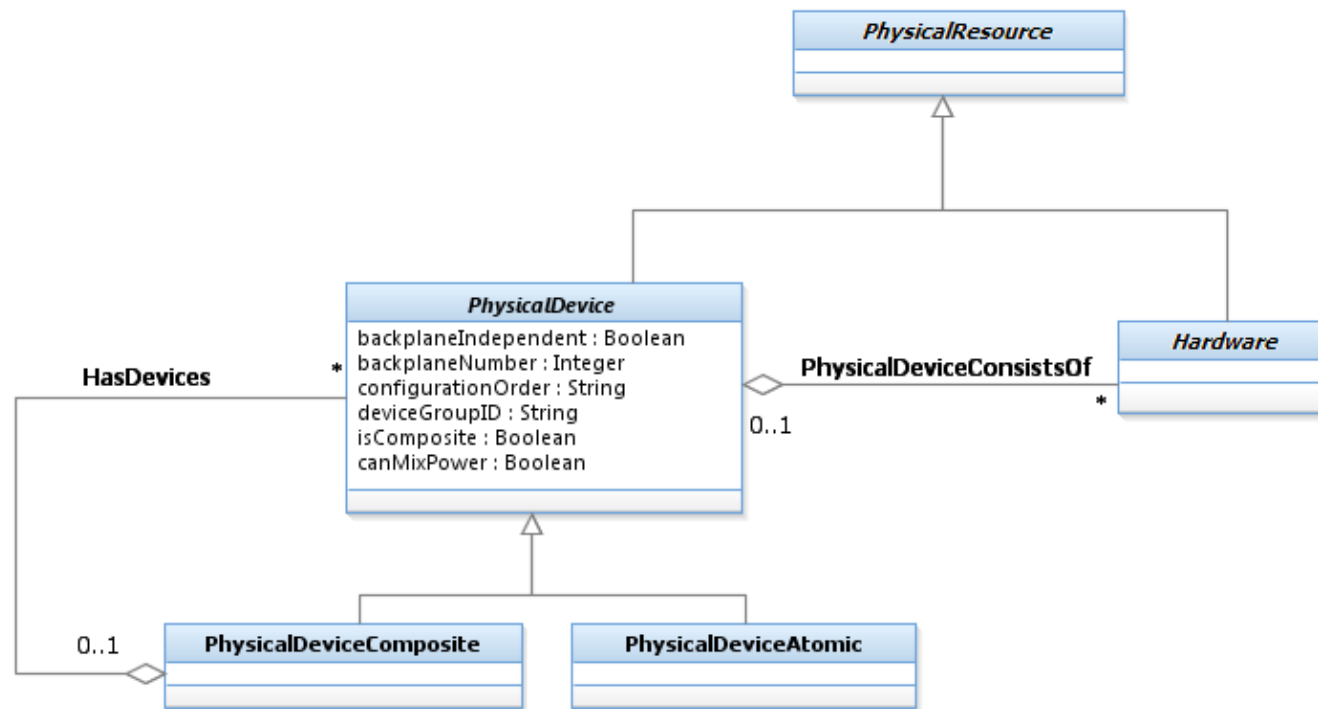


Figure PR.12 - Specifying the PhysicalDevice and Hardware Classes in More Detail

Hardware in More Detail

The various physical components that make up a PhysicalDevice have so far been grouped into a single class called Hardware. In looking for common attributes for these three very different types of entities, we find that dimensions and weight are all common attributes of these three entities. Therefore, we can add these attributes to the Hardware class (not to PhysicalDevice, since we can derive these attributes for a PhysicalDevice from the Hardware entities that make up a PhysicalDevice). Figure PR. shows these changes to the Hardware class.

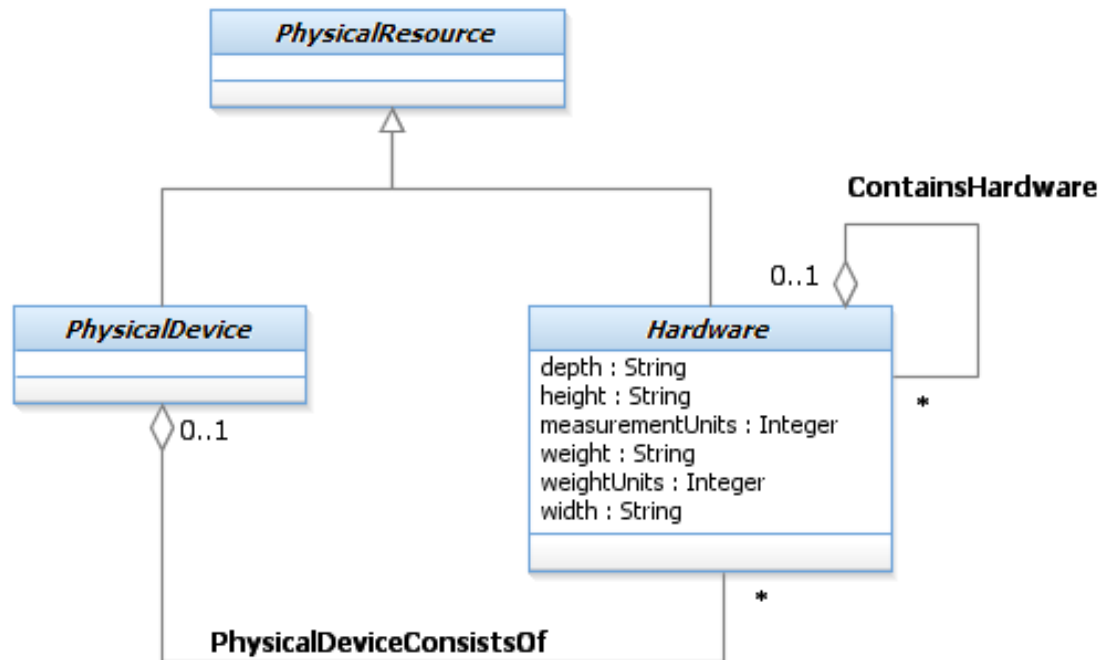


Figure PR.13 - Hardware in More Detail

The ContainsHardware aggregation defines the ability for a given Hardware object to contain other Hardware objects. This enables the different managed hardware components that make up a device to be explicitly defined.

PhysicalConnector

A PhysicalConnector is a concrete class that represents any type of hardware unit that is used to connect to other hardware units and transmit signals and/or power between them. The attributes of this entity are described in more detail in the Data Dictionary Section of this Addendum.

PhysicalConnector is a subclass of Hardware, as is shown in Figure PR. below.

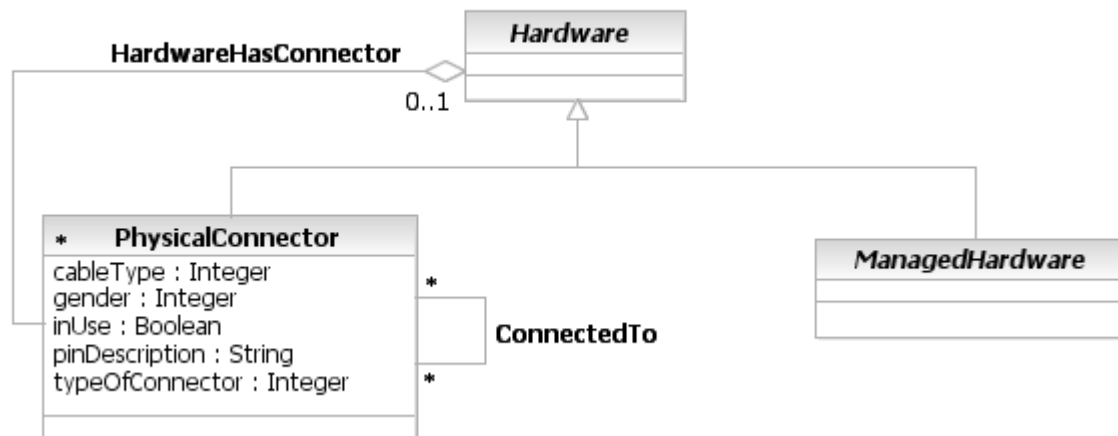


Figure PR.14 - Physical Connector

The cableType and gender attributes define the type of connector and the type of cable that is attached to this connector (if any). The pinDescription attribute is a free-form string describing the pin configuration and signal usage of a PhysicalConnector.

The ConnectedTo association indicates that two or more PhysicalConnectors are connected together.

Note that a PhysicalConnector is a sibling to the ManagedHardware class, which is discussed in the next section. Thus, a PhysicalConnector is a type of Hardware, but is not a type of ManagedHardware.

ManagedHardware

ManagedHardware is a subclass of Hardware, and is an abstract base class that adds additional semantics to the Hardware base class. These semantics are used to provide management information on the hardware. For example, attributes defined by this class can provide the administrative and operational state of the entity, and tell whether it has any alarms. Such attributes are physical in nature, and indicate physical things (e.g., a fiber cut). For most physical administrative and operational states, there is one or more corresponding logical administrative and operational states. ManagedHardware is shown in Figure PR. below.

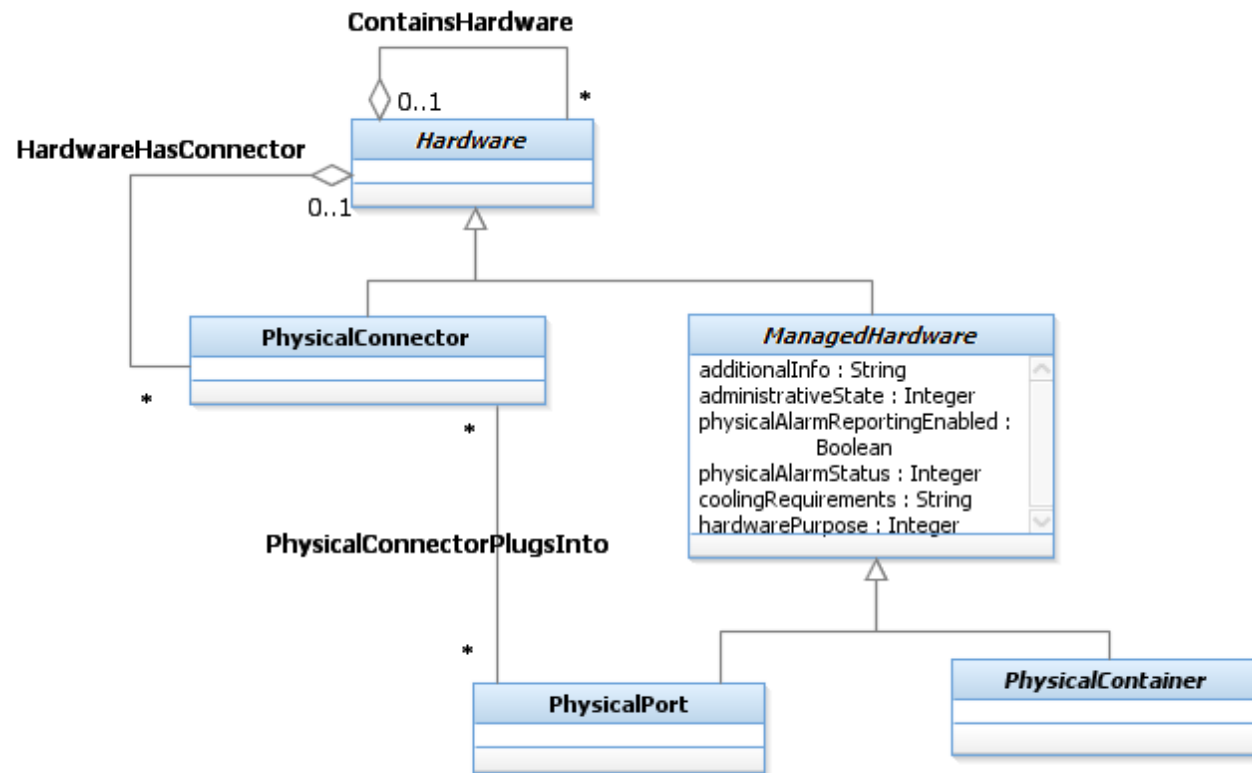


Figure PR.15 - ManagedHardware in More Detail



The main difference between ManagedHardware and Hardware is that ManagedHardware entities have more than just a physical presence – they also have physical semantics that describe how the entity is managed. This is why PhysicalConnector is not a type of ManagedHardware – a PhysicalConnector is managed by physically inserting and removing it.

So, what is meant by physically managing a hardware entity? This is best explained by briefly examining the three attributes of the ManagedHardware class.

The administrativeState attribute is an enumerated integer that describes the current physical state of the ManagedHardware. Examples of this include starting up, shutting down, and in test.

The physicalAlarmReportingEnabled attribute is a Boolean, and defines whether physical alarm reporting for this object instance is enabled or not. Note that some physical entities are not capable of reporting physical alarms, while some are. In most cases, there are corresponding logical alarms. The ManagementEntity class hierarchy (which is described in Addendum 5LR) defines logical alarms, and correlates them to physical alarms.

The physicalAlarmStatus attribute is an enumerated integer that indicates the occurrence of an abnormal physical condition relating to an object. This attribute may also function as a summary indicator of alarm conditions associated with a specific resource. This attribute expands on the standard ITU semantics and updates them to include eTOM concepts.

The ManagedHardware class has two important subclasses, called PhysicalPort and PhysicalContainer. These are described in the next two sections, respectively.

PhysicalPort

PhysicalPorts are an important type of object to keep track of. This is where communication begins and/or ends on a PhysicalDevice. PhysicalPorts also play a prominent role in topology and FCAPS applications, and enable Service Providers to determine what Customers are running what Services where in the network.

A PhysicalPort represents an actual or potential end point of a topological (physical) link, and corresponds directly to a physical port on a topology map. PhysicalPorts are always contained by another physical object - they can't exist by themselves. The two most common examples are PhysicalPorts on a Card and on a Chassis. These are represented using separate aggregations, and will be described in the sections that explain Card and Chassis. Figure PR. illustrates the concept of a PhysicalPort.

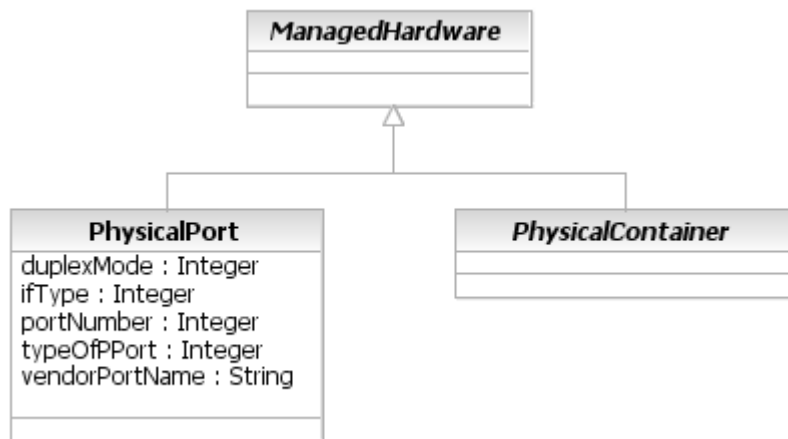


Figure PR.16 - PhysicalPort in More Detail

PhysicalContainer, as we will see in the next section, is the superclass for Equipment, EquipmentHolders, PhysicalComponents, and AuxiliaryComponents. Note that a PhysicalPort is not a type of PhysicalContainer. This is because a PhysicalPort contains *logical* components. This is described in more detail in Addendum 5LR.

The ifType attribute is an enumerated integer, and specifies the particular media type of the link. This is closely related to the typeOfPort attribute, which is an enumerated integer that defines the particular type of PhysicalPort this instance is (e.g., Ethernet vs. ATM). The portNumber attribute is the number of this physical port as defined by the application, and the vendorPortName is a critical attribute that contains the name of the port according to the vendor. This enables an enterprise-wide naming scheme to be correlated with vendor-specific names, which are necessary for configuring the port.



For example, an enterprise may need to assign either special strings, or a monotonically increasing number to distinguish all of their PhysicalPorts. Or, an enterprise may want to reuse PhysicalPort numbers, so that “port 1” on any device has special significance. Both of these schemes are very common, and must be accommodated in the model.

However, if an enterprise chooses to use their own naming scheme, then this makes it difficult, if not impossible, to program the device. This is because a (logical) DeviceInterface is contained within a PhysicalPort, as described in Addendum 5LR. Programming a device also means programming its device interfaces.

PhysicalContainer

A PhysicalContainer is an abstract class that is used to add additional semantics to the ManagedHardware class. Its attributes define whether a ManagedHardware object can be removed and/or replaced, and whether this action requires power to be removed or not when the action is performed. It is shown in Figure PR. below.

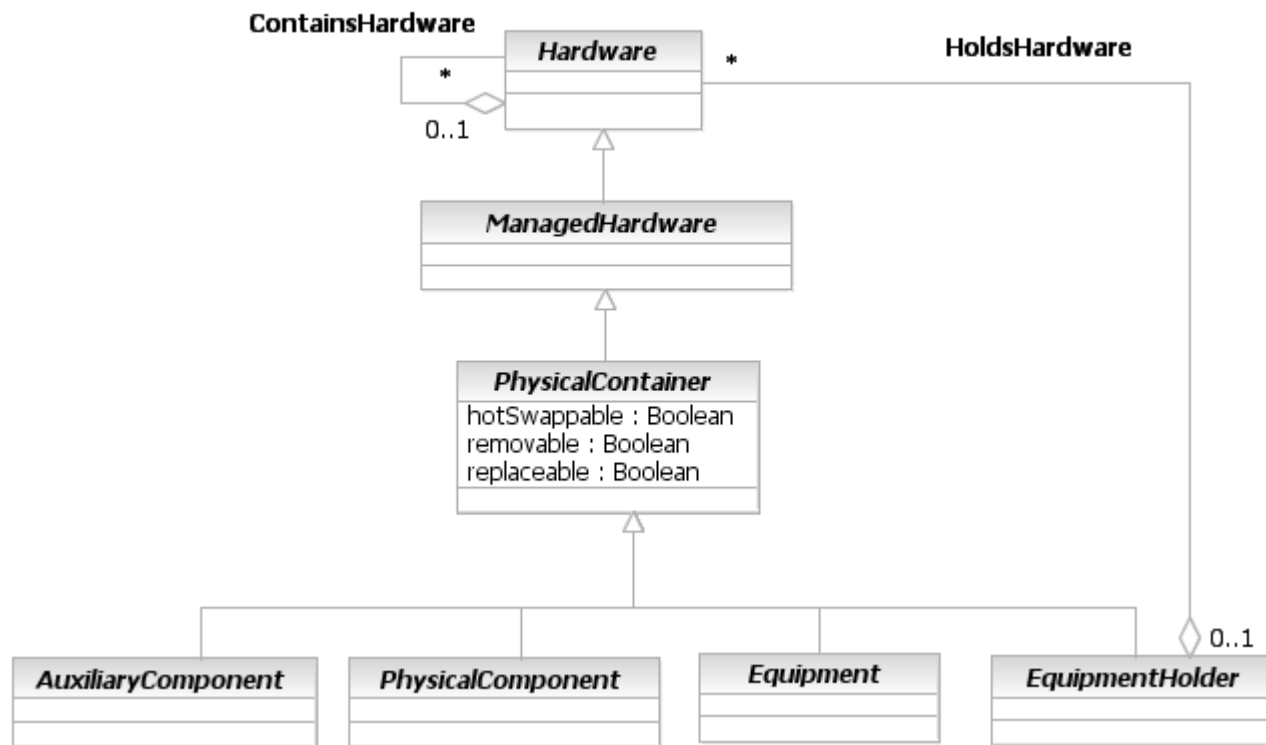


Figure PR.17 - PhysicalContainer in More Detail



The removable attribute is a Boolean that defines whether it is possible to insert and remove this object instance from the Equipment in which it is installed, without impairing the function or packaging of the Equipment. TRUE means that it is HotSwappable, and FALSE means that it is not.

The replaceable attribute is a Boolean that defines whether it is possible to replace this object instance with a physically different instance of the same type. For example, some types of device allow various Chips to be upgraded. TRUE means that it is HotSwappable, and FALSE means that it is not.

The hotSwappable attribute is a Boolean that defines whether it is possible to replace this object instance with a physically different, but equivalent, object instance while the containing Equipment has power applied to it. TRUE means that it is HotSwappable, and FALSE means that it is not. All HotSwappable PhysicalComponents are inherently Removable and Replaceable.

Thus, the purpose of the PhysicalContainer object is to add the removable, replaceable, and hotSwappable semantics to ManagedHardware entities. By doing this, it serves as the subclass for our four remaining types of objects: PhysicalComponents (such as chips and ASICs), AuxiliaryComponents (such as PowerSupplies), EquipmentHolders (such as Racks, Chassis and Slots) and Equipment (such as Cards). These will be described in subsequent subsections of this section.

PhysicalRoles in More Detail

We've talked about the different roles that a given device can have, and graphically represented this in Figure PR.. Most of the attributes (and all of the methods) defined for a PhysicalDevice are not appropriate for the business view – it is enough simply to be able to refer to a PhysicalDevice, such as a router, as a complete, atomic unit. Please refer to the Physical Resource System View Addendum for more information on non-business characteristics and behavior of this entity.

Physical roles, however, are appropriate for discussion in this Addendum. There are two types of physical roles that are called PhysicalDeviceRole and HardwareRole.

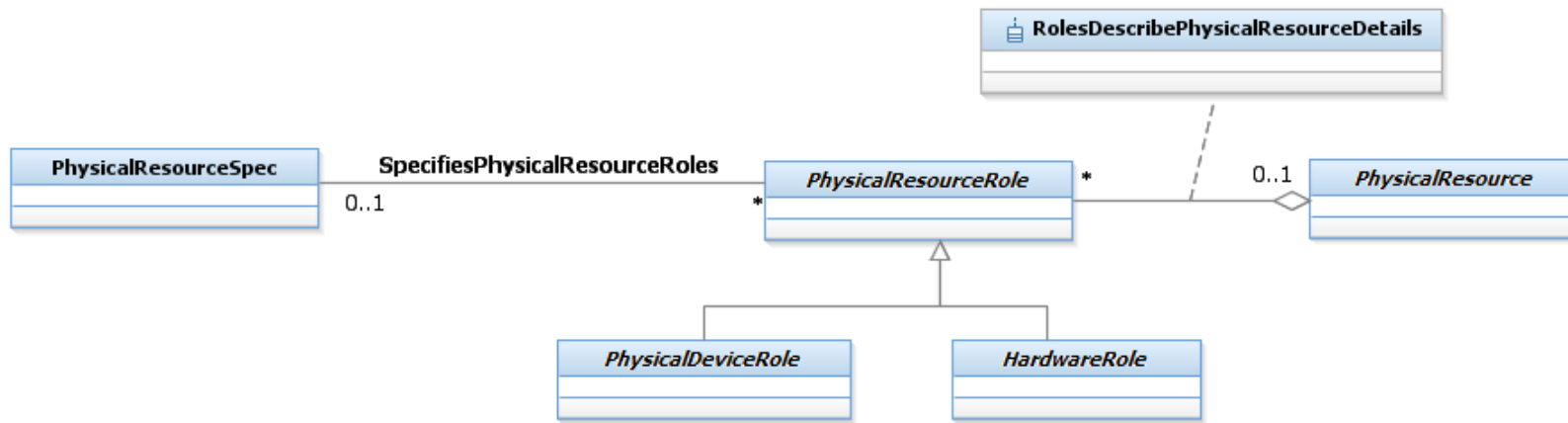


Figure PR.18 - PhysicalDeviceRole and HardwareRole

There is a profound difference between the **SpecifiesPhysicalResourceRoles** aggregation and the **RolesDescribePhysicalResource** aggregation. The former defines the set of **PhysicalResourceRoles** that a particular **PhysicalResource** must have (since it is defined by the specification for that **PhysicalResource**). This enables functionality to be specified for a particular physical component. The latter aggregation defines the set of physical roles that are used to describe an instance of a particular **PhysicalResource**.

In other words, the difference between the **SpecifiesPhysicalResourceRoles** aggregation and the **RolesDescribePhysicalResource** aggregation is that the former *defines* functionality of a **PhysicalResource** using **PhysicalResourceRoles**. In contrast, the **RolesDescribePhysicalResource** *describes* functionality using **PhysicalResourceRoles**.

The purpose of the **PhysicalDeviceRole** class is to define the different types of physical roles that a **PhysicalDevice** can take on. This enables us to correlate different logical roles to specific physical devices. For example, there are different logical characteristics that



differentiate a CustomerPremise router from a ProviderEdge router, even though they are both “edge” routers. Depending on the type of connection, different PhysicalDevices are often needed to support these logical functions. Figure PR. shows some example subclasses of PhysicalDeviceRole. The PhysicalRouterRole and its subclasses are explained in more detail in the MPLS VPN Addendum.

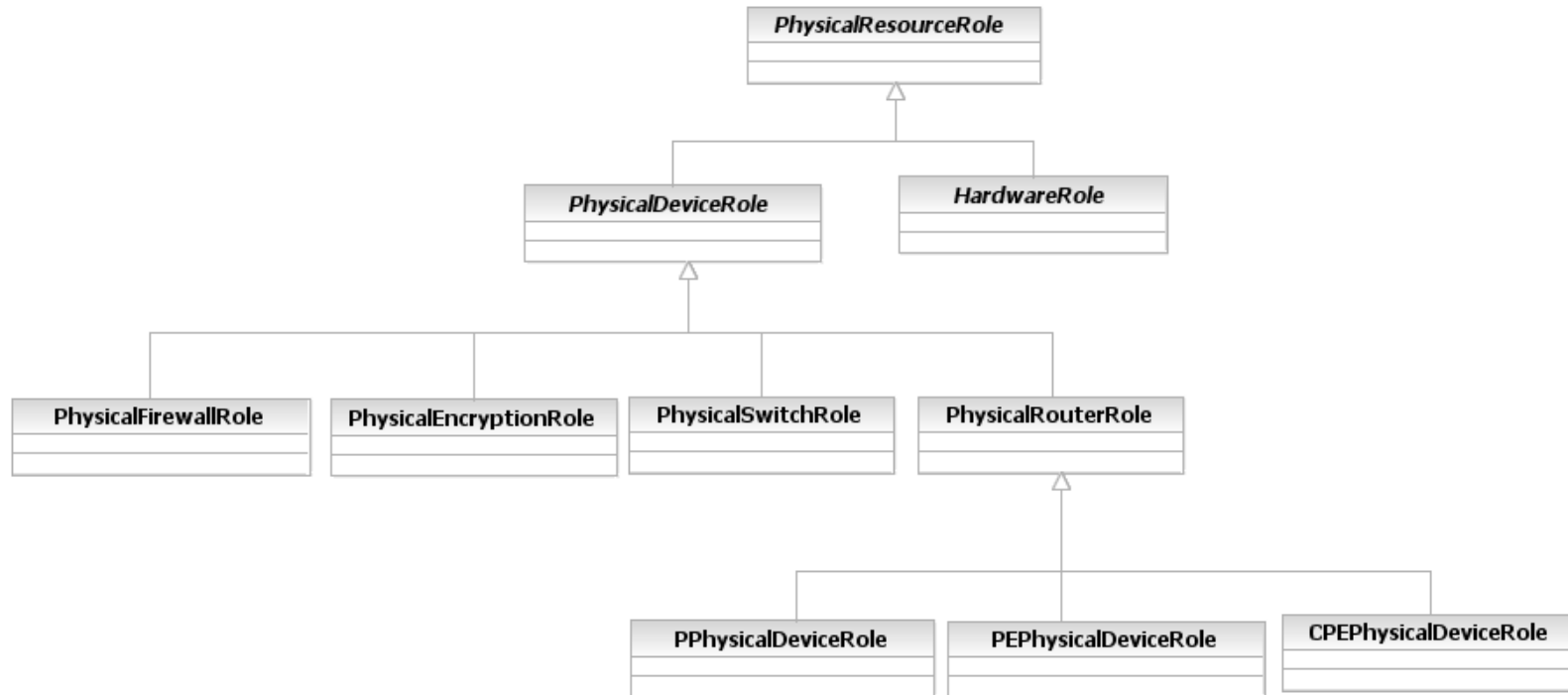


Figure PR.19 - Examples of PhysicalDeviceRoles

EquipmentHolder in More Detail

EquipmentHolder represents a specific category of physical objects. This means that it has its own hierarchy of objects. Specifically, we've already differentiated between Slots and Chassis (which are both subclasses of EquipmentHolder). The M.3100 definition of this class describes it as representing physical objects that are both manageable as well as able to host, hold, or contain other physical objects. Examples of physical objects that can be represented by instances of this object class are Racks, Chassis, Shelves, Cards, and Slots. (Note that M.3100 does not define all of these different types of EquipmentHolder.)

Some devices are built in such a way as to enable other devices to be mounted inside them. This can be modeled using the composite pattern, which divides an entity into two subclasses. One subclass is used to model stand-alone objects, and the other subclass is used to model objects that can contain more objects of that type. We can apply this pattern to the EquipmentHolder class to obtain the following:

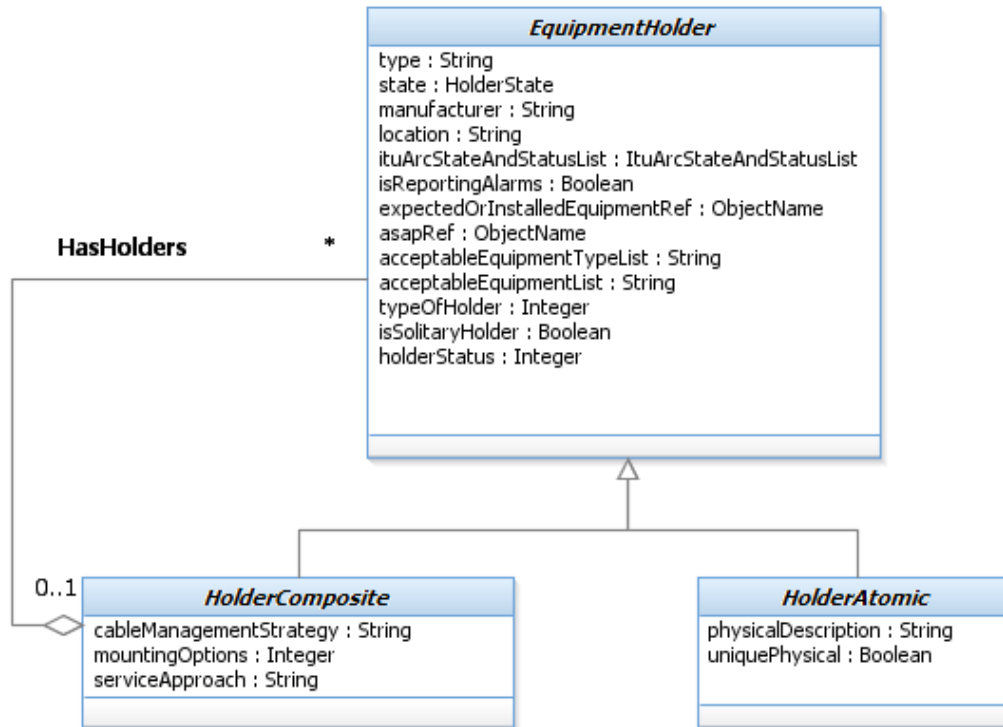


Figure PR.20 - Specifying Different Types of EquipmentHolders



The acceptableEquipmentList attribute is an array of strings, based on M.3100, which identifies the types of equipment objects that can be supported by this object. This attribute is defined for backwards compatibility with M.3100.

The typeOfHolder attribute is an enumerated integer that identifies the type of the Holder that this object instance is. It is based on M.3100 but includes additional values.

The holderStatus attribute indicates the status of the EquipmentHolder (e.g., is it installed).

The above figure has now formalized what we knew all along – there are really two different types of EquipmentHolders. One type, designated by the AtomicHolder class, is meant to be used in a stand-alone way. This doesn't mean that a Slot exists by itself; rather, the Slot cannot be used to form another type of EquipmentHolder. Contrast this to either a Rack or a Chassis. Both of these are subclasses of the CompositeHolder class, because both of them can be used to form more complex types of EquipmentHolders. For example, a Chassis can contain either another Chassis or a Slot, and a Rack can contain either a set of Chassis or a "sub-Rack".

HolderAtomic

The HolderAtomic class represents atomic holders of Equipment that are individually manageable and do NOT form composite, or nested, Equipment Holders. Each HolderAtomic object can be a Field Replaceable Unit.

The physicalDescription attribute is a free-form string that defines the physically unique characteristics of this holder. The uniquePhysical and uniqueLogical attributes are Boolean attributes that signify that this holder is physically or logically different from other holders.

A model of the HolderAtomic class is shown in Figure PR.21 below.

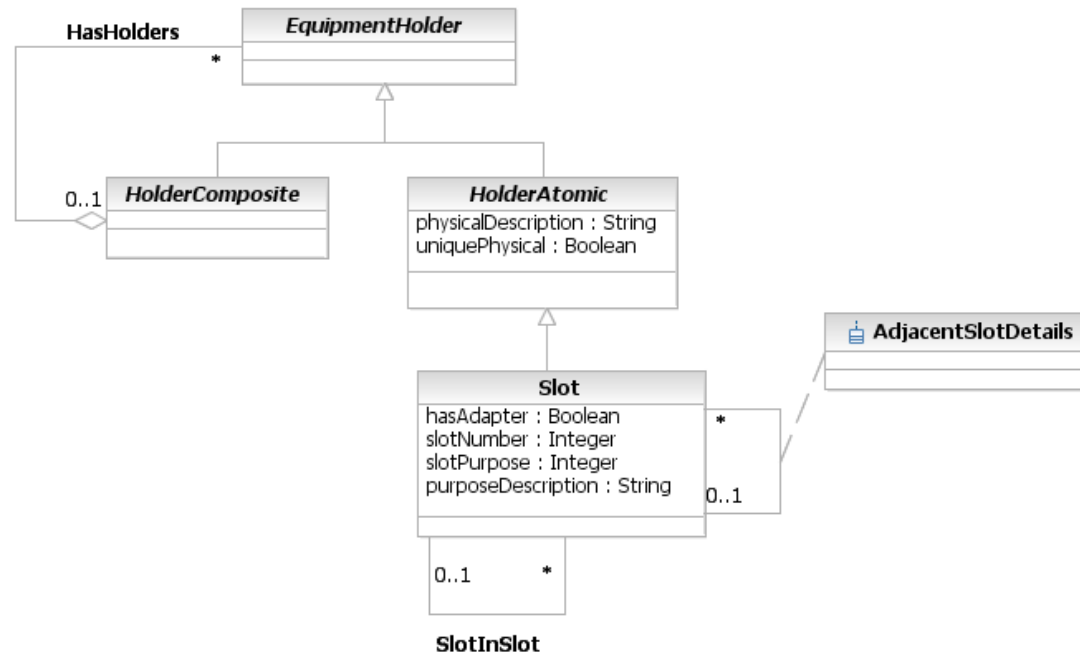


Figure PR.21 - HolderAtomic in More Detail

A Slot is a concrete class that has two main purposes. One is to model the ability of a hosting board to accept a daughter card to add or complete the base functionality of the hosting board. The second is to represent the different expansion slots supported by a Chassis.

All Slots have a unique number – this is represented by the *slotNumber* attribute. The *slotPurpose* is an enumerated integer that defines the purpose of this Slot. This enables Slots to be easily identified according to their functional purpose.



This leaves the `hasAdapter` attribute. This attribute is a Boolean attribute that, if TRUE, indicates that this slot has an adapter installed that enables it to accept other types of cards (e.g., fitting an adapter on two Slots enable them to accept a Card that otherwise could not be accommodated).

Adapters are very important, as they enable an organization to keep its investment in expensive line Cards. This is represented in more detail using the `SlotInSlot` association. This association represents the ability of a special adapter to extend the existing Slot structure to enable otherwise incompatible Cards to be plugged into an `EquipmentHolder`. The adapter effectively creates a new Slot and can be thought of (conceptually) as a Slot in a Slot.

The `AdjacentSlots` association describes the layout of Slots in an `EquipmentHolder`. This association class includes attributes that are used to provide general layout information describing the Slots in the `EquipmentHolder`. An important use of this association is to signal when two Slots are so close to each other that if one of these Slots is populated by an adapter Card, the other Slot must be left empty.

HolderComposite

The `HolderComposite` class represents `EquipmentHolders` that are made up of other `EquipmentHolders` (i.e., instances of this class as well as the `HolderAtomic` base class). This provides the semantics of collecting a set of components, each of which is individually manageable, and being able to manage the set of objects as a whole. This containment is modeled using the `HasHolders` aggregation.

Each `HolderComposite` element can be a FRU.

A characteristic of this class is that its subclasses are physical objects that have complex cabling and mounting options. This class is meant to differentiate complex holders, like a Chassis, from simple holders, like a Slot. Thus, the `cableManagementStrategy` attribute is a free-form string that contains information on how the various cables contained in the Chassis, Rack, or other type of `HolderComposite` object are connected and bundled.

The `mountingOptions` attribute defines how Equipment in this entity is mounted (e.g., rack-mounted, free-standing, or enclosed in another Chassis). Similarly `serviceApproach` defines how the entity is serviced (e.g., from the top or front), whether it has sliding trays or removable sides, and/or whether the Frame is moveable (e.g., it has rollers).

Figure PR.22 shows the prominent subclasses of a `HolderComposite` entity.

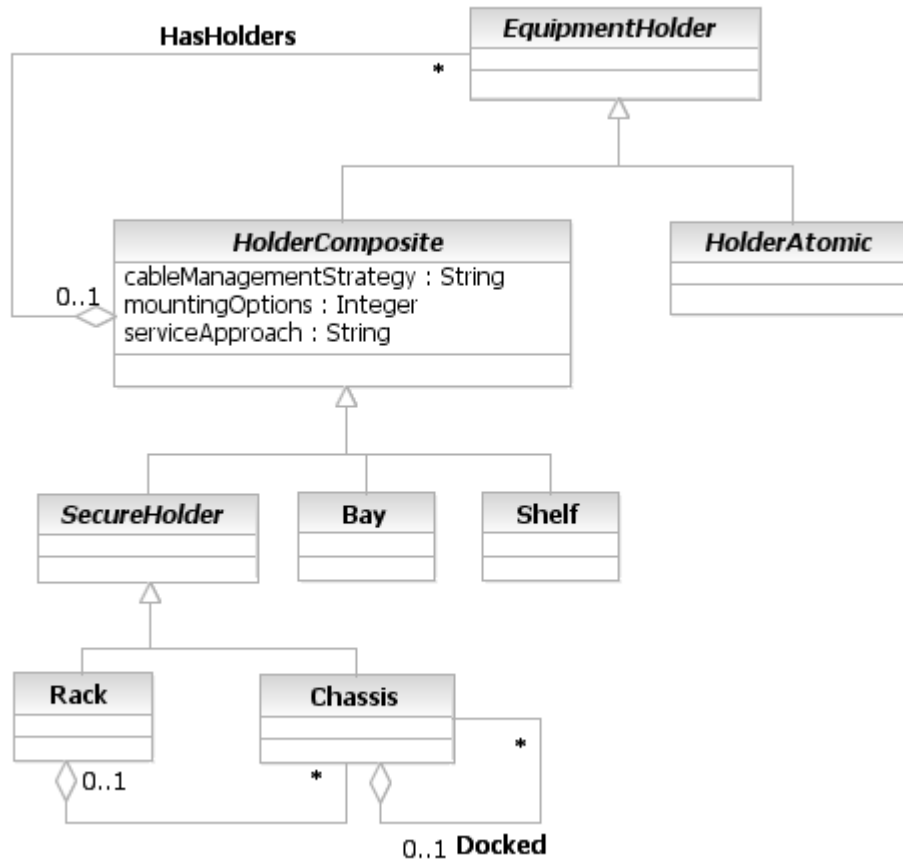


Figure PR.22 - HolderComposite in More Detail

The **SecureHolder** is a type of **HolderComposite** that serves as the parent for the **Rack** and **Chassis** classes. This class generalizes common properties that apply to **Racks** and **Chassis**. This class defines various types of locks and alarms, and adds the semantics of being able to indicate if a breach of this entity has been made.



A Bay is a type of EquipmentHolder that is usually found in optical devices. It has a logical identifier, usually a component of port AIDs in TL1 based switches. A Bay is usually built as a pre-wired hardware assembly that contains a set of shelves and accompanying ancillary equipment.

A Shelf is a type of EquipmentHolder that is usually found in optical devices. It has a logical identifier that is often relative to the Bay that contains the Shelf (i.e., the unique identifier for a Shelf is often a concatenation of the network element identifier, the Bay identifier, and the Shelf identifier). Often, a Shelf is used to contain not just pluggable components (e.g., Cards, Power Supplies, etc.) but also cabling (e.g., both fiber and wire), with optional connections to external fuse, alarm, and other types of panels.

A Rack is a type of SecureHolder that represents an enclosure in which EquipmentHolders, such as Chassis, are placed. Typically a Rack is nothing more than the enclosure, and all the functioning componentry is packaged in the Chassis. The Rack typically serves as the "master enclosure" for Chassis, Shelves and Bays. In addition, Racks can have multiple instances of multiple Devices mounted in them.

A Chassis is a type of SecureHolder that encloses other ManagedPhysicalEntities and provides a definable functionality in its own right, such as a desktop or a network device (e.g., a router or a switch).

Adapter and Holder Roles

While this is a workable model, it still isn't capable of modeling some important cases. For example, think of a Card that has a physical connector on it that accepts either another (daughter) Card or even a Chip (e.g., Memory, or a routing or encryption ASIC). The model shown in Figure PR. above mandates the use of a Holder. However, in the above example, there is no explicit EquipmentHolder – we are forced to either make a physical connector a holder, or insert an artificial holder. Both are incorrect.

Recall the HardwareRole from Figure PR. above. The purpose of the PhysicalHardwareRole class is to represent the different types of roles that various types of Hardware components can take on. For this first iteration, two specialized roles have been defined: HolderRole and AdapterRole. Their purpose is to cover situations where an Equipment also acts like an EquipmentHolder. Without these classes, such common situations are very difficult to model. We can use the PhysicalConnector class to connect appropriate Hardware entities together, which transmit signals and/or power between them. Thus, our previous model is modified as follows:

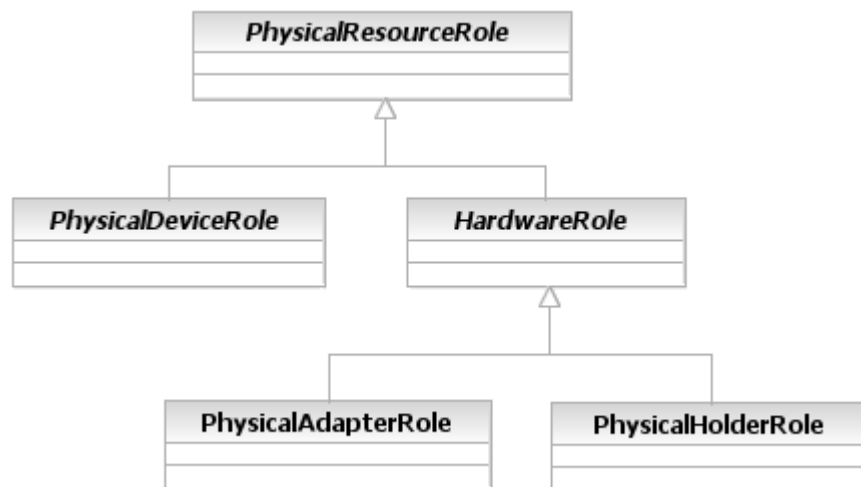


Figure PR.23 - AdapterRoles and HolderRoles



The HolderRole enables different types of PhysicalResources, such as a Card, to take on the role of holding other Equipment. This addresses the problem that we described above, where a Card took on the additional role of holding another Card. This enables a single managed entity, such as a NetworkCard, to have multiple roles - it can function as a Route Processor as well as a Holder of Equipment.

The Adapter role enables a piece of Hardware to adapt its use. For example, sometimes Cards and Chassis evolve along separate paths, causing future versions of one to no longer be physically compatible with present and/or future versions of the other. The solution to this is to use an intermediate piece of Hardware, called an Adapter, to extend the existing physical structure of an EquipmentHolder to enable otherwise incompatible Equipment to be plugged into an EquipmentHolder. The Adapter conceptually creates a new type of EquipmentHolder that fits into the existing EquipmentHolder. This enables Cards that would otherwise be physically and/or electrically incompatible with the existing EquipmentHolder to be supported, by interfacing to the old EquipmentHolder via the new Adapter.

The additional semantics provided by the Adapter can be captured as a second type of PhysicalResourceRole. This represents the common practice of vendors providing special adapters that enable old Equipment or EquipmentHolders to be used with new EquipmentHolders or Equipment, respectively.

Equipment in More Detail

Equipment represents a specific category of physical objects. This means that it has its own hierarchy of objects. For example, we've already identified Cards as a type of Equipment, and differentiated between Equipment and AuxiliaryComponents (which are both subclasses of Hardware).

The M.3100 specification defines Equipment as a class of managed objects that represents physical components of a managed element, including replaceable components. An instance of this object class is present in a single geographic location. Equipment may be nested within another Equipment, thereby creating a containment relationship. Equipment, and other sibling classes, are shown in Figure PR. below.

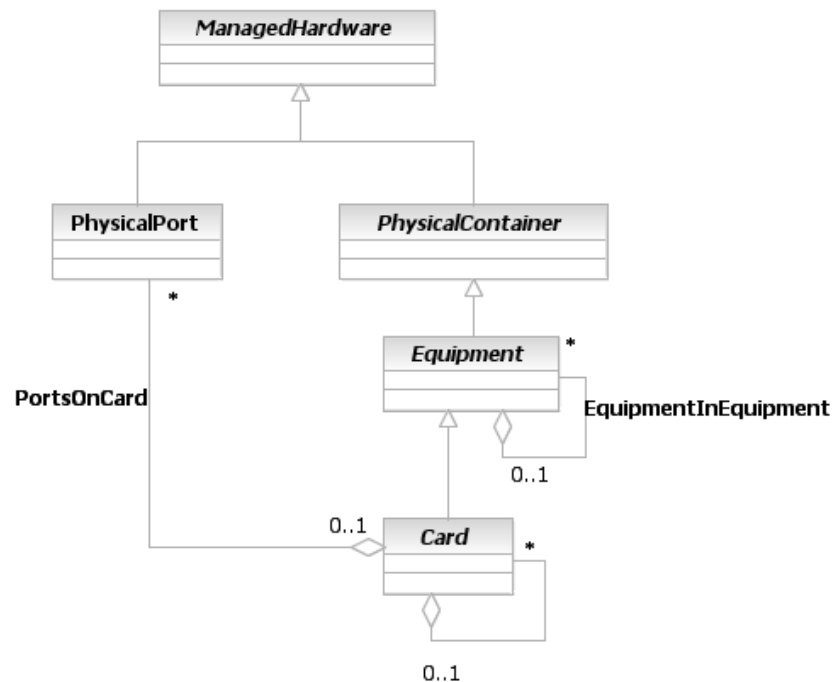


Figure PR.24 - Equipment in More Detail

PhysicalPorts can be placed on either a Card or a Chassis. Thus, we end up with:

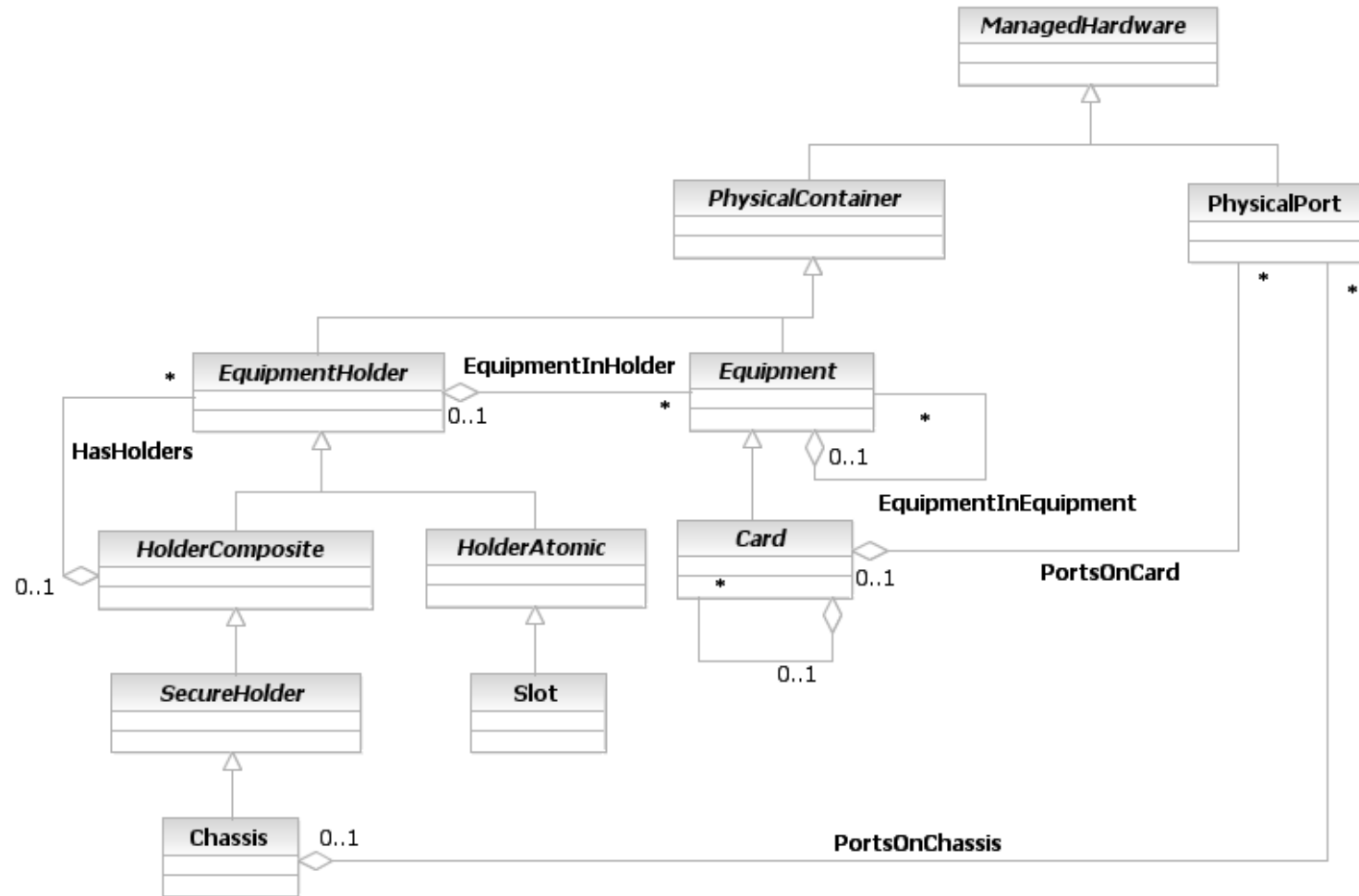


Figure PR.25 - Relating PhysicalPorts to Cards and Chassis

Note that the EquipmentInHolder relationship enables a LineCard to be attached to a Slot. We made two distinct relationships to model the containment of Ports (PortsOnCard and PortsOnChassis) because the common superclass of LineCard and Chassis is Hardware. If we had run a relationship between Hardware and PhysicalPort, then PhysicalPorts could contain PhysicalPorts, which is not correct. In addition, this

differentiation will help us in the System view, since the semantics of a PhysicalPort on a Chassis are different than those of a PhysicalPort on a Card.

Types of Cards

Figure PR. below illustrates the superclasses for defining different types of Cards.

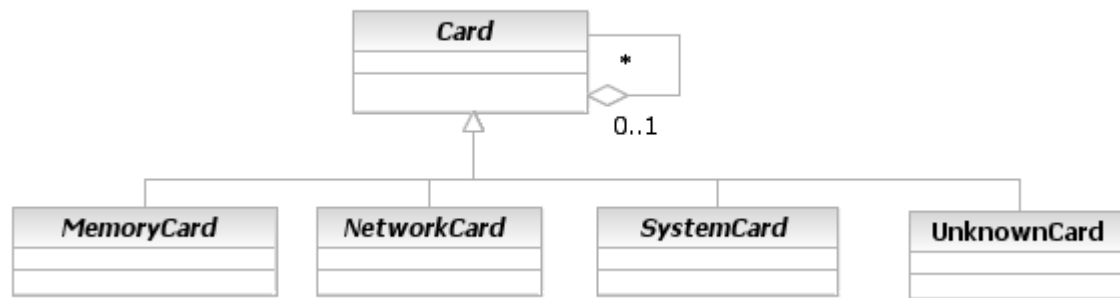


Figure PR.26 - Types of Cards

MemoryCards are dedicated to providing additional memory for use by other components of a Resource. NetworkCards are dedicated to providing networking functions, such as routing and forwarding. SystemCards are dedicated to providing system functions (e.g., controller cards). Finally, the UnknownCard object is used as a generic placeholder to represent Cards that are known to exist but are not yet identifiable via Software means.

AuxiliaryComponents

Figure PR. showed a third important subclass of PhysicalContainer, called AuxiliaryComponent. As stated previously, an AuxiliaryComponent is an abstract base class that represents managed entities, such as power supplies, fans and cables, which are required for the proper operation of the Device but have a primary function that is different than the primary end-user function(s) of the Device. Two exemplary subclasses of AuxiliaryComponent are shown in below.

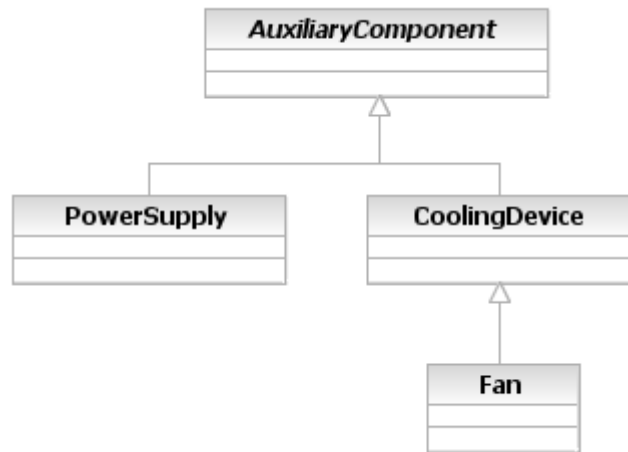


Figure PR.27 - Exemplary Subclasses of AuxiliaryComponent

The difference between AuxiliaryComponents and other subclasses of Equipment are whether or not the physical object performs a function intrinsic to the main function of the Device. For example, a PowerSupply is an AuxiliaryComponent, because even though it is needed for the proper operation of the Router, it does not directly help in routing and forwarding packets. A LineCard (that provides routing functionality) is a subclass of Equipment because its purpose is to route and forward packets.

PhysicalComponents

A PhysicalComponent was also shown in Figure PR.. A PhysicalComponent is a managed entity that can reside either in an Equipment or an EquipmentHolder object, but cannot be used as a stand-alone object. From a management point-of-view, this object either cannot or does not need to be split into its constituent parts. Any piece of hardware that is not a PhysicalLink, PhysicalConnector, Equipment, or EquipmentHolder, is a subclass of this class. Some important subclasses of PhysicalComponent are shown below in Figure PR..

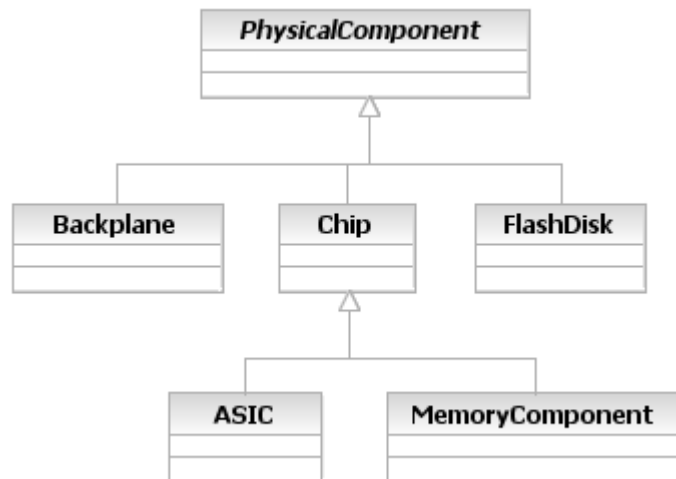


Figure PR.28 - Example Subclasses of PhysicalComponent

The example subclasses are exemplified by the following description of the Chip class. A Chip is a concrete class that models different types of Chips that are either directly configurable by the end-user (e.g., a programmable device) and/or represent FRUs (e.g., a special ASIC that can be upgraded by replacing it with a new version of that same ASIC).

For complete definitions of each of these objects, please see the data dictionary section of this document.



1.3. Basic Business Entities That Interact with Physical Resources

This section will outline some of the business entities that interact with business entities defined in the Physical Resource model. This section will of course be added to in every phase of the SID as appropriate.

Party and PartyRole

This has already been referenced briefly above. To recap, Party and PartyRole are needed for a number of important reasons, including:

- Identification of an owner of the Device or Hardware
- Identification of the manufacturer of the Device and possibly the manufacturer of some or all of its components
- Identification of who is responsible to administer the Device or Hardware
- Identification of who installed the device
- Identification of what management domain the Device or Hardware is in
- Identification of who in a particular management domain can administer the Resource
- Identification of who last changed the device

The above are exemplary, and identify some of the links between these classes and the physical resource model. The above is not meant to imply that these are the only links between these models.

It is important to realize that the owner of a Device or Hardware is not the same as the person or group that is responsible for administering the Device or Hardware.

While the eTOM talks about the need to administer different types of managed objects, it doesn't identify the concepts of owner and administrator. These are needed to support the inventory and other use cases we introduced at the start of this document. Every effort will be made to take this data and feed it back to the eTOM team.

Referring to the SID Party model (Addendum 1P), there is an 0..n to 0..n association (implemented as an association class) with a PartyInvolvement attribute that define what responsibility the PartyRole (e.g., customer, provider, employee, management domain) plays. Example responsibilities are owner, lessee, lessor, and administrator. Note that this involvement does not have to be established via an Interaction; there can be a direct relationship that defines this involvement. It is important that the SID not dictate this, so that applications are free to implement the precise semantics of this relationship according to their own needs. This is shown in Figure PR.29.

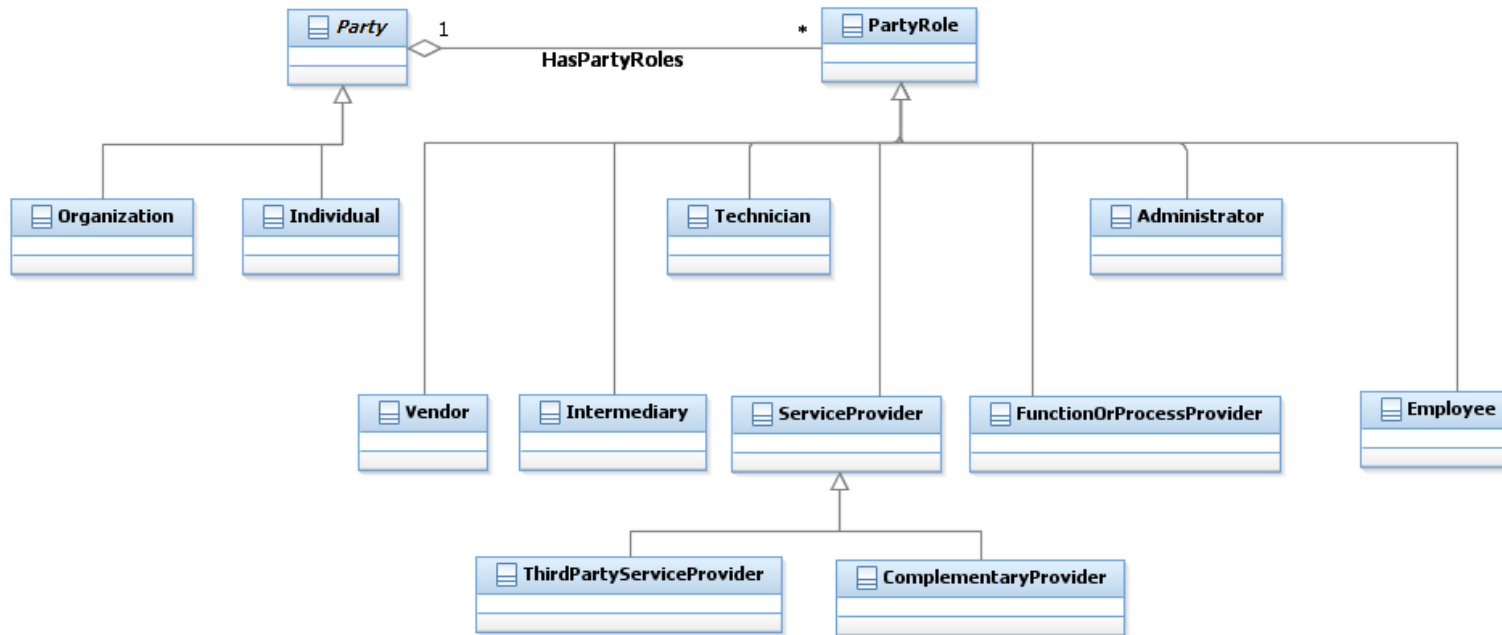


Figure PR.29 - Overview of Party and PartyRole

Given this background, we can define additional relationships needed to represent administration and ownership of PhysicalResources.



Briefly, the owner of the Resource is the person or group that is responsible (e.g., organizational, financial, and so forth) for the Resource. This is a different concept than the person or group that administers the Resource. From a business perspective, the owner has to either appoint or give permission to the administrator to administer the Resource that is owned. This is shown in Figure PR.30 - Representing Administering and Owning a Resource below.

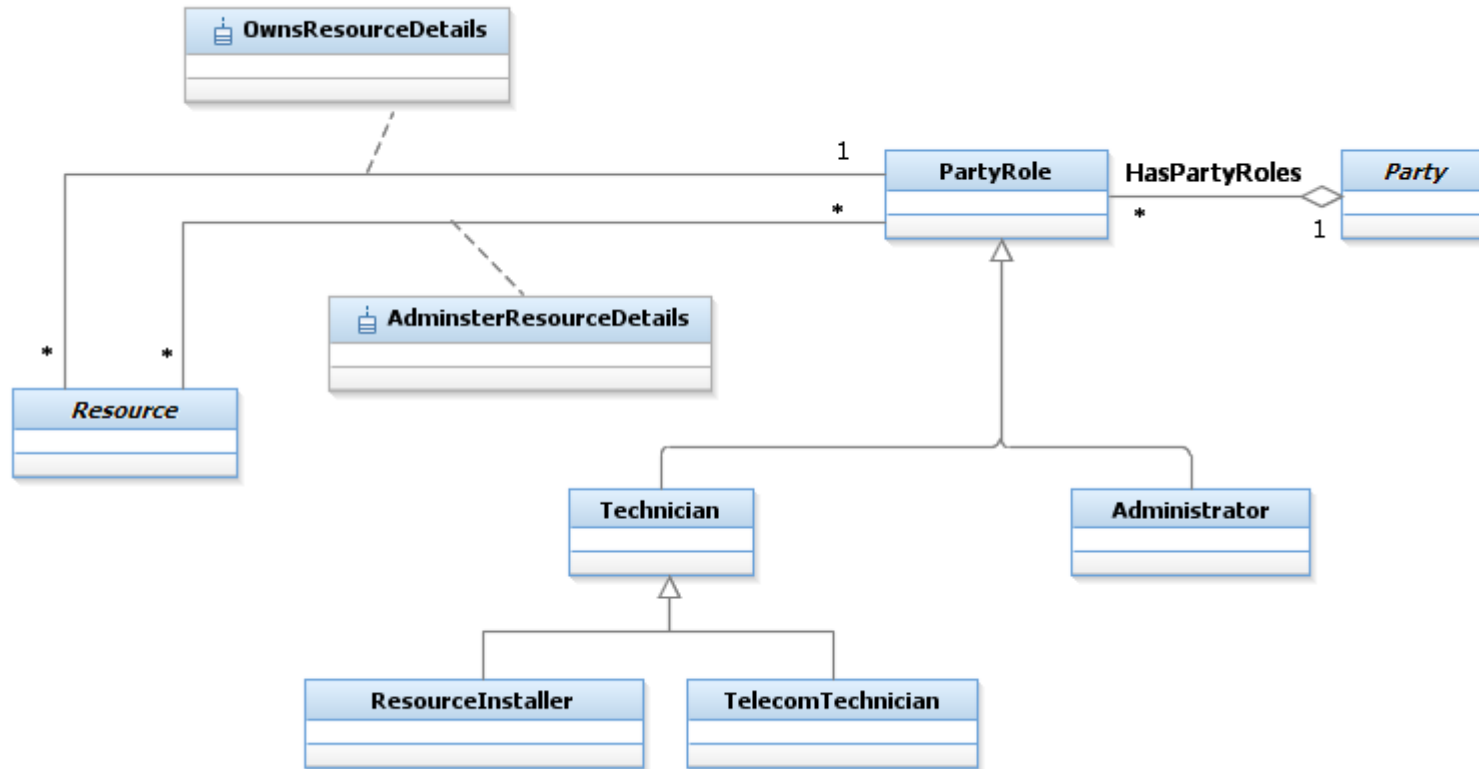


Figure PR.30 - Representing Administering and Owning a Resource

The two associations, called **OwnsResource** and **AdministersResource**, serve these roles. Note that the **OwnsResource** aggregation defines the set of **Resources** (physical and/or logical) that a particular **Party**, playing the role of **Administrator**, can administer (through the **AdministersResource** association). The owner of the **Resource** is the person or group that is responsible (e.g., organizational, financial, and so forth) for the **Resource**.



In contrast, the AdministersResource association defines the set of Resources (physical and/or logical) that a particular Party, playing the role of Administrator. From a business perspective, the owner has to either appoint or give permission to the administrator to administer the Resource that is owned.

The third item, ManagementDomain, is fundamental to how network management is done. Addendum 1-POL defines a ManagementDomain as follows: A ManagementDomain class represents a special grouping of ManagedEntities that has two important properties. First, it is used to partition managed objects into a meaningful logical grouping. One important use of such a grouping is to provide a means to define which EMS (as well as which NMS) manages, monitors, etc. which set of devices. It also provides a means to show how management functions are distributed and scaled. Second, it defines a common administrative domain that is used to administer the managed objects that it contains. This implies that all of the managed objects contained in this ManagementDomain are administered similarly - either by the same user, group of users or policy.

ManagementDomains often have a direct correlation with DeviceRoles. For example, in an MPLS VPN, there is a fundamental difference between routers that are operating as customer premise, provider edge, and provider core routers. (For those not familiar with MPLS VPNs, a customer premise device provides customer access to the Service Provider network over a data link to one or more provider edge routers; provider edge routers function as the ingress and egress routers of the Service Provider network; provider core routers provide connectivity between the provider edge routers. An example of a high-level MPLS VPN model is provided in part of Addendum 4). This can be done by associating a type of PartyRole (representing a ManagementDomain) with a type of DeviceRole. Note that in this use case, there are specific hardware as well as software requirements of the different types of routers playing the CustomerPremise, ProviderEdge, and ProviderCore roles. This is where the concepts of PhysicalDeviceRole and LogicalDeviceRole will pay dividends in the overall model.

Thus, ManagementDomains are intelligent containers of managed entities, where different container represent different domains of control that reflect one or more appropriate management structures, such as organizational structure. PhysicalResources within a particular domain are administered by a particular person or group of people. While it is easy enough to define a relationship between ManagementDomain and PartyRole, the management of PhysicalResources in ManagementDomains by Party and PartyRole entities should be represented and controlled using policies. Thus, this topic will be deferred to the Policy Addendum.

Location

Location is used in two different ways and has two very different semantics:

- The *place* of a Device (or perhaps an Equipment or EquipmentHolder)
- The *position* of a physical component

These are in reality two different concepts, though people tend to combine their meanings into one.

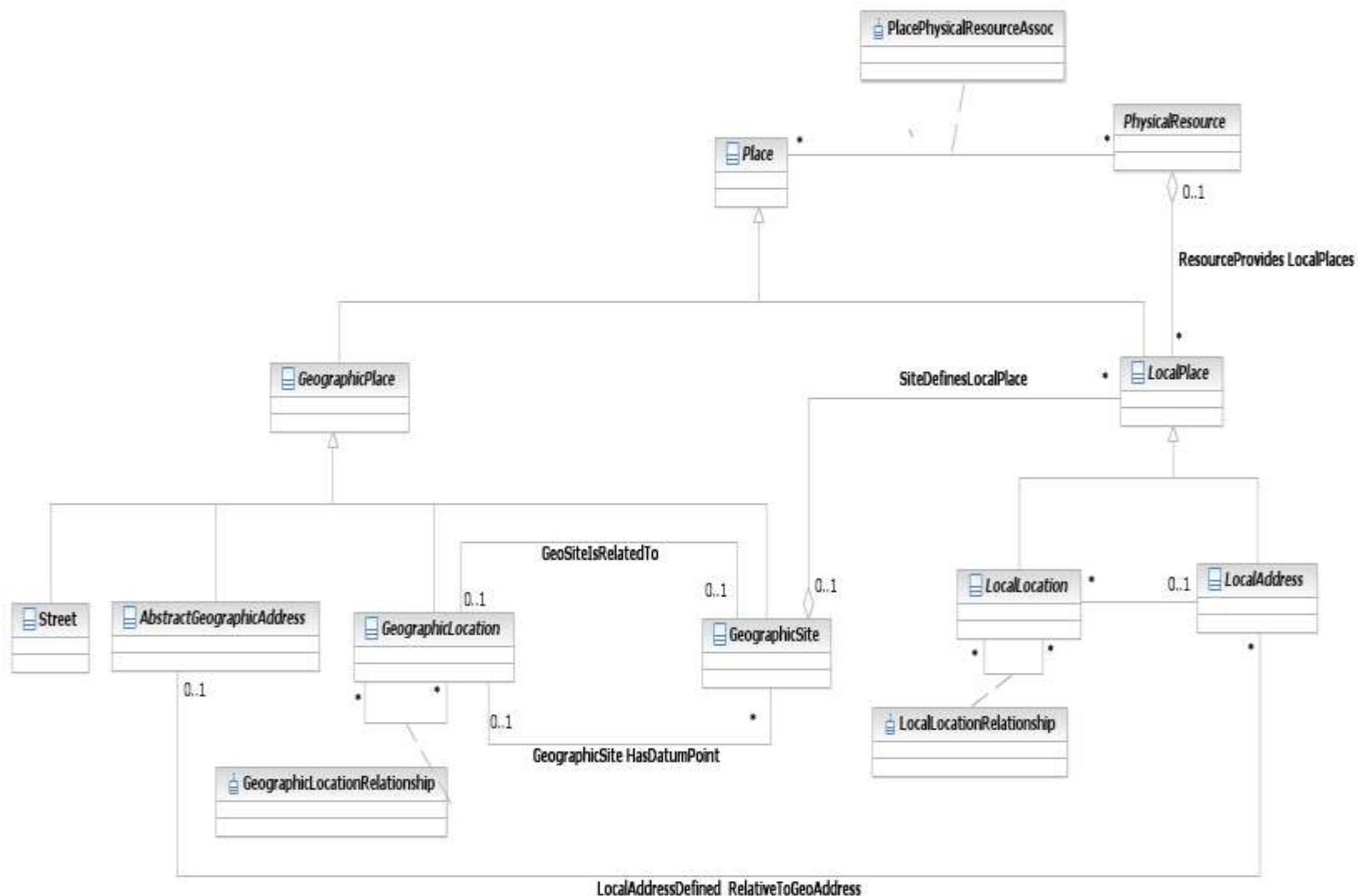


Figure PR.31 - Representing the Physical Location of Physical Objects

The PlacePhysicalResourceAssociation association defines the location(s) that a particular PhysicalResource can be found at. Note that there can be multiple locations assigned for a particular PhysicalResource. For example, a router can be installed in a POSITION in a Rack,



which is in a particular PLACE (e.g., a Room in a Floor), which is in a particular STRUCTURE (e.g., a building) at a particular ADDRESS (e.g., the postal address that the main building is located at). The PhysicalLocationDetails association class is used to help disambiguate these overlapping locations.

Figure PR 29 shows that “Place” is broken into two levels of granularity, represented by the two subclasses of Place.

- LocalPlace.
- GeographicPlace

Capacity and Redundancy

In theory, any physical entities can have one or both of these. However, in practice, Capacity and Redundancy are restricted in this model to apply only to entities that have Roles. This enables us to maximize the ability to carry out performance corrections. When Devices are identified that are negatively impacting management items of interest, such as network performance and Service quality, the close association between Roles and Capacity / Redundancy will allow Capacity and Redundancy associated with these Devices to be easily addressed.

Capacity implies a range of values – from a minimum acceptable level to a maximum permitted level – that a physical object can hold, store or accommodate. For example, a Device may require a minimum of 128MB of RAM to operate correctly, prefer 256MB, and accommodate up to 512MB.

Often, manufacturers make Devices using the same overall Chassis or frame and vary the number and type of cards that can be installed. Different cards have different power, cooling, memory and other requirements. Therefore, it is often economically easier to build one master frame and enable multiple variations of certain components, such as memory, to have a minimum and maximum capacity.

Redundancy connotes additional entities of identical capabilities that can be used in case the primary entity fails or is no longer able to provide its full capabilities. For example, Devices often have multiple power supplies to ensure that if one fails, the Device can use a backup power supply and continue to operate. Again, most manufacturers plan for physically housing one or more redundant components for essential physical hardware as part of the overall physical device.

It is recommended that the concepts of capacity and redundancy be further developed in SID phase IV. They will both require extensions to the SID that are not currently in this release, as well as require a deeper understanding of both the business and system views of physical objects.

Product

The SID Product business entities are defined in Addendum 3. The most important of these (from the point-of-view of the Physical Resource model) are the Product, ProductOffering, and ProductSpecification entities.

A ProductSpecification defines the requirements of physical and logical objects that collectively are accessed via a Product. Products are derived from ProductSpecifications, and Products can be wholly or partly physical objects. Products may define the use of various Physical Resources, as shown in the following excerpt from Addendum 3:

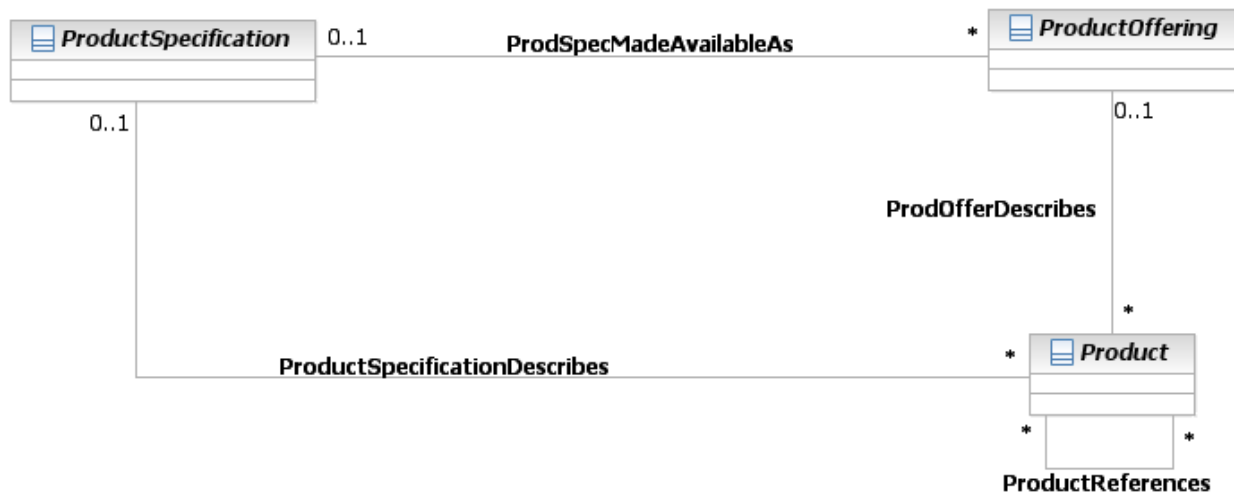


Figure PR.32 - Simplified Product Model

The Product model has two obvious attachments to Resources in general and PhysicalResources in particular. These are at the ProductSpecification and Product classes.

A ProductSpecification is a template for defining various ProductOfferings. Therefore, it is natural for a ProductSpecification to specify ServiceSpecifications as well as ResourceSpecifications. This is shown in PR.33 below.

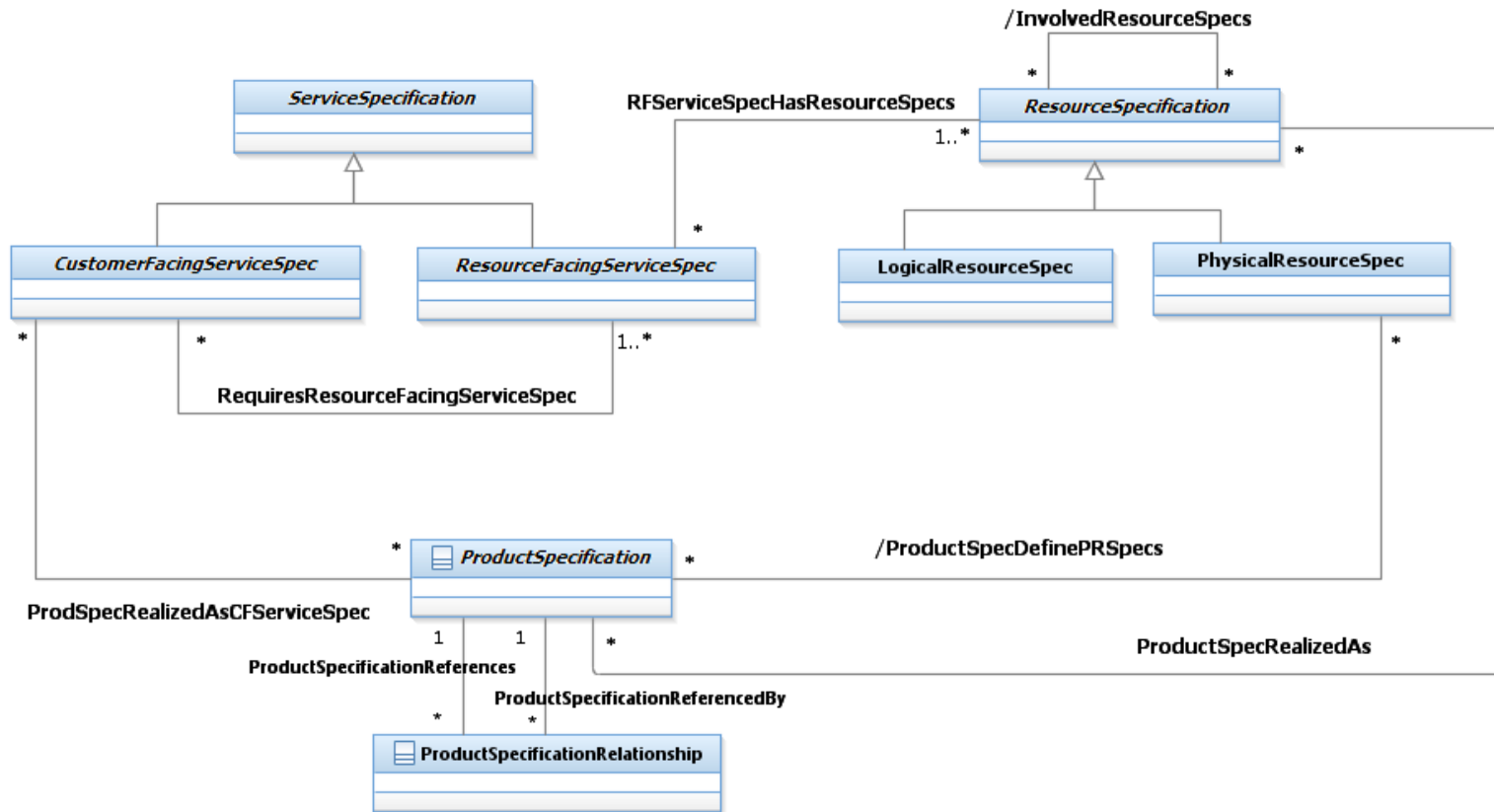


Figure PR.33 - Relationship Between ProductSpec, ResourceSpec and ServiceSpec



The key idea that we can take from the Product Addendum (Addendum 3) of the SID is the concept of a Specification that is used to define an object. This can be used to determine the essential physical characteristics of the PhysicalResource that can later be customized for a particular ProductOffering. The PhysicalResourceSpecification helps integrate the otherwise diverse worlds of Product and Physical Resources.

The ProductSpecDefinesCFSSpecs and ProductSpecDefinesPRSpecs enable a ProductSpecification to templatize the definition of CustomerFacingServiceSpecs and PhysicalResourceSpecs, respectively.

The LResSpecBindsToPResSpec association represents the binding between a set of LogicalResourceSpecifications and a set of PhysicalResourceSpecifications. The semantics of this binding are represented by the LogicalPhysicalResourceSpec association class (not shown to keep Figure PR. as simple as possible).

Thus, we may make the following important observations concerning the relationship between a ProductSpecification and PhysicalResourceSpecs and Services:

- A PhysicalResource is visible to a customer; therefore, a PhysicalResourceSpec can be directly aggregated by a ProductSpecification
 - A LogicalResourceSpec is *not* directly visible to a customer
 - LogicalResources require PhysicalResources to host them, which is done using the LResSpecBindsToPResSpec association
- A CustomerFacingServiceSpec is visible to a customer; therefore, a CustomerFacingServiceSpec can be directly aggregated by a ProductSpecification
 - A ResourceFacingServiceSpec is *not* directly visible to a customer
 - ResourceFacingServices are contained in CustomerFacingServices, which is done using the RequiresResourceFacingServiceSpec aggregation

Thus, the goals of separating PhysicalResources from LogicalResources (and also CustomerFacingServices from ResourceFacingServices) through Products has been enforced at the Specification level.

Similarly, Products are related to Services and Resources as shown in PR.34 below.

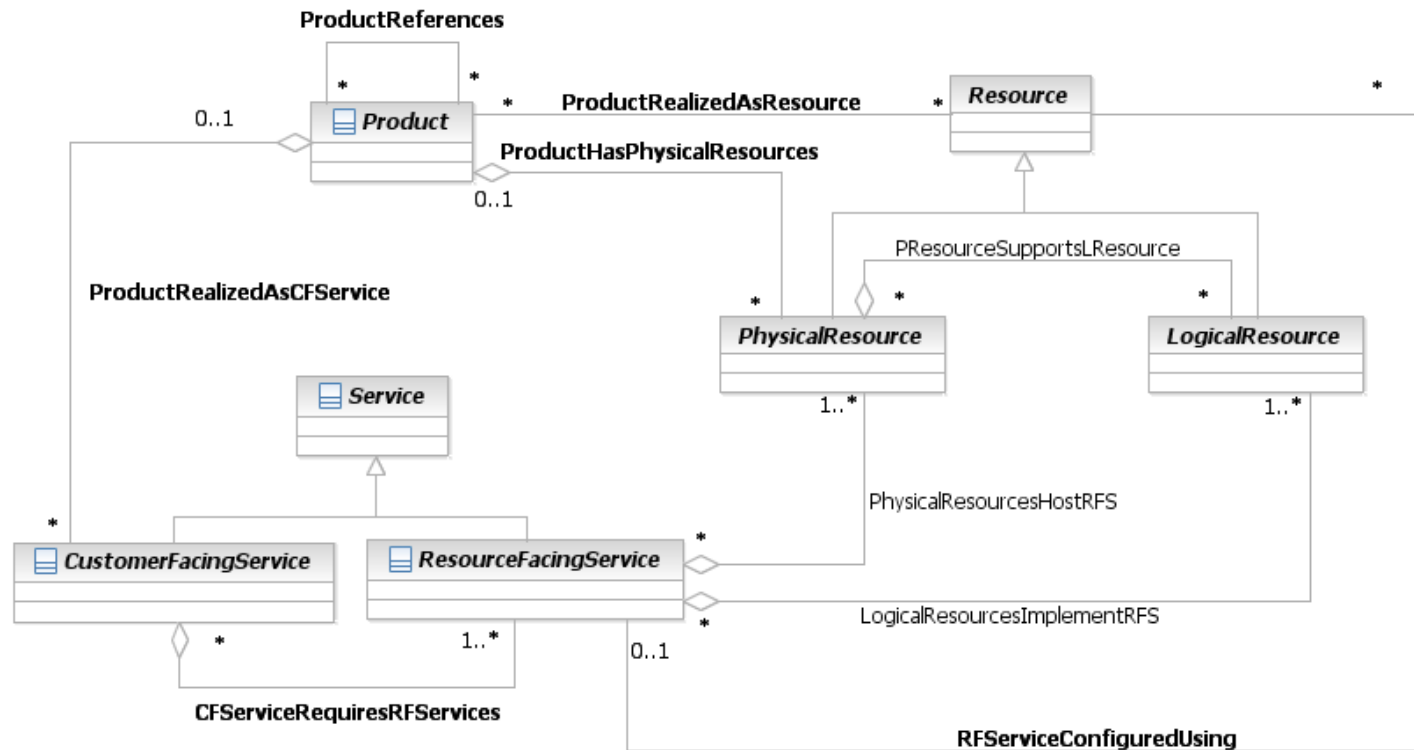


Figure PR.34 - Relationship Between Product, Service and Resource

The logic at the instance level (shown in PR.34 above) mirrors the logic at the specification level (shown in Figure PR. above). **PhysicalResources** and **CustomerFacingServices** are directly visible to a **Product**, using the **ProductHasPhysicalResources** and **ProductHasCustomerFacingServices** aggregations, respectively. A set of **PhysicalResources** supports a set of **LogicalResources** as defined by the **PResourceSupportsLResource** aggregation. This enables **Products** to *indirectly* specify **LogicalResources**, though the **Customer** is not aware of this.



Similarly, a set of CustomerFacingServices requires one or more ResourceFacingServices to be implemented, as defined by the CFServiceRequiresRFServices aggregation. Note that the cardinality of this aggregation is 0..n to 1..n, whereas the cardinality of the PResourceSupportsLResource aggregation is 0..n to 0..n. This is because a CustomerFacingService must have at least one ResourceFacingService in order for it to be implemented, whereas a PhysicalResource can exist independent of any LogicalResources.

Note that a ResourceFacingService can specify the set of LogicalResources and PhysicalResources that it requires using the LogicalResourcesImplementRFS and PhysicalResourcesHostRFS aggregations, respectively. This enables Services that are visible to the Customer (i.e., CustomerFacingServices) to be specified in terms of the Resources (physical and logical) that they need.

Policy

Policy is pervasive, even in the physical domain. For example, one can imagine policies for controlling:

- physical access to ManagedPhysicalDevices
- policies that define installation, power-up and power-down instructions for physical entities
- policies that define maintenance and service instructions

Clearly, there are many more types of policies that can be defined.

The interaction between entities in the Policy domain and entities in the PhysicalResource domain are many. Numerous of these interactions are rich in nature. Some of these relationships will be specified in the Policy Addendum (1-POL), because they require an understanding of the Policy model.



1.4. References

[053Main]	TMF053, "The NGOSS Technology Neutral Architecture", TMF
[Add 3]	SID Product Model, GB922, Addendum 3
[Add 4SO]	SID Service Overview Model, GB922, Addendum 4SO
[Add 5LR]	SID LogicalResource Model, GB922, Addendum 5LR
[GB922]	SID Concepts and Principles, GB922 (main document)
m.3100	Generic Network Information Model, ITU-T recommendation
CIM	Common Information Model, DMTF http://www.dmtf.org/standards/standard_cim.php
[CIMPROB]	Mining Information from the DMTF CIM into the TMF SID, ed by John Strassner (This was a deliverable from the TMF-DMTF liaison)
Zachman	Zachman Framework http://www.zifa.com/frmwork2.htm
Larman	Applying UML and Patterns, Second Edition, ISBN 0-13-095004-1 http://www.craiglarman.com/book_applying_2nd/Applying_2nd.htm
Baumer	The Role Object (Design) Pattern. Download PDF from http://www.riehle.org/computer-science-research/1997/plop-1997-role-object.pdf
[eTOM]	Enhanced Telecoms Operations Map

1.5. Business Entity Definitions

The following sections will define the business entities referred to in the preceding portion of this Addendum. This section will consist of a business entity definition, definition of any attributes appropriate for the business level, and finally provide a business model in UML of that entity and its main relationships.

NOTE: This section was produced using a model-driven technique which is still being refined and which results in the loss of some dictionary properties. The same section from the previous major release of the Information Framework is included here as an embedded archive. For conformance purposes, attributes that are not listed there can be assumed to be optional rather than required.



GB922_Addendum_5P
R_R3-5_v3-1_Business

PhysicalResource Business Entity Definition

Business Entity Name	PhysicalResource
Description	This is an abstract base class for describing different types of hardware that constitute a Product. It has two main purposes: (1) to collect common attributes and relationships for all hardware, and (2) to provide a convenient, single point where relationships with other managed objects can be defined.
Sources	DEN-ng, OSS/J
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. It is called PhysicalElement in the DMTF CIM (this is not a direct match with this class, although the concepts are similar); no direct analog in M.3100
Related Business Entities	Resource (superclass), LogicalResource (sibling), PhysicalDevice (subclass), Hardware (subclass), PhysicalResourceSpec (template)
Business Rules	<i>To be determined</i>



PhysicalResource Business Entity Attributes Definition

Business Entity Name	PhysicalResource		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
manufactureDate	This is a string attribute that defines the date of manufacture of this item in the fixed format "dd/mm/yyyy". This is an optional attribute.	String	This is a DEN-ng attribute. It is called ManufactureDate in the PhysicalElement class of the DMTF CIM. No analog in M.3100.
otherIdentifier	This is a string that is used to contain other important identifying data, such as a bar code, of the hardware item. This is an optional attribute.	String	
powerState	This is an enumerated integer that defines the current power status of the hardware item. Values include: 0: Unknown 1: Not Applicable 2: No Power Applied 3: Full Power Applied 4: Power Save - Normal 5: Power Save - Degraded 6: Power Save - Standby 7: Power Save - Critical 8: Power Save - Low Power Mode 9: Power Save - Unknown 10: Power Cycle 11: Power Warning 12: Power Off Value 1 means that the hardware item doesn't require the direct application of power (e.g., a nut or bolt). If the value for this item is 3, then the PowerCapability class will describe the particular power requirements of this item through the HasPower association. This is an optional attribute.	Integer	This is a DEN-ng attribute. CIM defines a PoweredOn attribute, but it is a Boolean. No analog in M.3100
serialNumber	This is a string that represents a manufacturer-allocated number used to identify different	String	This is a DEN-ng attribute. It is called SerialNumber in the PhysicalElement



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	instances of the same hardware item. The ModelNumber and PartNumber attributes are used to identify different types of hardware items. This is a REQUIRED attribute.		class of the DMTF CIM, but it is not required. No analog in M.3100.
versionNumber	This is a string that identifies the version of this object. This is an optional attribute.	String	

Business Entity Model – PhysicalResource

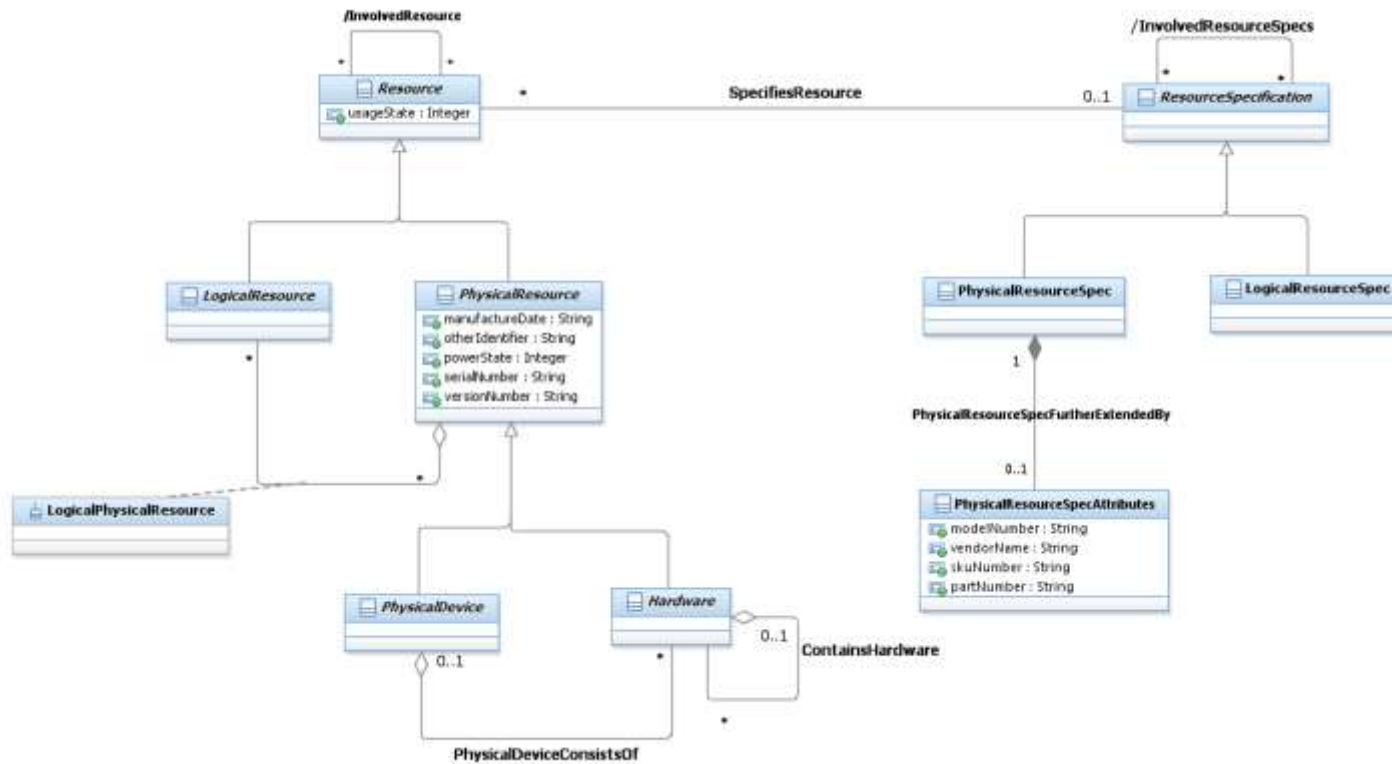


Figure PR- 35. PhysicalResource Business Entity Model



PhysicalResourceSpecification Business Entity Definition

Business Entity Name	PhysicalResourceSpecification
Description	This is a concrete class for describing specific attributes, behavior, relationships, constraints, and semantics for building PhysicalResource objects.
Sources	DEN-ng, OSS/J, MetaSolv model
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100, though analogs of some of the attributes can be found in the DMTF PhysicalElement class.
Related Business Entities	PhysicalResource (subject), ResourceSpecification (superclass), PhysicalResourceSpecAtomic (subclass), PhysicalResourceSpecComposite (subclass), LogicalResourceSpec, LogicalResourceSpecAtomic, LogicalResourceSpecComposite
Business Rules	<i>To be determined</i>



PhysicalResourceSpecification Business Entity Attributes Definition

Business Entity Name	PhysicalResourceSpecification		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
modelNumber	This is a string that represents a manufacturer-allocated number used to identify the general type and/or category of the hardware item. This, in combination with the PartNumber and the VendorName, identify different types of hardware items. The SerialNumber can then be used to differentiate between different instances of the same type of hardware item. This is an optional attribute.	String	This is a DEN-ng attribute; no direct equivalent in the DMTF CIM; could be represented by either the Tag or the OtherIdentifyingInfo attributes of PhysicalElement. No analog in M.3100.
partNumber	This is a string that defines a manufacturer-allocated part number assigned by the organization that manufactures the hardware item. This, in combination with the ModelNumber and the VendorName, identify different types of hardware items. The SerialNumber can then be used to differentiate between different instances of the same type of hardware item. This is a REQUIRED attribute.	String	This is a DEN-ng attribute; it is called PartNumber in the PhysicalElement class of the DMTF CIM, but it isn't required. No analog in M.3100.
skuNumber	This is a string that defines the manufacturer-allocated Stock Keeping Unit (SKU) number of the hardware item. This is an optional attribute.	String	This is a DEN-ng attribute; it is called SKU in the PhysicalElement class of the DMTF CIM. No analog in M.3100.
vendorName	This is a string that defines the name of the manufacturer. This, in combination with the ModelNumber and the PartNumber, identify different types of hardware items. The SerialNumber can then be used to differentiate between different instances of the same type of hardware item. This is a REQUIRED attribute.	String	This is a DEN-ng attribute; called Manufacturer in the PhysicalElement class of the DMTF CIM, but it isn't required. No analog in M.3100.

Business Entity Model – PhysicalResourceSpecification

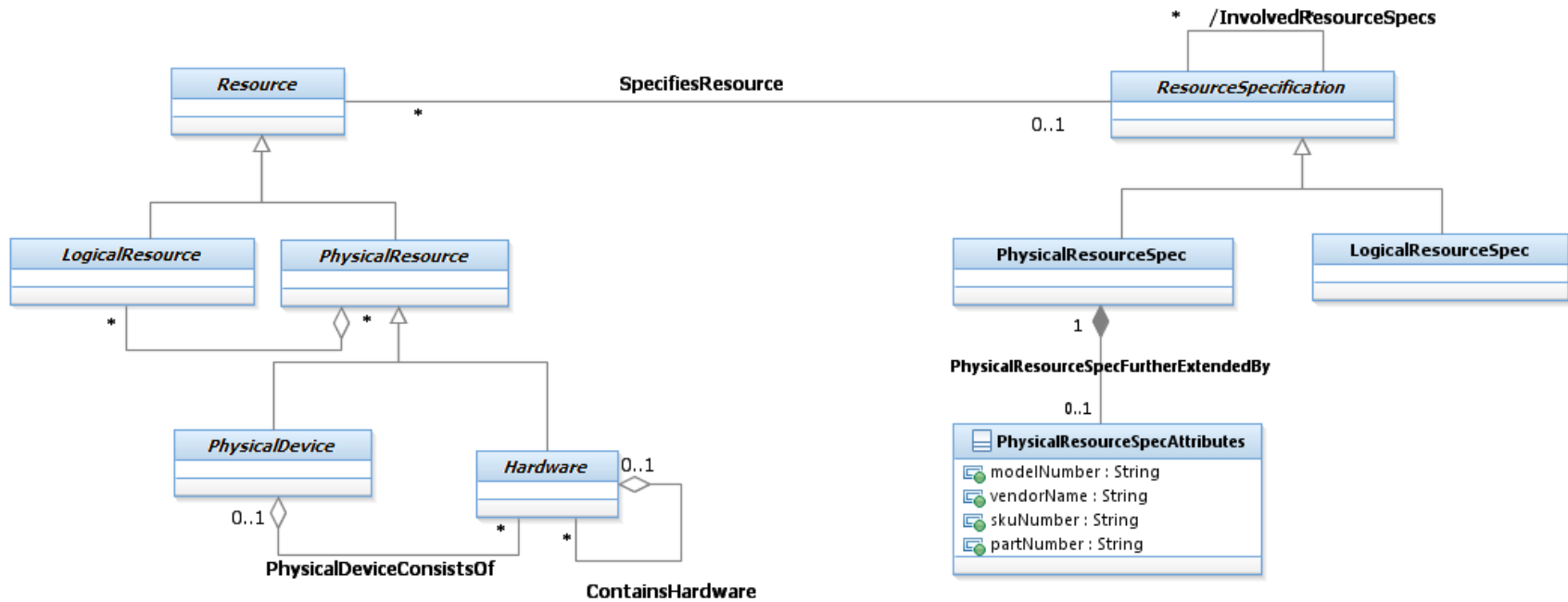


Figure PR- 36. PhysicalResourceSpecification Business Entity Model



PhysicalResourceRole Business Entity Definition

Business Entity Name	PhysicalResourceRole
Description	This is an abstract base class for representing the different types of roles that various physical resources can take on. For this first iteration, two specialized roles have been defined: holder and adapter. This enables a single object, such as a Card, to have additional functions. For example, a Card may also serve as a motherboard or hosting board for another Card. In this situation, there isn't a separate EquipmentHolder – rather, the Card acts as a holder in addition to providing its normal functions. This class is the base class for defining different types of physical resource roles.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100.
Related Business Entities	PhysicalResource, LogicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, EquipmentHolder, Equipment, AuxiliaryComponent, PhysicalComponent
Business Rules	<i>To be determined</i>



PhysicalResourceRole Business Entity Attributes Definition

Business Entity Name	PhysicalResourceRole		
Attribute Name	Description		Notes (used to map to other models; blank is other models don't have it)

Business Entity Model – PhysicalResourceRole

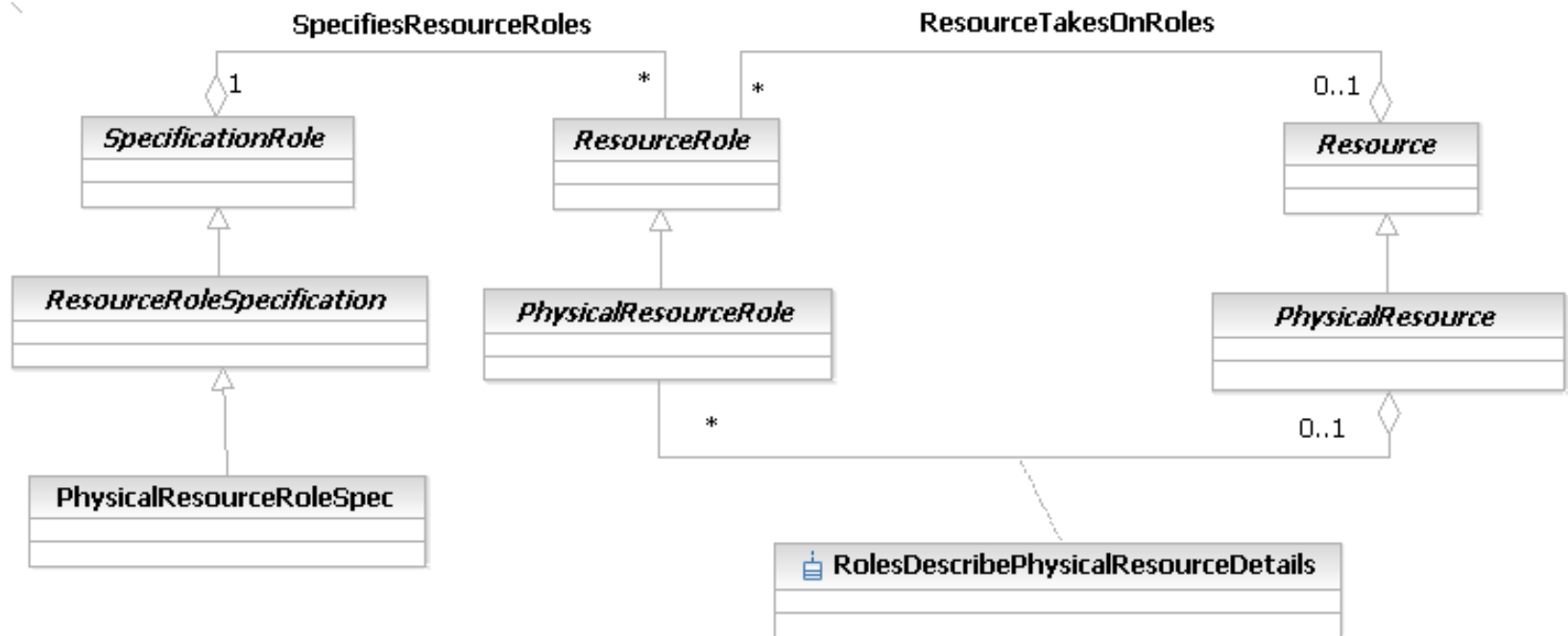


Figure PR- 37. PhysicalResourceRole Business Entity Model



PhysicalHolderRole Business Entity Definition

Business Entity Name	PhysicalHolderRole
Description	This is a concrete class that represents any type of physical resource that can play a holding role. A common example is a Card that can also serve as a motherboard, or a Card that can serve as a daughter card that attaches to a motherboard.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100.
Related Business Entities	PhysicalDevice, PhysicalResource, Equipment, EquipmentHolder, AuxiliaryComponent, HardwareRole (superclass), PhysicalAdapterRole, PhysicalDeviceRole, PhysicalResourceRole
Business Rules	<i>To be determined</i>



PhysicalHolderRole Business Entity Attributes Definition

Business Entity Name	PhysicalHolderRole		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
typeOfHolderRole	This is an enumerated integer that defines the various types of holding roles that this object can play. Values include: 0: undefined; 1: host (e.g., a motherboard) 2: client (e.g., a daughterboard) This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.

Business Entity Model – PhysicalHolderRole

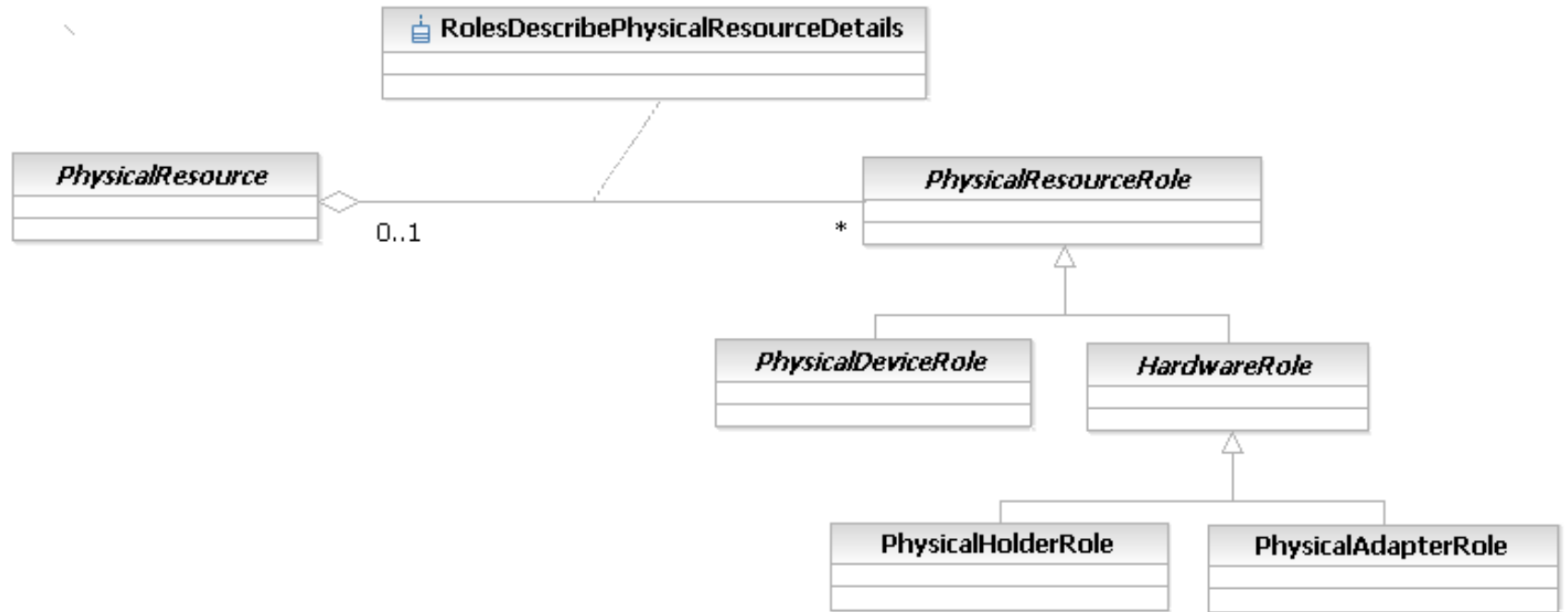


Figure PR- 38. PhysicalHolderRole Business Entity Model



PhysicalAdapterRole Business Entity Definition

Business Entity Name	PhysicalAdapterRole
Description	This is a concrete class that represents any type of physical resource that can play a physical adapter role. The Adapter role enables a piece of Hardware to adapt its use. For example, sometimes Cards and Chassis evolve along separate paths, causing future versions of one to no longer be physically compatible with present and/or future versions of the other. The solution to this is to use an intermediate piece of Hardware, called an Adapter, to extend the existing physical structure of an EquipmentHolder to enable otherwise incompatible Equipment to be plugged into an EquipmentHolder. The Adapter conceptually creates a new type of EquipmentHolder that fits into the existing EquipmentHolder. This enables Cards that would otherwise be physically and/or electrically incompatible with the existing EquipmentHolder to be supported, by interfacing to the old EquipmentHolder via the new Adapter.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100.
Related Business Entities	PhysicalDevice, PhysicalResource, Equipment, EquipmentHolder, AuxiliaryComponent, HardwareRole (superclass), PhysicaHolderRole, PhysicalDeviceRole, PhysicalResourceRole
Business Rules	<i>To be determined</i>



PhysicalAdapterRole Business Entity Attributes Definition

Business Entity Name	PhysicalAdapterRole		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
typeOfAdapterRole	This is an enumerated integer that defines the various types of adapter roles that this object can play. Values include: 0: undefined; 1: host (e.g., a motherboard) 2: client (e.g., a daughterboard) This is a REQUIRED attribute.	Integer	No analog in the DMTF CIM or M.3100.



Business Entity Model – PhysicalAdapterRole

The business entity model for the PhysicalAdapterRole is the same as that of the PhysicalHolderRole, and was shown in Figure PR.37 above.



Hardware Business Entity Definition

Business Entity Name	Hardware
Description	This is an abstract base class that represents any type of hardware unit that exists as an atomic unit that is not a PhysicalLink or a PhysicalConnector. Its principal subclasses are Equipment, PhysicalDevice, and PhysicalPort.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class; no direct analogs in the DMTF CIM or M.3100; some parts of the CIM PhysicalPackage can be mapped to this class, but that destroys the separation achieved by splitting PhysicalResource, Hardware, ManagedHardware, and PhysicalContainer
Related Business Entities	Resource, PhysicalResource (superclass), LogicalResource (sibling), PhysicalDevice (which consists of a set of Hardware), ManagedHardware (subclass), PhysicalConnector (subclass), PhysicalResourceRole, PhysicalDeviceRole
Business Rules	<i>To be determined</i>



Hardware Business Entity Attributes Definition

Business Entity Name		Hardware		
Attribute Name		Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
depth		This attribute defines the depth of the ManagedComponent using the units specified in the MeasurementUnits attribute. This is an optional attribute.	String	This is a DEN-ng attribute; called Depth in the PhysicalPackage class of the DMTF CIM; datatype is real32. No analog in M.3100.
height		This attribute defines the height of the ManagedComponent using the units specified in the MeasurementUnits attribute. This is an optional attribute.	String	This is a DEN-ng attribute; called Height in the PhysicalPackage class of the DMTF CIM; datatype is real32. No analog in M.3100.
measurementUnits		This attribute defines the MeasurementUnits for the Depth, Height, and Width attributes of this object. Values include: 0: Unknown (or not measured) 1: inches 2: feet 3: millimeters 4: centimeters 5: meters This is an optional attribute. However, if any of the Depth, Height, or Width attributes are defined, then this attribute is REQUIRED.	Integer	This is a DEN-ng attribute; not present in the DMTF CIM.
weight		This attribute defines the weight of the ManagedComponent using the units specified in the WeightUnits attribute. This is an optional attribute.	String	This is a DEN-ng attribute; called Weight in the PhysicalPackage class of the DMTF CIM; datatype is real32. No analog in M.3100.
weightUnits		This attribute defines the Units for the Weight attribute of this object. Values include: 0: Unknown (Not Measured) 1: ounces 2: pounds 3: grams 4: kilograms	Integer	This is a DEN-ng attribute; not present in the DMTF CIM.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	This is an optional attribute. However, if Weight attribute is defined, then this attribute is REQUIRED.		
width	This attribute defines the width of the ManagedComponent using the units specified in the MeasurementUnits attribute. This is an optional attribute.	String	This is a DEN-ng attribute; called Width in the PhysicalPackage class of the DMTF CIM; datatype is real32. No analog in M.3100.

Business Entity Model – Hardware

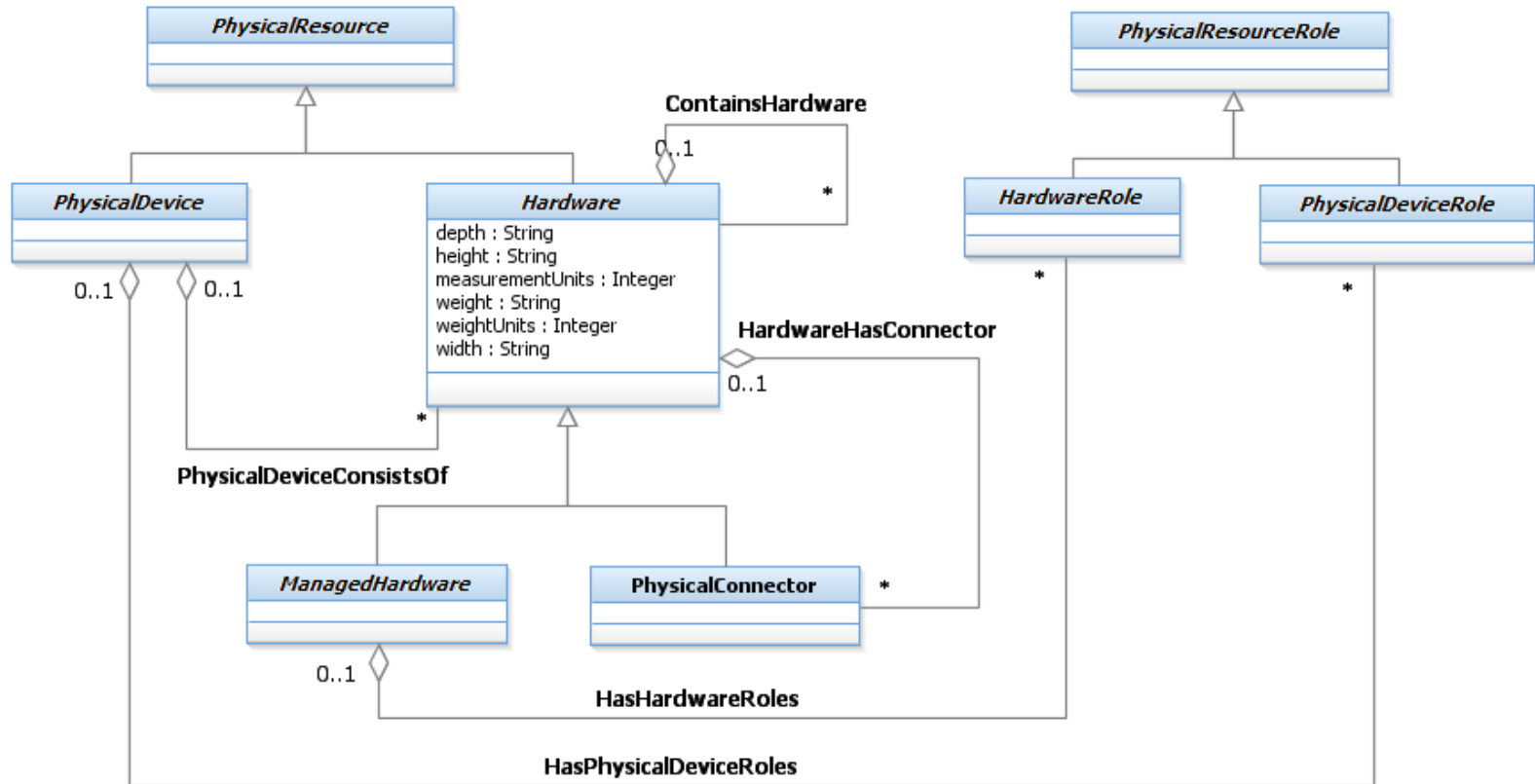


Figure PR- 39. Hardware Business Entity Model



ManagedHardware Business Entity Definition

Business Entity Name	ManagedHardware
Description	This is an abstract base class that adds additional semantics to the Hardware base class. These semantics are used to provide management information on the hardware. For example, attributes defined by this class can provide the administrative and operational state of the entity, and tell whether it has any alarms.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analog in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware (superclass), PhysicalContainer (subclass), Equipment, EquipmentHolder, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



ManagedHardware Business Entity Attributes Definition

Business Entity Name	ManagedHardware		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
additionalInfo	This is a free-form string that is used to contain additional vendor-specific information about the managed element, such as a vendor-specific asset tracking number or special installation notes. It can also be used to map vendor-specific naming (e.g., Port 1 is the same as E0/2/1). This can be NULL. This is an optional attribute.	String	
administrativeState	This attribute is an enumerated integer that describes the current physical state of the ManagedHardware. Values include: 0: Unknown 1: Unlocked 2: Locked 3: Shutting Down 4: Starting Up 5: Testing 6: Maintenance 7: Not Applicable 8: Not able to inform This is a REQUIRED attribute.	Integer	This is based on similar M.3100 concepts, but extends their semantics. No real analog in the DMTF CIM.
physicalAlarmReportingEnabled	This is a Boolean attribute, and defines whether alarm reporting for this object instance is enabled or not. TRUE means that reporting is allowed, and FALSE means that reporting is inhibited. Note that some physical entities are not	Boolean	No concept of this in either M.3100 or the DMTF CIM.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	capable of reporting physical alarms, while some are. For those that are not capable of reporting physical alarms, this value MUST be set to FALSE. Remember that this is for physical alarm reporting. In most cases, there are corresponding logical alarms. The ManagementEntity class hierarchy describes and correlates these. This is a REQUIRED attribute.		
physicalAlarmStatus	This is an enumerated integer that indicates the occurrence of an abnormal physical condition relating to an object. This attribute may also function as a summary indicator of alarm conditions associated with a specific resource. It is used to indicate the existence of an alarm condition, a pending alarm condition such as threshold situations, or (when used as a summary indicator) the highest severity of active alarm conditions. This attribute expands on the standard ITU semantics and updates them to include eTOM concepts. Values include: 0: unknown 1: activeReportable-Critical 2: activeReportable-Major 3: activeReportable-Minor 4: activeReportable-Indeterminate 5: activeReportable-Warning 6: activePendingDecision 7: active-underRepair 8: active-beingReplaced	Integer	This attribute expands on the standard ITU semantics and updates them to include eTOM concepts. No concept of this in the DMTF CIM.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	9: cleared This is a REQUIRED attribute.		
coolingRequirements	This is a free-form string that specifies the cooling requirements for this ManagedComponents. Specific cooling information is defined by the Cooling association. This is an optional attribute.	String	
hardwarePurpose	This is an enumerated integer that defines the purpose of the ManagedHardware. Values include: 1: Required 2: Optional 3: Redundant 4: Fail-Over 5: Other This is an optional attribute.	Integer	

Business Entity Model – ManagedHardware

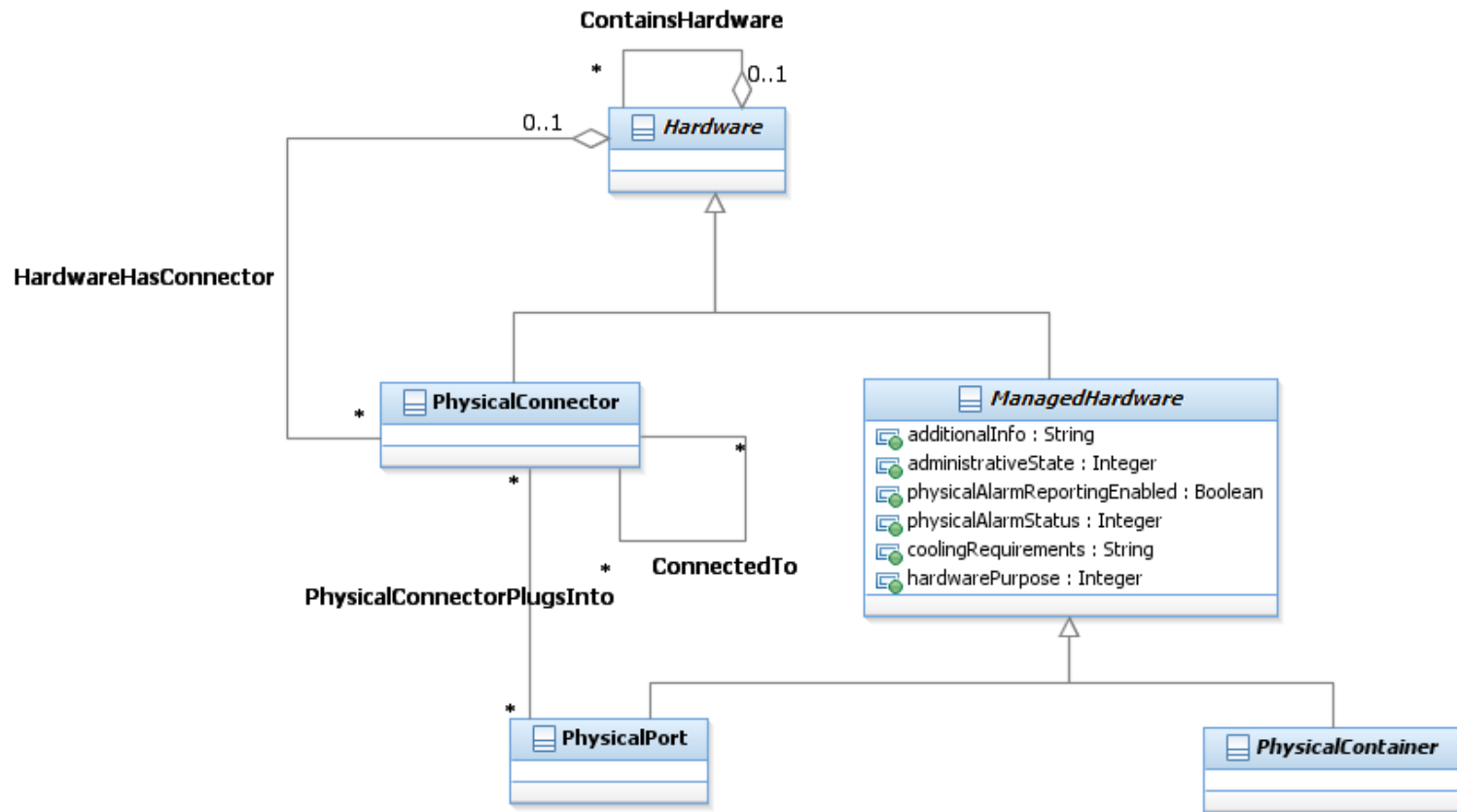


Figure PR- 40. ManagedHardware Business Entity Model



PhysicalContainer Business Entity Definition

Business Entity Name	PhysicalContainer
Description	This is an abstract class that is used to add additional semantics to the ManagedHardware class. Its attributes define whether a ManagedHardware object can be removed and/or replaced, and whether this action requires power to be removed or not when the action is performed.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analog in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware (superclass), Equipment (subclass), EquipmentHolder (subclass), AuxiliaryComponent (subclass), PhysicalComponent (subclass), PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



PhysicalContainer Business Entity Attributes Definition

Business Entity Name	PhysicalContainer		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
hotSwappable	This is a Boolean attribute that defines whether it is possible to replace this object instance with a physically different, but equivalent, object instance while the containing Equipment has power applied to it. TRUE means that it is HotSwappable, and FALSE means that it is not. All HotSwappable PhysicalComponents are inherently Removable and Replaceable. This is a required attribute.	Boolean	This is a DEN attribute, and is also present in the DMTF CIM PhysicalComponent class. Not present in M.3100.
removable	This is a Boolean that defines whether it is possible to insert and remove this object instance from the Equipment in which it is installed, without impairing the function or packaging of the Equipment. TRUE means that it is removable, and FALSE means that it is not. A Package can still be Removable if power must be 'off' in order to perform the removal. If power can be 'on' and this object instance can still be removed, then this object instance is both Removable and HotSwappable. This is a required attribute.	Boolean	This is a DEN attribute, and is also present in the DMTF CIM PhysicalComponent class. Not present in M.3100.
replaceable	This is a Boolean that defines whether it is possible to replace this object instance with a physically different instance of the same type. For example, some types of device allow various Chips to be upgraded. TRUE means that it is replaceable, and FALSE means that it is not. All Removable packages are inherently Replaceable. This is a required attribute.	Boolean	This is a DEN attribute, and is also present in the DMTF CIM PhysicalComponent class. Not present in M.3100.

Business Entity Model – PhysicalContainer

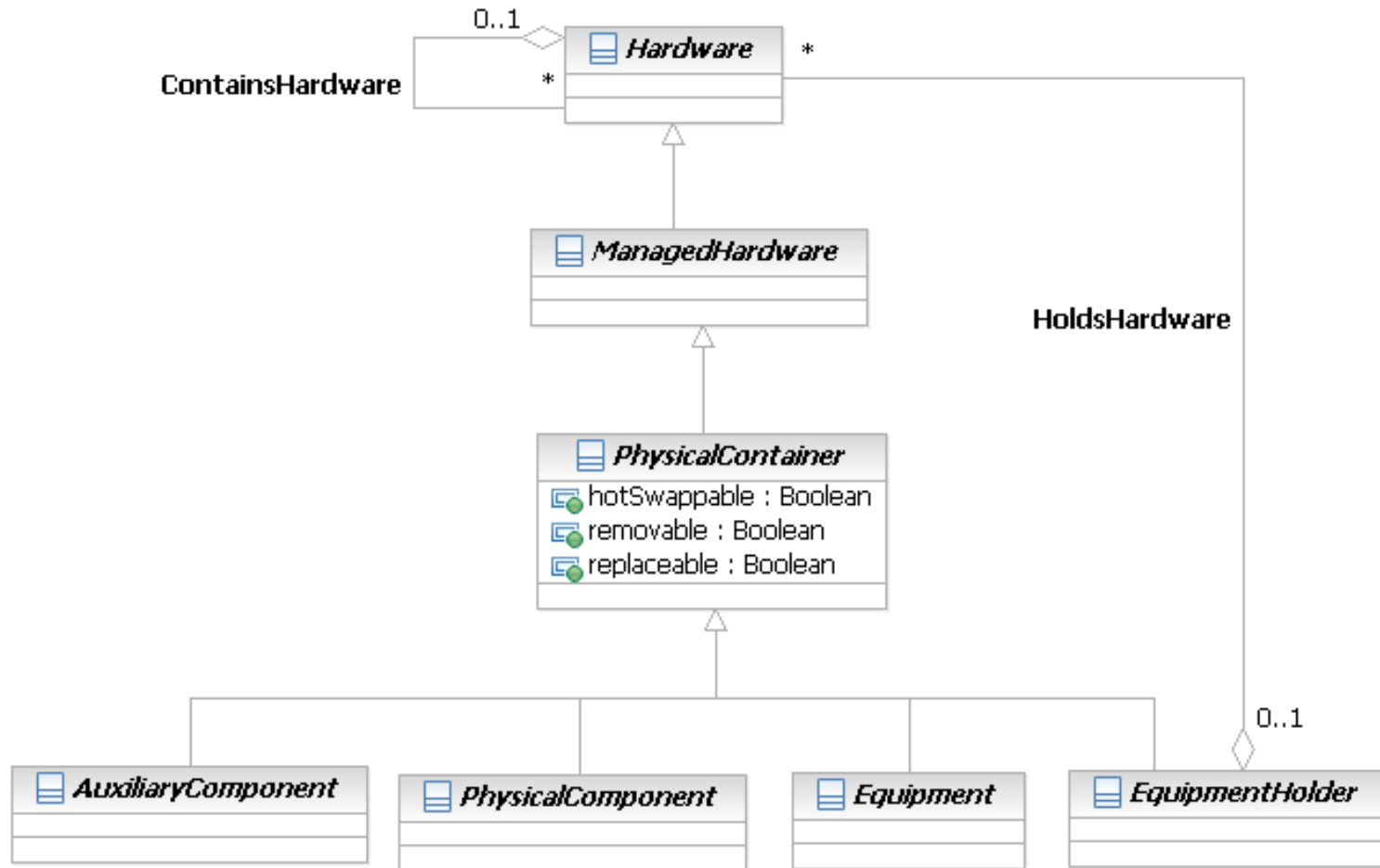


Figure PR- 41. PhysicalContainer Business Entity Model



PhysicalDevice Business Entity Definition

Business Entity Name	PhysicalDevice
Description	This is an abstract base class for representing hardware devices that can be managed. This class represents a convenient aggregation point for combining different aspects of a device (e.g., its physical composition as well as the set of services that it offers). It also enables the device itself to have a physical manifestation. Examples of this class include routers and switches, computers, and other end-devices that are managed.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100; this is similar to the DEN NetworkPackage class.
Related Business Entities	Resource, PhysicalResource (superclass), LogicalResource, PhysicalDeviceAtomic (subclass) , PhysicalDeviceComposite (subclass), Hardware, PhysicalResourceRole, PhysicalDeviceRole
Business Rules	<i>To be determined</i>



PhysicalDevice Business Entity Attributes Definition

Business Entity Name	PhysicalDevice		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
backplaneIndependent	This is a boolean attribute that, if TRUE, indicates that this ManagedDevice has independent backplanes that can be managed separately. This is an optional attribute.	Boolean	
backplaneNumber	This is an integer that defines the number of backplanes that this device has. This is an optional attribute.	Integer	
configurationOrder	This is a free-form string that provides any order-specific instructions for configuring the set of components that together constitute this PhysicalDevice. This is an optional attribute.	String	This is a DEN-ng attribute; not present in the DMTF CIM or in M.3100.
deviceGroupID	This is a string, and is used to uniquely identify this device as a member of a group of devices. This is an optional attribute.	String	This is a DEN-ng attribute; not present in the DMTF CIM or in M.3100.
isComposite	This is a Boolean attribute that, if TRUE, means that this physical device is in reality made up of a set of physical devices, each of which can be individually managed. This is an optional attribute.	Boolean	
canMixPower	This is a Boolean attribute that, if TRUE, means that AC and/or DC power supplies can be used in this device. If it is false, then only one or the other can be used.	Boolean	

Business Entity Model – PhysicalDevice

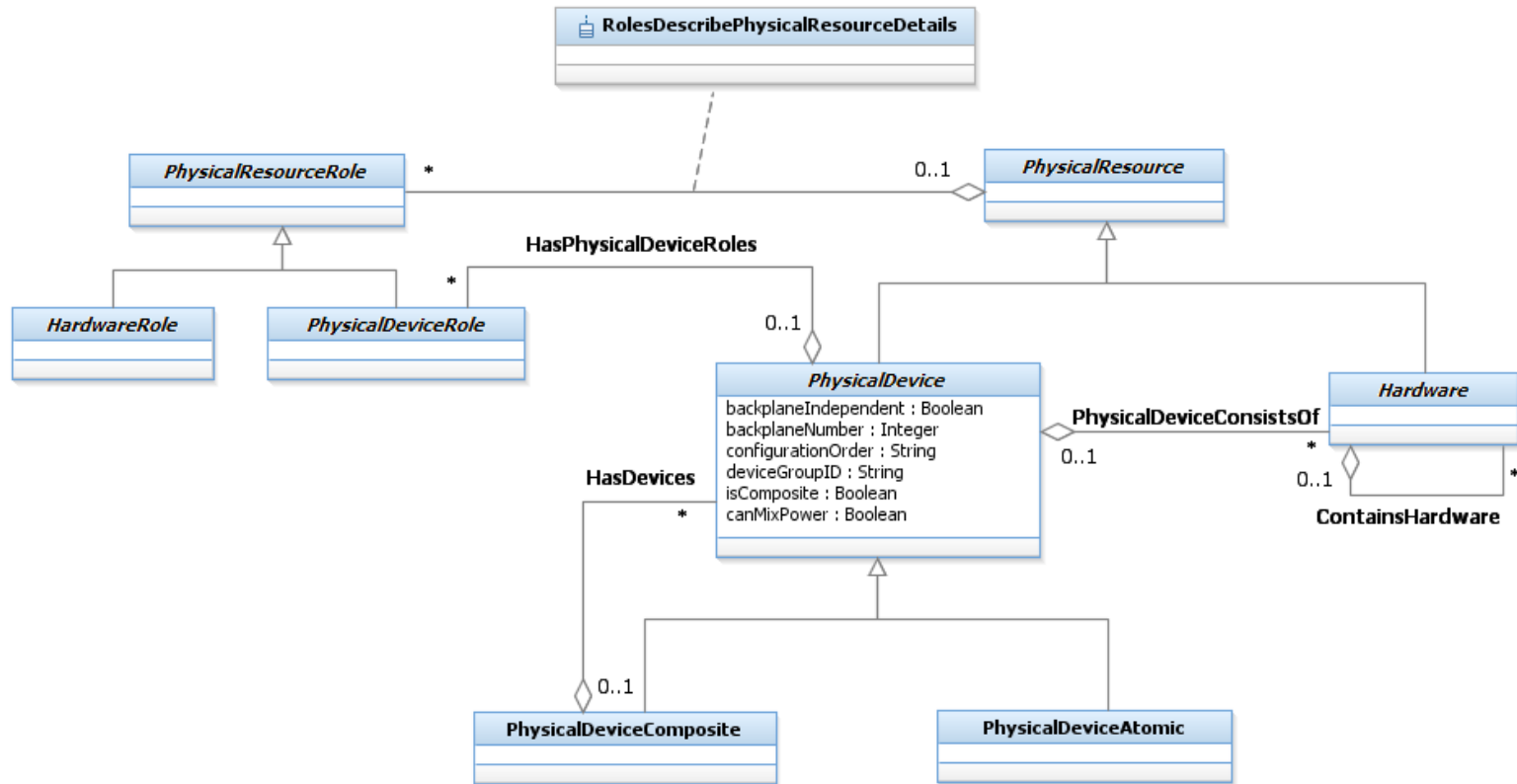


Figure PR- 42. PhysicalDevice Business Entity Model



PhysicalDeviceAtomic Business Entity Definition

Business Entity Name	PhysicalDeviceAtomic
Description	This is a concrete base class for representing hardware devices that can be managed that contains no subordinate devices. In other words, this physical device is a stand-alone physical device. This class represents a convenient aggregation point for combining different aspects of a device (e.g., its physical composition as well as the set of services that it offers). It also enables the device itself to have a physical manifestation. Examples of this class include routers and switches, computers, and other end-devices that are managed.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, LogicalResource, PhysicalDevice (superclass) , PhysicalDeviceComposite (sibling), Hardware, PhysicalResourceRole, PhysicalDeviceRole
Business Rules	<i>To be determined</i>



PhysicalDeviceAtomic Business Entity Attributes Definition

Business Entity Name	PhysicalDeviceAtomic		
Attribute Name	Description		Notes (used to map to other models; blank is other models don't have it)



Business Entity Model – PhysicalDeviceAtomic

The business entity model for the PhysicalDeviceAtomic class is the same as that for the PhysicalDevice class, which was shown in Figure PR.42 above.



PhysicalDeviceComposite Business Entity Definition

Business Entity Name	PhysicalDeviceComposite
Description	This is a concrete base class for representing hardware devices that can be managed that contains one or more sub-ordinate devices. In other words, this physical device is not a stand-alone physical device; rather, it represents an aggregation of physical devices. Each physical device in this aggregation can be managed. This class represents a convenient aggregation point for combining different aspects of a device (e.g., its physical composition as well as the set of services that it offers). It also enables the device itself to have a physical manifestation. Examples of this class include routers and switches, computers, and other end-devices that are managed.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, LogicalResource, PhysicalDevice (superclass) , PhysicalDeviceAtomic (sibling), Hardware, PhysicalResourceRole, PhysicalDeviceRole
Business Rules	<i>To be determined</i>



PhysicalDeviceComposite Business Entity Attributes Definition

Business Entity Name	PhysicalDeviceComposite		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
numberOfDevicesTotal	This attribute defines the total number of PhysicalDevices aggregated by this PhysicalDeviceComposite object. Note that this aggregation is for that particular level of aggregation. Thus, if a PhysicalDeviceComposite, called A, contains another PhysicalDeviceComposite, called B, then B's instance of this attribute will define how many PhysicalDevices are aggregated by B, whereas A's instance of this attribute will defined how many PhysicalDevices are aggregated by A (which includes those aggregated by B).	Integer	
numberOfDevicesCurrent	This attribute defines the current number of PhysicalDevices aggregated by this PhysicalDeviceComposite object that are active and manageable. Note that this aggregation is for that particular level of aggregation. Thus, if a PhysicalDeviceComposite, called A, contains another PhysicalDeviceComposite, called B, then B's instance of this attribute will define how many PhysicalDevices are aggregated by B, whereas A's instance of this attribute will defined how many PhysicalDevices are aggregated by A (which includes those aggregated by B).	Integer	



Business Entity Model – PhysicalDeviceComposite

The business entity model for the PhysicalDeviceComposite class is the same as that for the PhysicalDevice class, which was shown in Figure PR.42 above.



PhysicalConnector Business Entity Definition

Business Entity Name	PhysicalConnector
Description	This is a concrete class that represents any type of hardware unit that is used to connect to other hardware units and transmit signals and/or power between them.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. Not present in M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, Equipment, EquipmentHolder, AuxiliaryComponent, PhysicalComponent, PhysicalPort
Business Rules	<i>To be determined</i>



PhysicalConnector Business Entity Attributes Definition

Business Entity Name	PhysicalConnector		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
cableType	This is an enumerated integer, and defines the particular type of cable that is attached to this connector. Values include: 0: Unknown 1: RS-232 2: RS-422 3: RS-423 4: RS-449 5: RS-485 6: RS-530 7: V.35 8: X.21 9: 9 um single-mode 10: 62.5/125 um multi-mode 11: USB to be continued, not done! This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute. It is similar to the ConnectorType attribute of the DMTF CIM PhysicalConnector class. Main difference is that the DMTF attribute was an array that combined this, gender, and other attributes into one. Also, this was not a required attribute in the DMTF CIM. Not present in M.3100.
gender	This is an enumerated integer that defines the gender type of the connector. Values are: 0: Unknown 1: Not Applicable 2: Male 3: Female This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute. It is similar to the ConnectorType attribute of the DMTF CIM PhysicalConnector class. Main difference is that the DMTF attribute was an array that combined this, gender, and other attributes into one. This was not a required attribute in the DMTF CIM. Not present in M.3100.
inUse	This is a boolean attribute that, if TRUE, indicates that this PhysicalConnector is in use by some other component of the system. This is an optional attribute.	Boolean	This is a DEN-ng attribute; not present in the DMTF CIM or M.3100.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
pinDescription	This is a free-form string describing the pin configuration and signal usage of a PhysicalConnector. This is a REQUIRED attribute.	String	This is a DEN-ng attribute; it is similar to the ConnectorPinout of the DMTF CIM PhysicalConnector class. Not present in M.3100.
typeOfConnector	This is an enumerated integer that defines the type of connector that this instance is. Values include: 0: Unknown 1: DB-9 2: DB-15 3: DB-25 4: DB-36 5: DB-60 6: SC 7: SG to be continued, not done! This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute; it is similar to the ConnectorType attribute of the DMTF CIM PhysicalConnector class. Main difference is that the DMTF attribute was an array that combined this, gender, and other attributes into one. This was not a required attribute in the DMTF CIM. Not present in M.3100.

Business Entity Model – PhysicalConnector

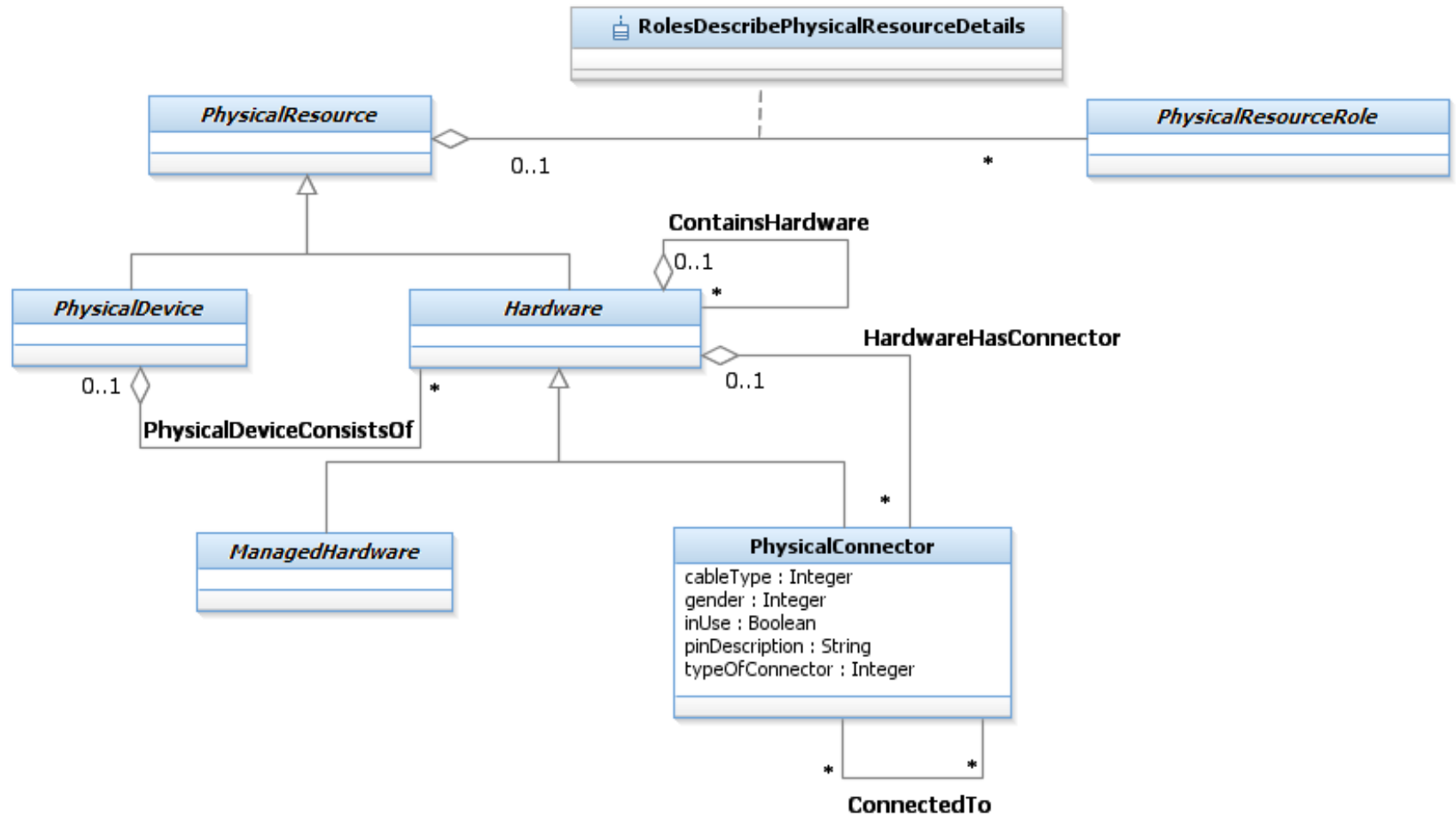


Figure PR- 43. PhysicalConnector Business Entity Model



PhysicalPort Business Entity Definition

Business Entity Name	PhysicalPort
Description	This class represents an actual or potential end point of a topological (physical) link, and corresponds directly to a physical port on a topology map. PhysicalPorts are always contained by another physical object - they can't exist by themselves. The two most common examples are PhysicalPorts on a Card and on a Chassis.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analogs in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, Equipment, EquipmentHolder, Card, Chassis, ResourceRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



PhysicalPort Business Entity Attributes Definition

Business Entity Name	PhysicalPort		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
duplexMode	This is an enumerated integer that defines the duplex mode of this port. Values are: 0: Unknown 1: Full Duplex 2: Half Duplex This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100 since no PhysicalPort exists.
ifType	This is an enumerated integer, and specifies the particular media type of the link. This attribute provides additional detail beyond that provided in the ifType of an ifEntry of a MIB (e.g., distinguishing between 10Base and 100Base ethernet). Values include: 0: Unknown 1: 10BaseT 2: 100BaseT 3: 10-100BaseT 4: 1000BaseT 5: 10000BaseT 6: DS-0 7: DS-1 8: DS-3 9: OC-3 10: OC-12 11: OC-48 12: OC-192 This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100 since no PhysicalPort exists.
portNumber	This is a non-zero integer that uniquely identifies this PhysicalPort instance from all other instances. This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100 since no PhysicalPort exists.
typeOfPPort	This is an enumerated integer that defines the particular	Integer	This is a DEN-ng attribute; no



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	type of PhysicalPort this instance is. Values include: 0: Unknown 1: Ethernet 2: FastEthernet 3: Auto-Sensing 4: GigabitEthernet 5: FastGigabitEthernet 6: DS-0 7: DS-1 8: DS-3 9: T1 10: T3 11: E1 12: E3 13: OC-3 14: OC-12 15: OC-48 16: OC-192 17: RS-232C This is a REQUIRED attribute.		analog in the DMTF CIM or M.3100 since no PhysicalPort exists.
vendorPortName	This is a string that contains the vendor-specific name of this port. This is different from the commonName attribute, which represents a system-wide naming structure for all ManagedEntities.	String	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100 since no PhysicalPort exists.

Business Entity Model – PhysicalPort

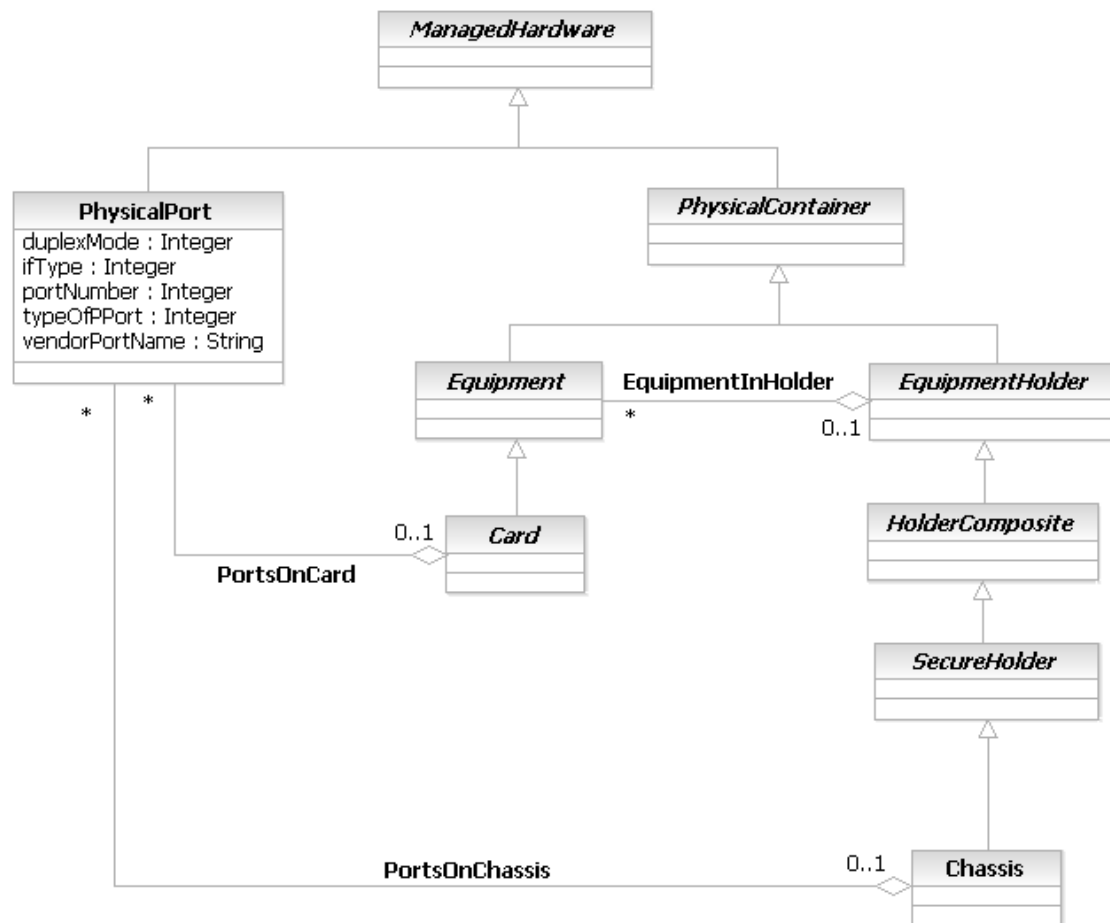


Figure PR- 44. PhysicalPort Business Entity Model



Equipment Business Entity Definition

Business Entity Name	Equipment
Description	This class is based on the M.3100 specification, and represents a class of managed objects that represents physical components of a Resource, including replaceable components. An instance of this object class is present in a single geographic location. An Equipment may be nested within another Equipment, thereby creating a containment relationship. The Equipment type shall be identified by sub-classing this object class. Either the name of the sub-class or an attribute may be used for identifying the Equipment type.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is an M.3100 class that has been modified in DEN-ng. No direct analog in the DMTF CIM.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, EquipmentHolder, Card, Chassis, AuxiliaryComponent, PhysicalComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Equipment Business Entity Attributes Definition

Business Entity Name	Equipment		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
resourceFulfillmentState	This attribute supports basic administration of plug-ins.	ResourceFulfillmentState	
protectionSchemeState	This attribute identifies the individual lock of this equipment. In case the equipment is not protected, the value "UNKNOWN" shall also be used.	ProtectionSchemeState	
protectionRole	This attribute defines the protection role that this equipment plays. In case the equipment is not protected, the value "NOT_APPLICABLE" shall be used.	ProtectionRole	
manufacturer	This attribute identifies the equipment manufacturer name. It is defined as a non-empty free format string with no semantics.	String	
ituArcStateAndStatusList	See R_TMF518_NRB_I_0001 and R_TMF518_NRB_I_0004. See supporting document SD1-8_encodingX731M3100.	ItuArcStateAndStatusList	
isReportingAlarms	This attribute provides an indication of whether alarm reporting for this Equipment is enabled (true) or disabled (false).	Boolean	
installedVersion	This attribute identifies the vendor's resource version of the installed equipment.	String	This is a DEN-ng attribute; no analog in the DMTF CIM or



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
			M.3100.
installedSerialNumber	This attribute contains the vendor's serial number of the installed equipment. Unique, if no default is provided. At least one serial number has to be provided.	String	
installedPartNumber	This attribute identifies the vendor's resource Part Number (PN) of the installed equipment. If PN is not available empty string shall be used. If the part and serial number are both non-null then the part+serial number together shall be unique.	String	
installedEquipmentObjectType	This attribute identifies the type of the installed resource. For example, "Fan" or "STM16" for the Equipment class and "Line Shelf" for the Equipment Holder class.) The installed equipment type is invariant for the lifetime of the hardware. This is an empty string if there is no expected equipment.	String	
expectedEquipmentObjectType	This attribute identifies the type of the expected resource. For example, "Fan" or "STM16" for the Equipment class and "Line Shelf" for the Equipment Holder class.) This is an empty string if there is no expected equipment.	String	



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
installStatus	This is an m.3100 attribute that represents the availability of the type of Equipment that this object represents. Its semantics are as follows. The attribute availability status is used to indicate whether the correct physical piece of equipment (in m.3100, it is called a "circuit pack") is installed or not. This is a set valued attribute and includes the values notInstalled and empty. If the type of the inserted physical circuit pack matches the value of the circuitPackType attribute (relating to the circuitPack instance) then the value of the availabilityStatus is an empty set. Otherwise, the value of the availabilityStatus attribute is notInstalled even if it is one of the acceptable circuit pack type This is implemented as an enumerated integer. The values will include at least the following: 0: Unknown 1: Operational (installed and matches expected type) 2: Installed (but does not	Integer	



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	match expected type) 3: Not Installed 4: In Maintenance 5: Failed 6: Not operational This is an optional attribute.		
expectedEquipmentType	This attribute identifies the type of the expected resource. For example, "Fan" or "STM16" for the Equipment class and "Line Shelf" for the Equipment Holder class. This is an empty string if there is no expected equipment. This is an optional attribute.	String	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.
installedEquipmentType	This attribute identifies the type of the installed resource. For example, "Fan" or "STM16" for the Equipment class and "Line Shelf" for the Equipment Holder class. The installed equipment type is invariant for the lifetime of the hardware. This is an empty string if there is no installed equipment. This is an optional attribute.	String	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.
redundancy	This is an enumerated integer that describes the redundancy capabilities of this particular Equipment. Values include: 0: Unknown	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	1: Primary (supported by a Redundant Equipment) 2: Redundant (supports a Primary Equipment) 3: Stand-alone (no Redundancy possible) This is an optional attribute.		
asapRef		ObjectName	

Business Entity Model – Equipment

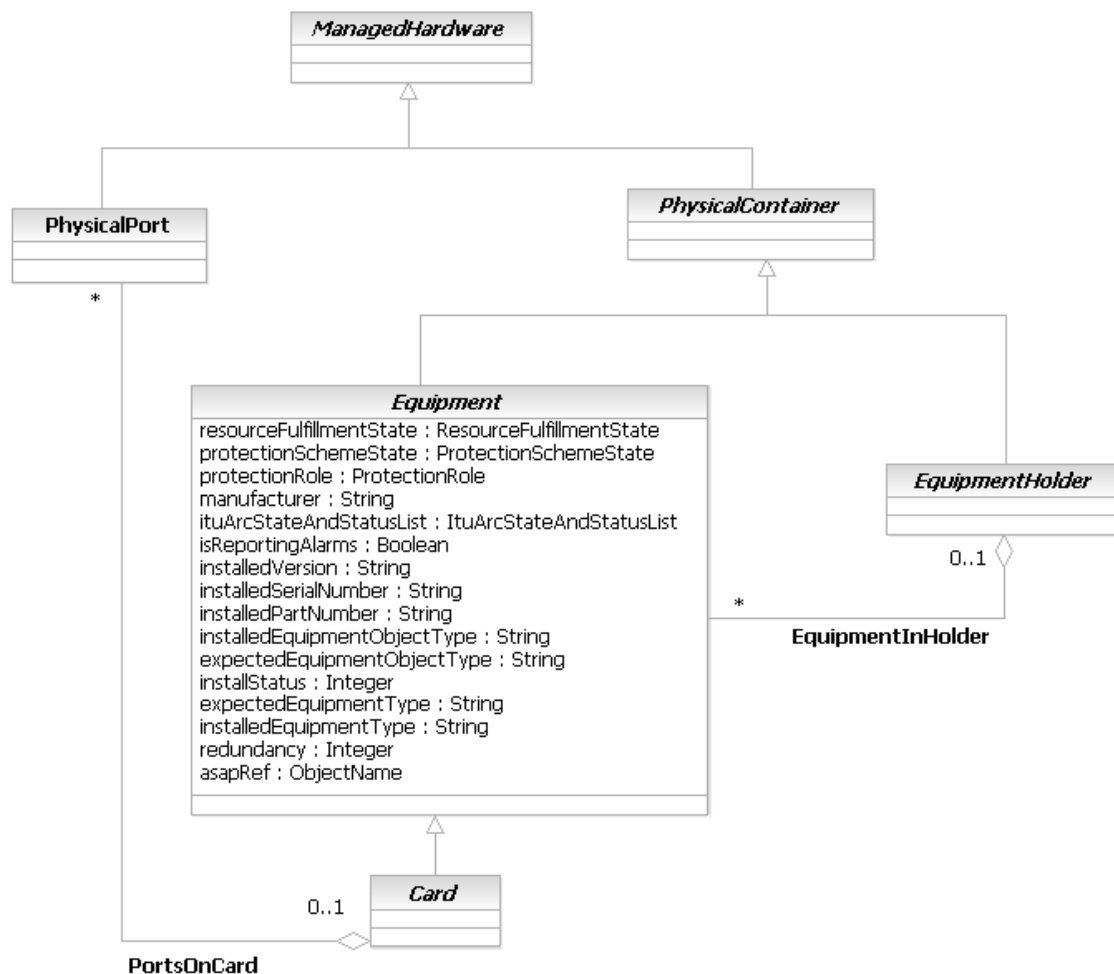


Figure PR- 45. Equipment Business Entity Model



Card Business Entity Definition

Business Entity Name	Card
Description	This class is based on the M.3100 specification, and represents a class of managed objects that represents physical components of a Resource, including replaceable components. An instance of this object class is present in a single geographic location. An Equipment may be nested within another Equipment, thereby creating a containment relationship. The Equipment type shall be identified by sub-classing this object class. Either the name of the sub-class or an attribute may be used for identifying the Equipment type.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is an M.3100 class that has been modified in DEN-ng. No direct analog in the DMTF CIM.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer (superclass), EquipmentHolder, MemoryCard (subclass), NetworkCard (subclass), SystemCard (subclass), UnknownCard (subclass), Chassis, AuxiliaryComponent, PhysicalComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Card Business Entity Attributes Definition

Business Entity Name	Card		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
cardCompatibilityResults	This is an attribute that is used to store the result of the negotiation process between a Card and its EquipmentHolder to see if they are compatible. This is the result of the isCompatible() method, with the Card being the source and the EquipmentHolder being the target, translated into a textual description.	String	
daughterCardInstallStatus	This is an enumerated integer that defines the current installation status of this Card's daughter Cards. Note that this defines the status of daughter Cards as viewed by the hosting Card. Status values of individual daughter Cards are defined by attributes in the daughter card itself. Values include: 0: Not Applicable (doesn't have any DaughterCards) 1: All Daughter Cards are installed 2: Some Daughter Cards are installed 3: No Daughter Cards are installed	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.
daughterCardOperatingStatus	This is an enumerated integer that defines the current operating status of this Card's daughter Cards. Note that this defines the operating status of daughter Cards as viewed by the hosting Card. Status values of individual daughter Cards are defined by attributes in the daughter card itself. This attribute only defines the physical operating characteristics of the daughter card. It does not say whether the daughter Card is functioning correctly, as that is a logical attribute.	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	Values include: 0: Not Applicable (doesn't have any DaughterCards) 1: All Daughter Cards are operating correctly 2: Some Daughter Cards are operating incorrectly 3: No Daughter Cards are operating correctly		
daughterCardRequirements	This is an enumerated integer that defines the relationship between this Card and all DaughterCards. Values include: 1: No DaughterCard can be attached 2: Requires 1 or more DaughterCards to function correctly 3: Can optionally use 1 or more DaughterCards	Integer	This is a DEN-ng attribute; no analog in M.3100. Called RequiresDaughterBoard in the CIM, but lacks the semantics of the DEN-ng attribute.
isConfigurablePhysically	This is a boolean attribute that, if TRUE, indicates that this Card has one or more options that can be physically configured. Each of these options has a distinct physical manifestation (e.g., additional memory, or faster CPU) that usually (but not always) results in occupying more room in the Card.	Boolean	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.
isMotherBoard	This is a Boolean attribute that, if TRUE, defines this Card as either a motherboard or another type of hosting board. When FALSE, it isn't.	Boolean	This is a DEN-ng attribute; no analog in M.3100. Called HostingBoard in the CIM, though the semantics are slightly different.
isUniquePhysical	This is a boolean attribute that, if TRUE, defines this Card to be physically different from other Cards of the same type and therefore requires a special slot. The	Boolean	This is a DEN-ng attribute; no analog in M.3100. The CIM has a similar attribute called SpecialRequirements but



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	unique aspects of this Card are described in the UniqueRequirementsPhysical attribute. An example might be a different form factor than other Cards of its type, or the ability to set jumpers on the Card to control its functionality (e.g., clocking).		there is a bug in the text.
maxDataWidth	This is an integer that defines the maximum bus width of this Card. Values include: 0: Unknown 1: Special 8: 8 bit data bus 16: 16 bit data bus 24: 24 bit data bus 32: 32 bit data bus 64: 64 bit data bus 128: 128 bit data bus The value "1" can be used for any non-standard data bus width.	Integer	
slotLayout	This is a free-form string that describes the positioning, spacing, typical usage, restrictions, and any other pertinent information that defines how the Card is to be positioned into the Slot.	String	This is a DEN-ng attribute; no analog in M.3100. Called SlotLayout in the CIM.
slotsRequired	This is an integer that defines the number of slots required to hold this Card. Since this is usually 1, that value is assigned as its default value.	Integer	This is a DEN-ng attribute; no analog in the DMTF CIM or M.3100.
uniqueRequirementsPhysical	This is a free-form string that contains the physically unique requirements of this Card. For example, it must go in a certain slot number because it has special dimensions. This attribute should only be filled in if the value of the IsUniquePhysical attribute is TRUE; otherwise, it should be	String	This is a DEN-ng attribute; no analog in M.3100. Called RequirementsDescription in the CIM



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	NULL.		
hardwareVersion	This is a string attribute that contains the hardware version number of this Card.	String	

Business Entity Model – Card

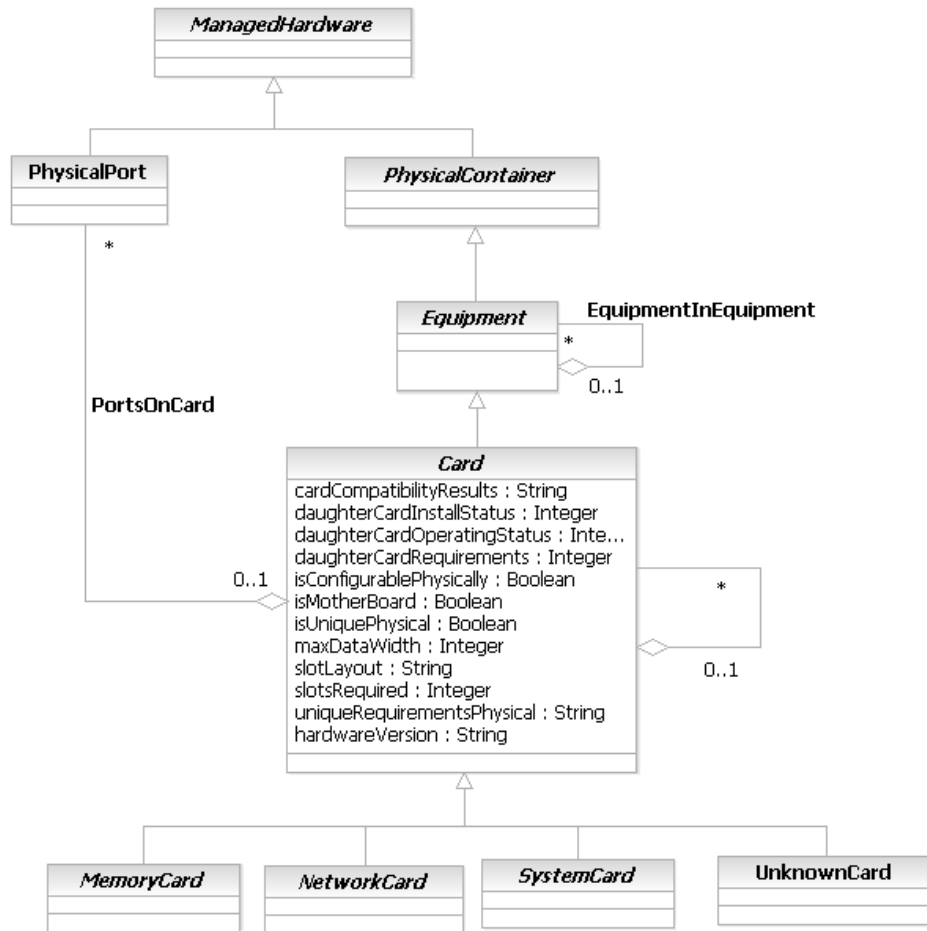


Figure PR- 46. Card Business Entity Model



MemoryCard Business Entity Definition

Business Entity Name	MemoryCard
Description	This is a type of Card that is dedicated to providing additional memory for use by other components of a Resource. Cards that are used to expand memory, or provide different types of memory, are examples of this type of Card. DEN-ng differentiates between this and other types of Cards, such as NetworkCards and MemoryCards.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analog in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, EquipmentHolder, Card, NetworkCard, SystemCard, UnknownCard, Chassis, AuxiliaryComponent, PhysicalComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



MemoryCard Business Entity Attributes Definition

Currently, no attributes have been defined for this object since it is there primarily for classification purposes. Attributes may be added to this object in the next phase of the SID once a more complete survey of MemoryCards has been completed.

Business Entity Name	MemoryCard		
Attribute Name	Description		Notes (used to map to other models; blank is other models don't have it)



Business Entity Model – MemoryCard

The MemoryCard business entity model is the same as that of the Card business entity model, which was shown in Figure PR.46 above.



NetworkCard Business Entity Definition

Business Entity Name	NetworkCard
Description	This is a type of Card that is dedicated to providing networking functions, such as routing and forwarding. Line cards and port adapter cards are examples of this type of card. DEN-ng differentiates between this and other types of Cards, such as NetworkCards and MemoryCards.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analog in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, EquipmentHolder, Card, MemoryCard, SystemCard, UnknownCard, Chassis, AuxiliaryComponent, PhysicalComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



NetworkCard Business Entity Attributes Definition

Currently, no attributes have been defined for this object since it is there primarily for classification purposes. Attributes may be added to this object in the next phase of the SID once a more complete survey of NetworkCards has been completed. However, since NetworkCards are so diverse (both in form as well as in function), it is highly likely that this class will remain as is and function as a means to classify different types of Cards.

Business Entity Name	NetworkCard		
Attribute Name	Description		Notes (used to map to other models; blank is other models don't have it)



Business Entity Model – NetworkCard

The NetworkCard business entity model is the same as that of the Card business entity model, which was shown in Figure PR.46 above.



SystemCard Business Entity Definition

Business Entity Name	SystemCard
Description	This is a type of Card that is dedicated to providing system functions. Main processor boards and controller boards are examples of this type of Card. DEN-ng differentiates between this and other types of Cards, such as NetworkCards and MemoryCards.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analog in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, EquipmentHolder, Card, MemoryCard, NetworkCard, UnknownCard, Chassis, AuxiliaryComponent, PhysicalComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



SystemCard Business Entity Attributes Definition

Currently, no attributes have been defined for this object since it is there primarily for classification purposes. Attributes may be added to this object in the next phase of the SID once a more complete survey of SystemCards has been completed. However, since SystemCards are so diverse (both in form as well as in function), it is highly likely that this class will remain as is and function as a means to classify different types of Cards

Business Entity Name	SystemCard		
Attribute Name	Description		Notes (used to map to other models; blank is other models don't have it)



Business Entity Model – SystemCard

The SystemCard business entity model is the same as that of the Card business entity model, which was shown in Figure PR.46 above.



UnknownCard Business Entity Definition

Business Entity Name	UnknownCard
Description	This object is used as a generic placeholder to represent Cards that are known to exist but are not yet identifiable via Software means.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analog in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, EquipmentHolder, Card, MemoryCard, SystemCard, UnknownCard, Chassis, AuxiliaryComponent, PhysicalComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



UnknownCard Business Entity Attributes Definition

Currently, no attributes have been defined for this object since it is there primarily for classification purposes. It is unlikely that any business attributes will be added to this object, since the purpose of this Card is to serve as a placeholder to log unknown Cards as part of an inventory process. Thus, it is highly likely that this class will remain as is and function as a means to classify different types of unknown Cards.

Business Entity Name	UnknownCard		
Attribute Name	Description		Notes (used to map to other models; blank is other models don't have it)



Business Entity Model – UnknownCard

The UnknownCard business entity model is the same as that of the Card business entity model, which was shown in Figure PR.46 above.



EquipmentHolder Business Entity Definition

Business Entity Name	EquipmentHolder
Description	This class is based on the M.3100 specification, and is a base class that represents physical objects that are both manageable as well as able to host, hold, or contain other physical objects. Examples of physical objects that can be represented by instances of this object class are Racks, Chassis, Shelves, Cards, and Slots.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class, based on an M.3100 class. No direct analog in the DMTF CIM.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer (superclass), Equipment, Card, EquipmentHolderAtomic (subclass), EquipmentHolderComposite (subclass), SecureHolder, Chassis, Rack, Slot, Shelf, Bay, PhysicalHolderRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



EquipmentHolder Business Entity Attributes Definition

Business Entity Name	EquipmentHolder		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
type	This attribute identifies the type of the Holder (e.g., Rack (or Bay), Shelf, Sub-shelf, Slot, Subslot, Remote-unit or Remote-subslot).	String	
state	This attribute identifies the state of the Equipment Holder.	HolderState	
manufacturer	This attribute identifies the Equipment Holder manufacturer name. It is defined as a non-empty free format string with no semantics.	String	
location	This attribute identifies the geographical location of the Equipment Holder.	String	
ituArcStateAndStatusList	See R_TMF518_NRB_I_0001 and R_TMF518_NRB_I_0004. See supporting document SD1-8_encodingX731M3100.	ItuArcStateAndStatusList	
isReportingAlarms	This attribute provides an indication of whether alarm reporting for this Equipment Holder is enabled (true) or disabled (false).	Boolean	
expectedOrInstalledEquipmentRef	This attribute identifies the	ObjectName	



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	equipment object expected or installed in the equipment holder, if any. Null if the equipment holder is empty or if it only contains other equipment holders.		
asapRef	This attribute indicates the assignment of an Alarm Severity Assignment Profile (ASAP) to the EquipmentHolder.	ObjectName	
acceptableEquipmentTypeList	This attribute identifies the types of equipment that can be supported by the Equipment Holder.	String	
acceptableEquipmentList	This is an array of strings, based on M.3100, that identifies the types of equipment objects that can be supported by this object. This is an optional attribute.	String	This is a DEN-ng attribute, based on an M.3100 class; no analog in the DMTF CIM.
typeOfHolder	This is an enumerated integer that identifies the type of the Holder that this object instance is. It is based on M.3100 but includes additional values: 0: Unknown 1: Rack 2: Bay 3: Chassis 4: Shelf	Integer	This is a DEN-ng attribute, based on an M.3100 class; no analog in the DMTF CIM.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	5: Slot 6: Sub-Slot 7: Sub-Rack This is a REQUIRED attribute.		
isSolitaryHolder	This is a Boolean attribute that, if TRUE, defines this EquipmentHolder as containing only one ManagedComponent. If this is FALSE, then this EquipmentHolder contains nested ManagedComponents. This is a REQUIRED attribute.	Boolean	
holderStatus	This attribute, based on M.3100, indicates the status of the EquipmentHolder. Values include: 0: Unknown 1: Installed And Acceptable 2: Installed And Not Acceptable 3: Not Installed 4: Mismatch Of Installed and Acceptable 5: Unavailable This is an optional attribute.	Integer	This is a DEN-ng attribute, based on an M.3100 class; no analog in the DMTF CIM.

Business Entity Model – EquipmentHolder

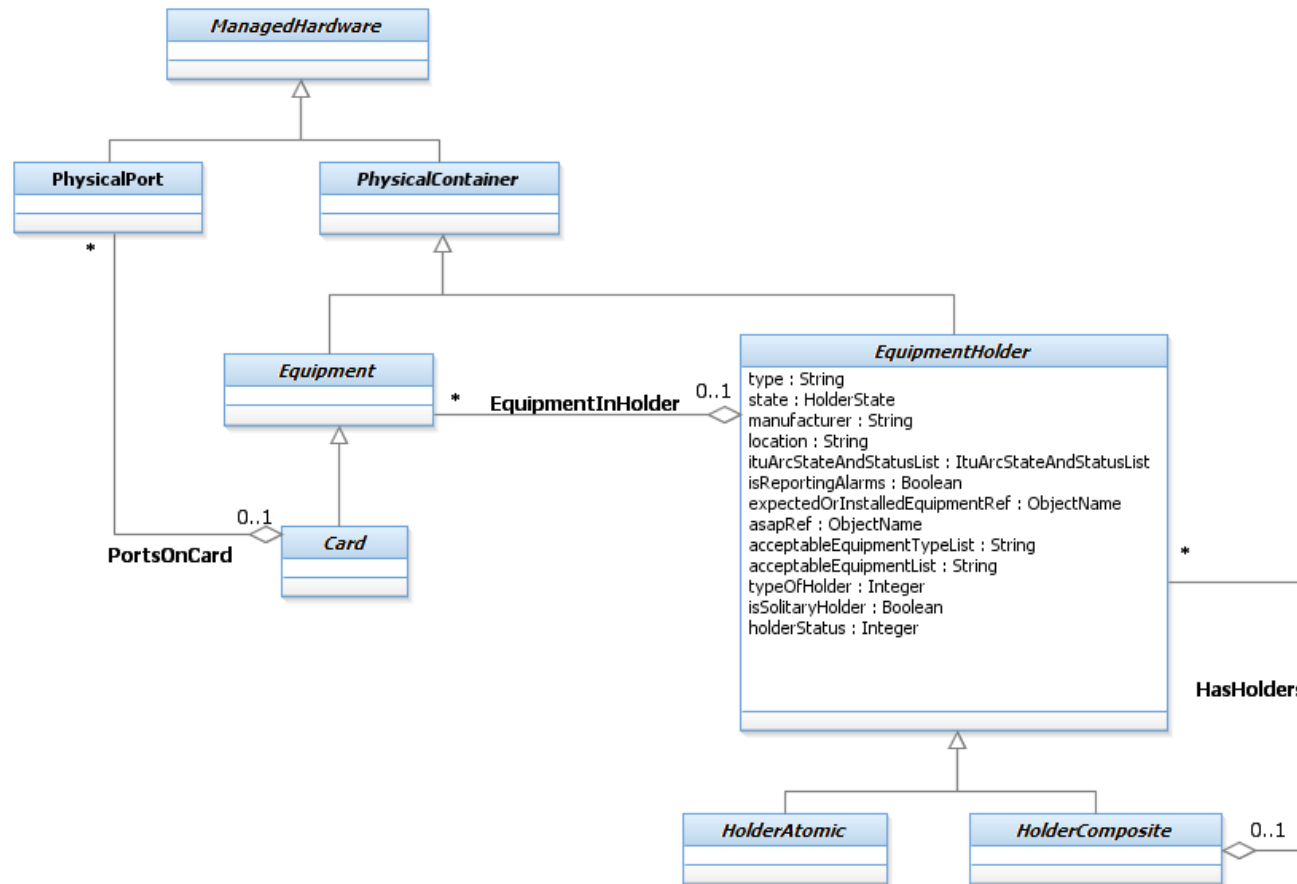


Figure PR- 47. EquipmentHolder Business Entity Model



HolderAtomic Business Entity Definition

Business Entity Name	HolderAtomic
Description	This class represents atomic holders of Equipment that are individually manageable and do NOT form composite, or nested, Equipment Holders. Each HolderAtomic object can be a FRU.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class, based on an M.3100 class. No direct analog in the DMTF CIM, because there is no difference between HolderAtomic and HolderComposite objects.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, Equipment, Card, EquipmentHolder (superclass), EquipmentHolderComposite (sibling), SecureHolder, Chassis, Rack, Slot (subclass), Shelf, Bay, PhysicalHolderRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



HolderAtomic Business Entity Attributes Definition

Business Entity Name	HolderAtomic		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
physicalDescription	This is a free-form string that defines the physically unique characteristics of this holder. This is an optional attribute.	String	This is a DEN-ng attribute, based on an M.3100 class; no analog in the DMTF CIM.
uniquePhysical	This is a Boolean attribute that, if TRUE, means that this holder is physically different from other holders, and is intended to hold a special type of equipment (e.g., a doublewide card, or a longer card than normal). This is an optional attribute.	Boolean	This is a DEN-ng attribute, based on an M.3100 class; no analog in the DMTF CIM.

Business Entity Model – HolderAtomic

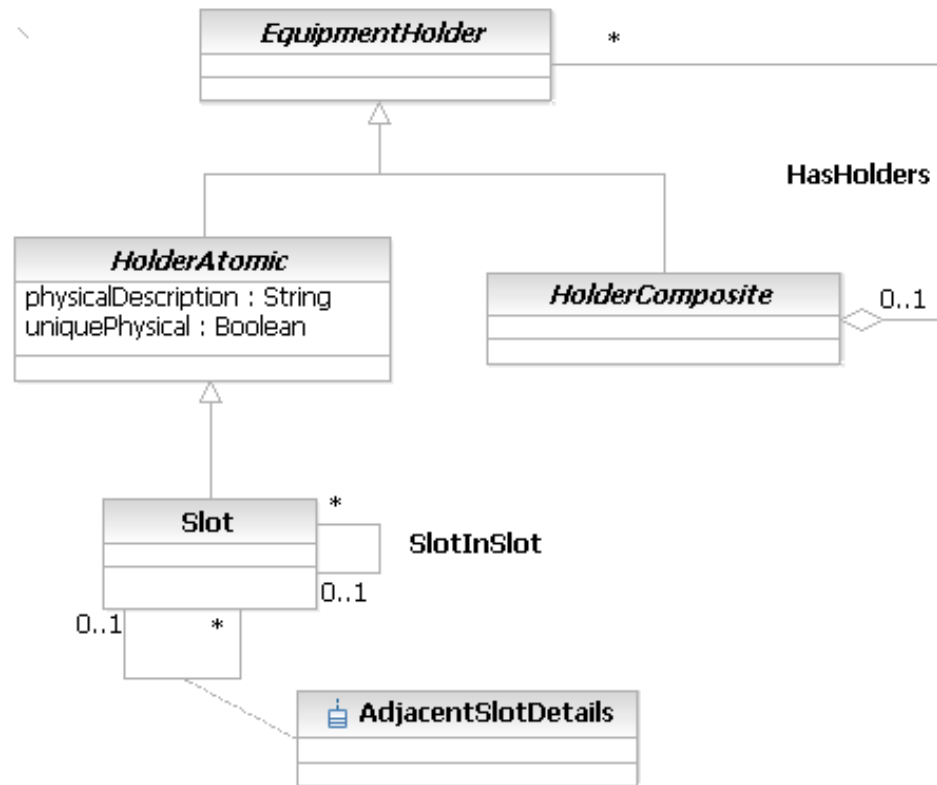


Figure PR- 48. HolderAtomic Business Entity Model



Slot Business Entity Definition

Business Entity Name	Slot
Description	This is a concrete class that has two main purposes. One is to model the ability of a hosting board to accept a daughter card to add or complete the base functionality of the hosting board. The second is to represent the different expansion slots supported by a Chassis.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No counterpart in M.3100; the DMTF CIM has a Slot class but the semantics are different.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, Equipment, Card, EquipmentHolder, EquipmentHolderAtomic (superclass), EquipmentHolderComposite, SecureHolder, Chassis, Rack, Shelf, Bay, PhysicalHolderRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Slot Business Entity Attributes Definition

Business Entity Name	Slot		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
hasAdapter	This is a Boolean attribute that, if TRUE, indicates that this slot has an adapter installed that enables it to accept other types of cards (e.g., fitting an adapter on two Slots enable them to accept a Card that otherwise could not be accommodated). If its value is FALSE, then no adapter is present. This is an optional attribute.	Boolean	This is a DEN-ng; no analog in the DMTF CIM or M.3100.
slotNumber	This is a 16-bit unsigned integer attribute that represents an index into the system slot table. For example, this could be the hardware ID number (starting with 1) for each expansion slot. The number is independent of whether or not the Slot is occupied. This is a REQUIRED attribute.	Integer	This is a DEN-ng attribute; no analog in M.3100. The DMTF CIM has an attribute named SlotNumber; however, the semantics of the Slot class are different.
slotPurpose	This is an enumerated integer that defines the purpose of this Slot. A specific value below, such as System, means that the Slot is intended only to accept System cards. Values include: 0: Unknown 1: System 2: Networking 3: Port Adapter 4: Memory 5: Hardware Assist 6: Video 7: General Computing 8: General Purpose Hardware assist is a generic category for specialty boards that provide hardware functionality to assist in the processing of one or more functions. Examples are special cards	Integer	This is a DEN-ng attribute; no analog in M.3100. The DMTF CIM has an attribute called SpecialPurpose, but it is only a Boolean and thus has more restricted semantics.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	for processing IPsec-based encryption. General computing boards represent cards that have additional and/or auxiliary computing power that can be used for a variety of tasks (not just video rendering or encryption). General purpose boards represent cards that have a variety of features (e.g., memory and computing). This is an optional attribute.		
purposeDescription	This is a free-form string that defines the physically unique characteristics of this Slot. This is an optional attribute.	String	This is a DEN-ng attribute; no analog in M.3100. The DMTF CIM has an attribute called PurposeDescription, but the Slot class has different semantics.

Business Entity Model – Slot

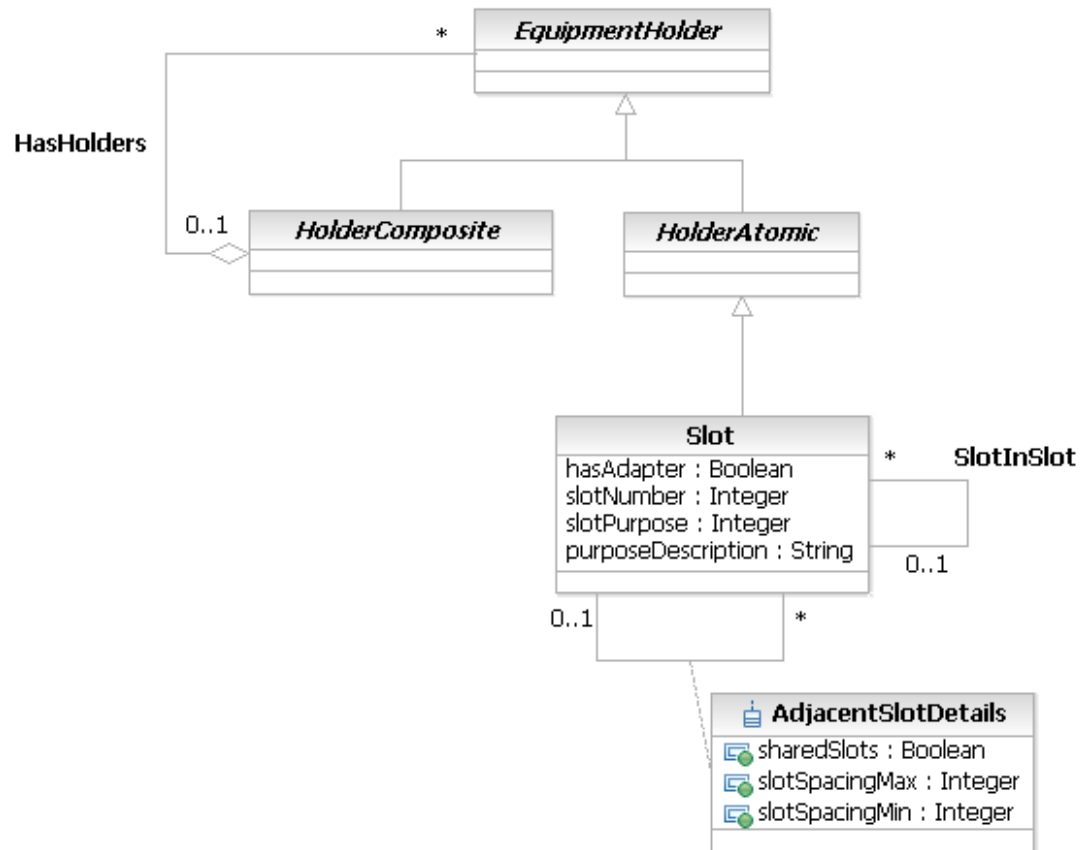


Figure PR- 49. Slot Business Entity Model



HolderComposite Business Entity Definition

Business Entity Name	HolderComposite
Description	This class represents EquipmentHolders that are made up of other EquipmentHolders (i.e., instances of this class as well as the HolderAtomic base class). This provides the semantics of collecting a set of components, each of which is individually manageable, and being able to manage the set of objects as a whole. This containment is modeled using the HasHolders aggregation. Each HolderComposite element can be a FRU. A characteristic of this class is that its subclasses are physical objects that have complex cabling and mounting options. This class is meant to differentiate complex holders, like a Chassis, from simple holders, like a Slot.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class, based on an M.3100 class. No direct analog in the DMTF CIM, because there is no difference between HolderAtomic and HolderComposite objects.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, Equipment, Card, EquipmentHolder (superclass), EquipmentHolderAtomic (sibling), SecureHolder (subclass), Chassis, Rack, Slot, Shelf, Bay, PhysicalHolderRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



HolderComposite Business Entity Attributes Definition

Business Entity Name	HolderComposite		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
cableManagementStrategy	This is a free-form string that contains information on how the various cables contained in the Chassis, Rack, or other type of HolderComposite object are connected and bundled. This property contains information to aid in the assembly and service of the cables contained in a SecureHolder object. This is an optional attribute.	String	This is a DEN-ng attribute; no analog in M.3100. The DMTF CIM has a similarly named attribute in its PhysicalFrame class, though the semantics are slightly different.
mountingOptions	This is an enumerated 16-bit unsigned integer that defines how Equipment in this entity is mounted. Values include: 0: Unknown 1: Stand-alone 2: Rack-mounted, free access 3: Rack-mounted, restricted access 4: Enclosed in another chassis This is an optional attribute.	Integer	This is a DEN-ng class; no analog in the DMTF CIM or M.3100.
serviceApproach	This is an enumerated, integer-valued array that defines how this entity is serviced (e.g., from the top or front), whether it has sliding trays or removable sides, and/or whether the Frame is moveable (e.g., it has rollers). Values include: 0: Unknown 1: Custom 2: Service From Top 3: Service From Front 4: Service From Back 5: Service From Side 6: Sliding Trays	String	This is a DEN-ng class; no analog in the DMTF CIM or M.3100.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	7: Removable Sides 8: Moveable This is an optional attribute.		

Business Entity Model – HolderComposite

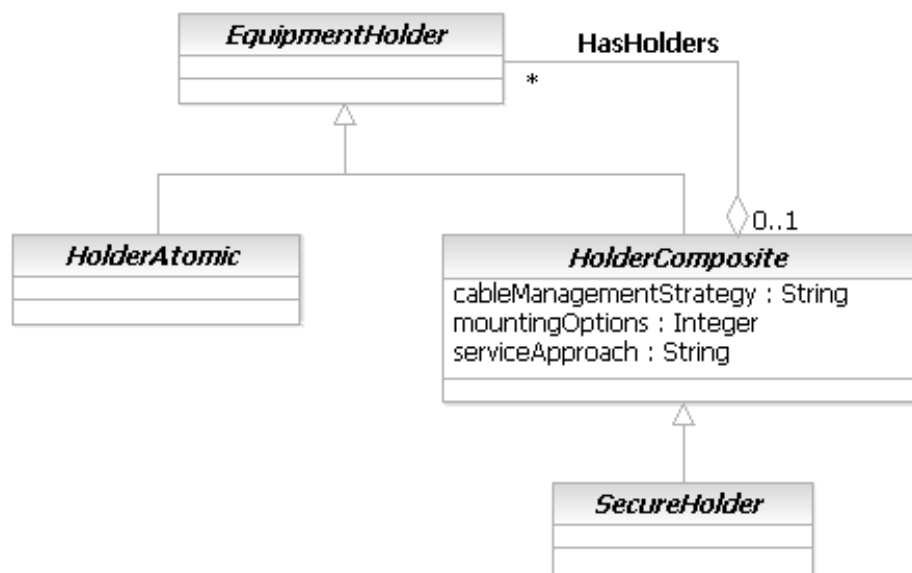


Figure PR- 50. HolderComposite Business Entity Model



SecureHolder Business Entity Definition

Business Entity Name	SecureHolder
Description	This class is a type of HolderComposite that serves as the parent for the Rack and Chassis classes. This class generalizes common properties that apply to Racks and Chassis.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class, based on the original DEN spec. No direct analog in M.3100. The DMTF CIM defines similar attributes, but as part of the PhysicalFrame class, and lacks the semantics and classification of the DEN-ng hierarchy.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, Equipment, Card, EquipmentHolderAtomic, EquipmentHolderComposite (superclass), Chassis (subclass), Rack (subclass), Slot, Shelf, Bay, PhysicalHolderRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



SecureHolder Business Entity Attributes Definition

Business Entity Name	SecureHolder		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
audibleAlarm	This is a boolean attribute that, if TRUE, indicates that this SecureHolder is equipped with an audible alarm. This is an optional attribute.	Boolean	No analog in M.3100; the DMTF CIM has a similarly named attribute in PhysicalFrame.
audibleAlarmDescription	This is a free-form string that provides supplementary information for the AudibleAlarm attribute. It should only be filled in when the value of the AudibleAlarm attribute is TRUE. This is an optional attribute.	String	No analog in M.3100; the DMTF CIM defines an array called ServiceDescriptions in PhysicalFrame to do the same thing.
lockPresent	This is a boolean attribute that, if TRUE, indicates that this SecureHolder is protected by some type of lock. This is an optional attribute.	Boolean	No analog in M.3100; the DMTF CIM has a similarly named attribute in PhysicalFrame.
securityBreach	This is an enumerated 16-bit unsigned integer attribute indicating whether a breach of the Rack was attempted. Values include: 0: Unknown 1: Other 2: No Breach 3: Unsuccessful Breach (but attempted) 4: Successful Breach This is an optional attribute.	Integer	No analog in M.3100; the DMTF CIM has a similarly named attribute in PhysicalFrame.
securityBreachDescription	This is a free-form string attribute that provides supplementary information for the SecurityBreach attribute. It should only be filled in when the value of SecurityBreach is 1 ("Other"). This is an optional attribute.	String	No analog in M.3100; the DMTF CIM has a similarly named attribute in PhysicalFrame.
visibleAlarm	This is a boolean attribute that, if TRUE,	Boolean	No analog in M.3100; the DMTF



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	indicates that the SecureHolder is equipped with one or more visible alarms (e.g., LEDs or gauges). This is an optional attribute.		CIM has a similarly named attribute in PhysicalFrame.
visibleAlarmDescription	This is a free-form string attribute that provides supplementary information for the VisibleAlarm attribute. It should only be filled in when the value of VisibleAlarm is TRUE. This is an optional attribute.	String	No analog in M.3100; the DMTF CIM defines an array called ServiceDescriptions in PhysicalFrame to do the same thing.

Business Entity Model – SecureHolder

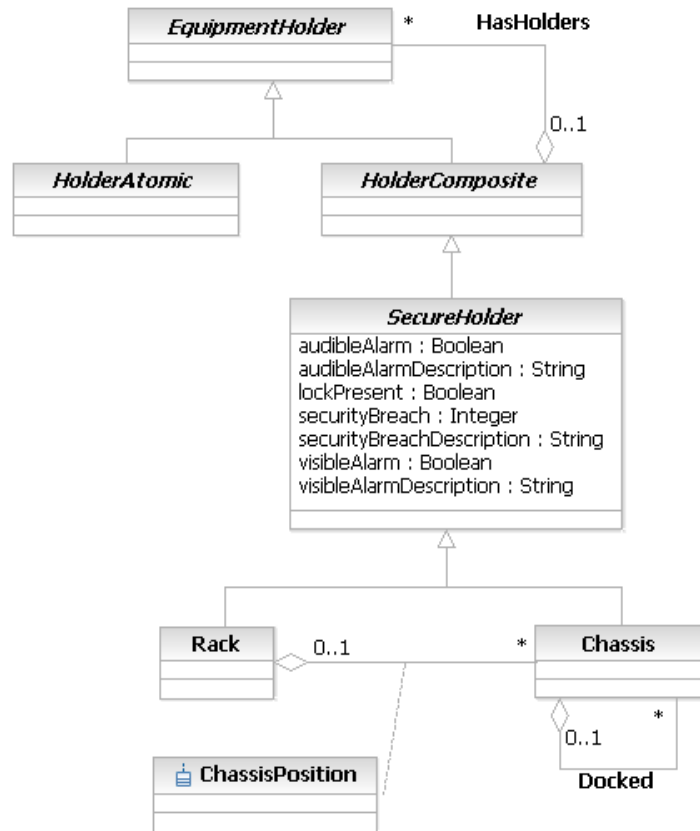


Figure PR- 51. SecureHolder Business Entity Model



Rack Business Entity Definition

Business Entity Name	Rack
Description	A Rack is a type of SecureHolder that represents an enclosure in which EquipmentHolders, such as Chassis, are placed. Typically a Rack is nothing more than the enclosure, and all the functioning componentry is packaged in the Chassis. Note that the logical identifier of a Rack is NOT typically associated with the Device (i.e., the NetworkElement). Compare this to either a Bay or a Shelf, whose logical identifier IS associated with the Device. This means that the Rack is explicitly NOT a part of the logical model of a network. The Rack typically serves as the "master enclosure" for Chassis, Shelves and Bays. In addition, Racks can have multiple instances of multiple Devices mounted in them.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class, based on the original DEN spec. No direct analog in M.3100. The DMTF CIM has a Rack class, but its semantics are slightly different.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer (superclass), Equipment, Card, EquipmentHolderAtomic, EquipmentHolderComposite, SecureHolder (superclass), Chassis (sibling), Slot, Shelf, Bay, PhysicalHolderRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Rack Business Entity Attributes Definition

Business Entity Name	Rack		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
country	This is the designation of the country for which the Rack is designed. Country code strings are as defined by ISO/IEC 3166. This is an optional attribute.	String	No analog in M.3100; the DMTF CIM has a similar attribute named CountryDesignation.
heightInUs	This is the height of the Rack in 'U's. A 'U' is a standard unit of measure for the height of a Rack or rack-mountable components. It is equal to 1.75 inches or 4.445 cm. This is an optional attribute.	Integer	No analog in M.3100; the DMTF CIM has a height attribute higher in the hierarchy and overrides that attribute in the Rack class.
typeOfRack	This is an enumerated integer that defines the type of Rack. Values include: 0: Unknown 1: Standard 19 Inch 2: Telco 3: Equipment Shelf 4: Non-Standard This is an optional attribute.	Integer	No analog in M.3100; the DMTF CIM has a similarly name attribute.

Business Entity Model – Rack

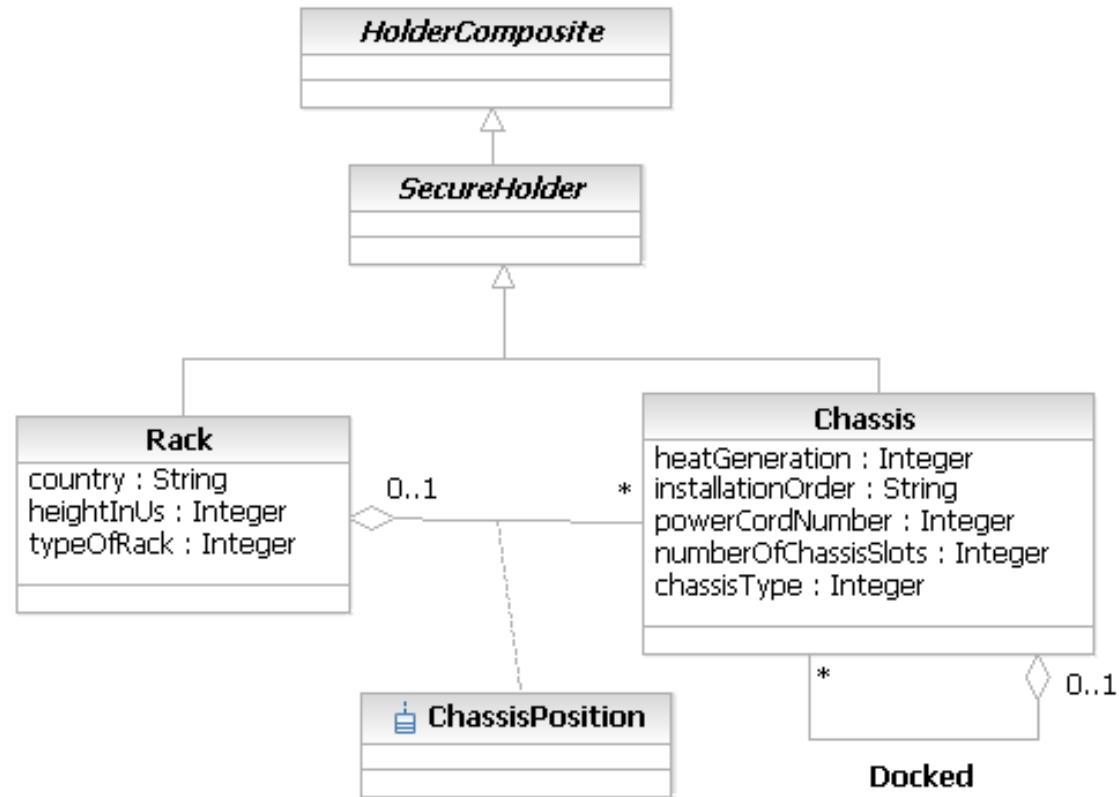


Figure PR- 52. Rack and Chassis Business Entity Model



Chassis Business Entity Definition

Business Entity Name	Chassis
Description	A Chassis is a type of SecureHolder that encloses other ManagedPhysicalEntities and provides a definable functionality in its own right, such as a desktop or a network device (e.g., a router or a switch).
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	A Chassis is a type of SecureHolder that encloses other ManagedPhysicalEntities and provides a definable functionality in its own right, such as a desktop or a network device (e.g., a router or a switch).
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer (superclass), Equipment, Card, EquipmentHolderAtomic, EquipmentHolderComposite, SecureHolder (superclass), Rack (sibling), Slot, Shelf, Bay, PhysicalHolderRole, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Chassis Business Entity Attributes Definition

Business Entity Name	Chassis		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
heatGeneration	This is an integer that defines the amount of heat generated by the Chassis in BTU/hour. This is an optional attribute.	Integer	No analog in M.3100; the DMTF CIM has a similarly name attribute.
installationOrder	This is a free-form string that defines the order of installation of components into the Chassis. This is an optional attribute.	String	No analog in M.3100 or the DMTF CIM.
powerCordNumber	This is an integer that defines the number of power cords which must be connected to the Chassis in order for all of its contained ManagedEntities to operate correctly. This is an optional attribute.	Integer	
numberOfChassisSlots	This defines the number of Slots that are in the Chassis. This does NOT account for any SlotAdapters used - these are described by the CardInSlot association. This is an optional attribute.	Integer	No analog in M.3100 or the DMTF CIM.
chassisType	This is an enumerated integer that defines the type of Chassis that this object is. Values include: 0: Unknown 1: Desktop 2: Low Profile Desktop 3: Pizza Box 4: Mini Tower 5: Tower 6: Portable 7: LapTop 8: Notebook 9: Sub-Notebook 10: Hand Held 11: Docking Station 12: Main System Chassis 13: Expansion Chassis	Integer	



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	14: Bus Expansion Chassis 15: Peripheral Chassis 16: Storage Chassis 17: Rack Mount Chassis This is an optional attribute.		



Business Entity Model – Chassis

The Chassis business entity model is the same as the Rack business entity model, which was shown in Figure PR.52 above.



AuxiliaryComponent Business Entity Definition

Business Entity Name	AuxiliaryComponent
Description	This is an abstract base class that represents managed entities, such as power supplies, fans and cables, which are required for the proper operation of the Device but have a primary function that is different than the primary end-user function(s) of the Device. The difference between AuxiliaryComponents and other subclasses of Equipment are whether or not the physical object performs a function intrinsic to the main function of the Device. This is best understood by way of example. Consider a Router. Its main function is to route and forward packets. A PowerSupply is an AuxiliaryComponent, because even though it is needed for the proper operation of the Router, it does not directly help in routing and forwarding packets. A LineCard (that provides routing functionality) is a subclass of Equipment because its purpose is to route and forward packets. Similar examples exist for different types of equipment, where their criteria may be different. For example, instead of whether it routes or forwards packets, the criterion "does it carry signal" may be useful to appropriately classify components.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This is a DEN-ng class. No direct analog in the DMTF CIM or M.3100.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer (superclass), PowerSupply, CoolingDevice, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



AuxiliaryComponent Business Entity Attributes Definition

The purpose of this class is to serve as a base class for defining different types of AuxiliaryComponents. Thus, it is primarily used for classification purposes, and has no business attributes. Additional types of AuxiliaryComponents will be added in a future phase of the SID.

Business Entity Name		AuxiliaryComponent	
Attribute Name	Description		Notes (used to map to other models; blank is other models don't have it)

Business Entity Model – AuxiliaryComponent

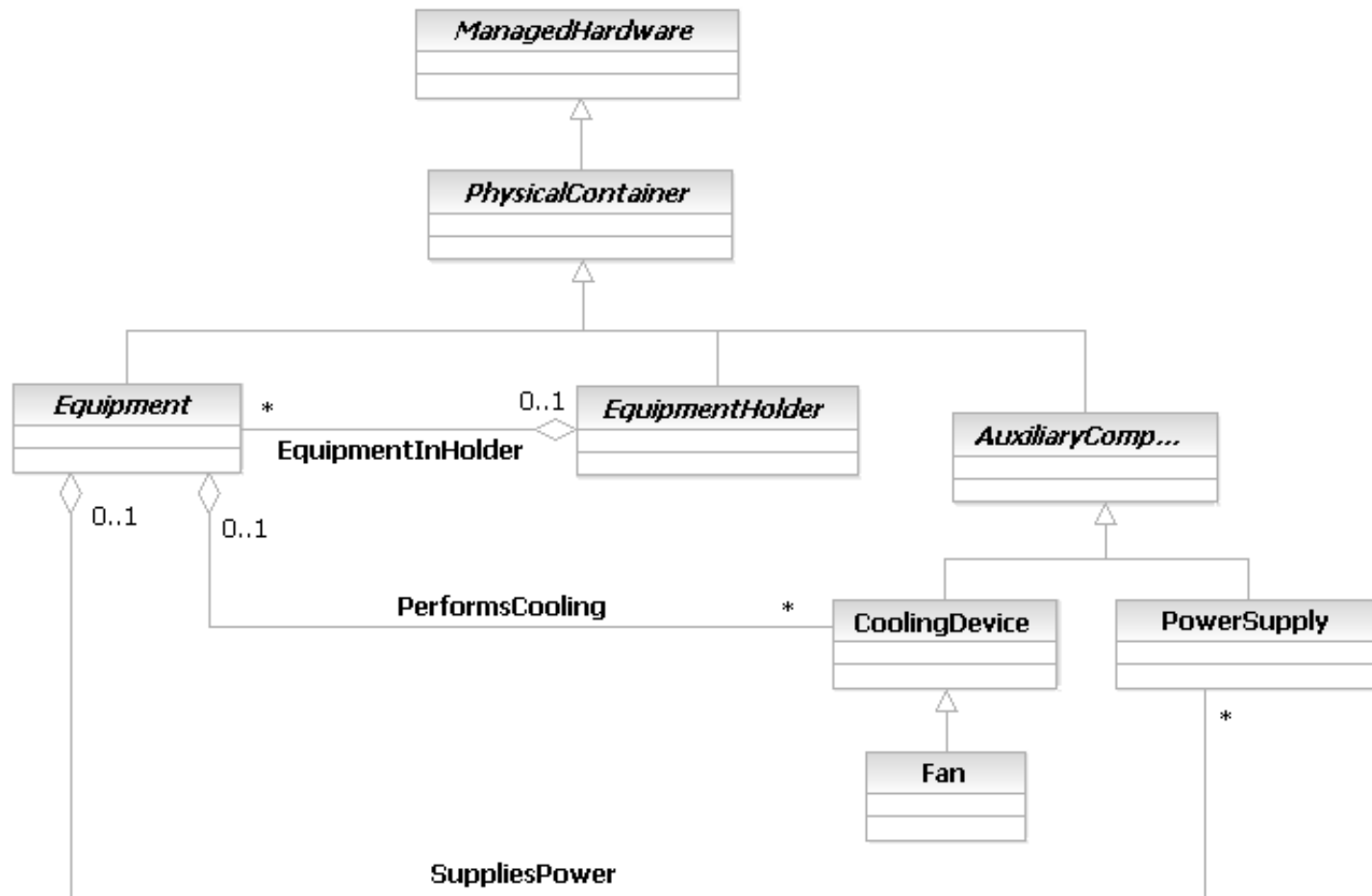


Figure PR- 53. AuxiliaryComponent Business Entity Model



CoolingDevice Business Entity Definition

Business Entity Name	CoolingDevice
Description	This class represents the capabilities of a generic cooling device. Additional detail will be supplied in a later version of the SID. It is present to provide an example of a type of AuxiliaryComponent.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	No direct analog in M.3100. (Note that the CIM has a completely different class hierarchy).
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, AuxiliaryComponent (superclass), PowerSupply, Fan, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



CoolingDevice Business Entity Attributes Definition

Business Entity Name	CoolingDevice		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
hasActiveCooling	This is a Boolean that, if TRUE, means that this CoolingDevice provides active cooling. If it is false, then cooling is provided by a passive means.	Boolean	Not present in M.3100. Present in the CoolingDevice class of the CIM, and called ActiveCooling. However, this class is in the LogicalDevice model. Thus, the class hierarchy and semantics are completely different than those of DEN-ng.



Business Entity Model – CoolingDevice

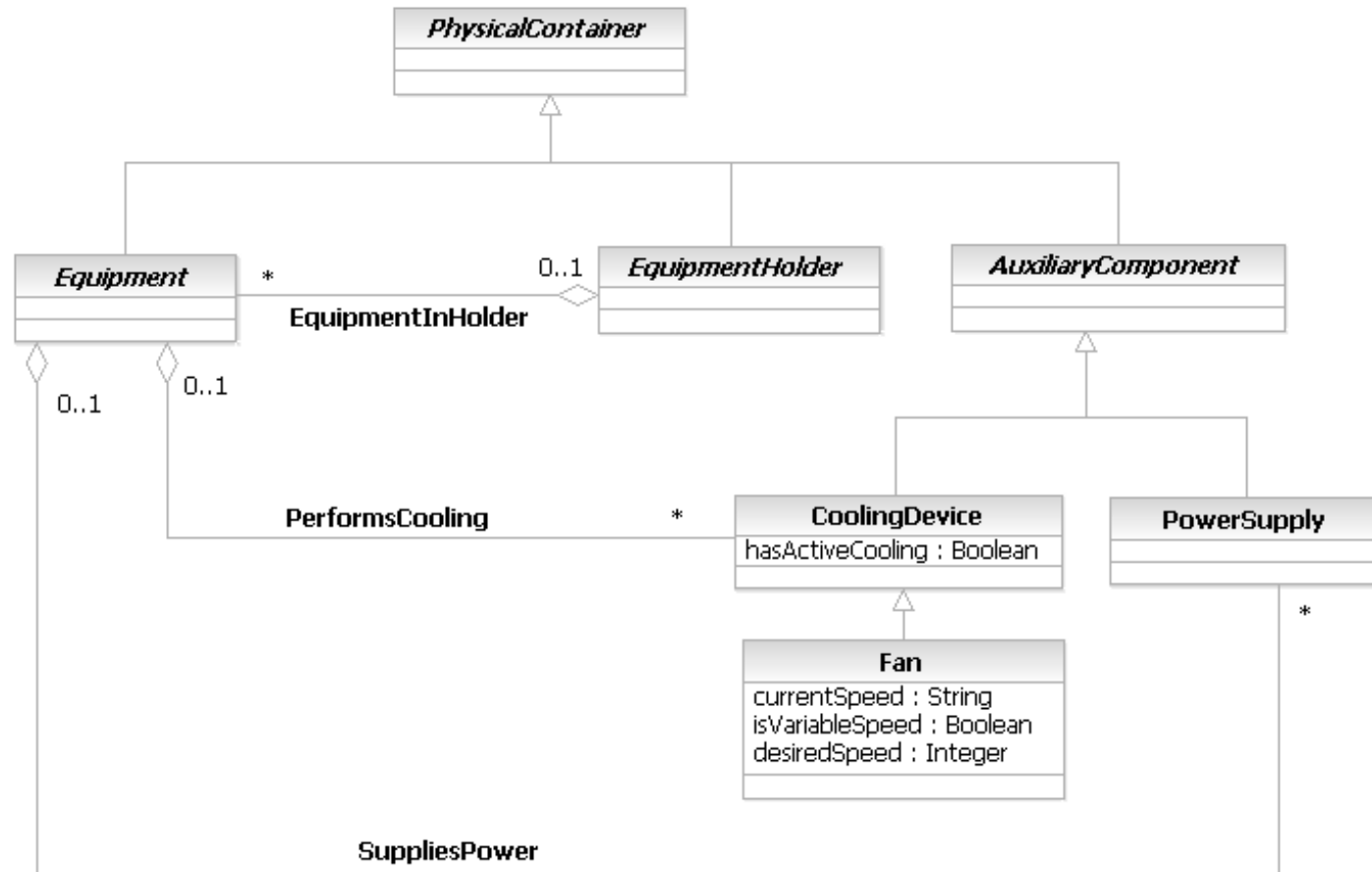


Figure PR- 54. CoolingDevice and Fan Business Entity Model



Fan Business Entity Definition

Business Entity Name	Fan
Description	This class represents the capabilities of a generic Fan. Additional detail will be supplied in a later version of the SID. It is present to provide an example of a type of AuxiliaryComponent.
Sources	DEN-ng, CIM
Cross-References	
Synonyms / Aliases	No direct analog in M.3100. (Note that the CIM has a completely different class hierarchy).
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, AuxiliaryComponent, PowerSupply, CoolingDevice (superclass), Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Fan Business Entity Attributes Definition

Business Entity Name	Fan		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
currentSpeed	This is the current speed of the Fan in revolutions per minute.	String	Not present in M.3100. The CIM derives this attribute through an association with a tachometer.
isVariableSpeed	This is a Boolean attribute that, if TRUE, means that this fan supports variable cooling speeds. If it is FALSE, then this fan only provides a single cooling speed.	Boolean	Not present in M.3100. Present in the Fan class of the CIM. However, this class is in the LogicalDevice model. Thus, the class hierarchy and semantics are completely different than those of DEN-ng.
desiredSpeed	This is an integer attribute that defines the currently requested fan speed, defined in revolutions per minute. For non-variable speed Fans, this attribute has the same semantics as turning the Fan on or off. For variable-speed Fans, this attribute represents the desired speed of the Fan.	Integer	Not present in M.3100. Present in the Fan class of the CIM. However, this class is in the LogicalDevice model. Thus, the class hierarchy and semantics are completely different than those of DEN-ng. In addition, the CIM only allows this attribute for variable speed Fans.



Business Entity Model – Fan

The Fan business entity model is the same as the CoolingDevice business entity model, which was shown in Figure PR.54 above.



PowerSupply Business Entity Definition

Business Entity Name	PowerSupply
Description	This class represents the capabilities of a generic PowerSupply. Additional detail will be supplied in a later version of the SID. It is present to provide an example of a type of AuxiliaryComponent.
Sources	DEN-ng, CIM
Cross-References	
Synonyms / Aliases	No direct analog in M.3100. (Note that the CIM has a completely different class hierarchy).
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, AuxiliaryComponent (superclass), CoolingDevice, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



PowerSupply Business Entity Attributes Definition

Note: the CIM has a similarly named class. However, it is in the LogicalDevice model. Thus, the class hierarchy, attributes, and semantics are completely different than those of DEN-ng.

Business Entity Name	PowerSupply		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
acFrequencyLow	This defines the low value of the AC Frequency range in Hertz. If this is a DC PowerSupply, this value is not applicable and should be set to NULL. This is an optional attribute.	String	Not present in M.3100. The CIM has two attributes for defining two ranges, which are Integer datatypes.
acFrequencyHigh	This defines the high value of the AC Frequency range in Hertz. If this is a DC PowerSupply, this value is not applicable and should be set to NULL. This is an optional attribute.	String	Not present in M.3100. The CIM has two attributes for defining two ranges, which are Integer datatypes.
isAC	This is a Boolean attribute that, if TRUE, signifies this to be an AC PowerSupply. If FALSE, then it is a DC PowerSupply. This is a required attribute.	Boolean	Not present in M.3100 or the CIM (it can be derived by examining various other attributes).
isSwitching	This is a Boolean attribute that, if TRUE, indicates that this PowerSupply is a switching (vs linear) PowerSupply. This is a required attribute.	Boolean	Not present in M.3100. The CIM calls this IsSwitchngSupply.
inputVoltageLow	This is a string that defines the low value for the Input Voltage Range for this PowerSupply. This is an optional attribute.	String	Not present in M.3100. The CIM has two attributes for defining two ranges, which are Integer datatypes.
inputVoltageHigh	This is a string that defines the high value for the Input Voltage Range for this PowerSupply. This is an optional attribute.	String	Not present in M.3100. The CIM has two attributes for defining two ranges, which are Integer datatypes.
inputCurrentMax	This is a string that defines the input current rating in amperes for a fully populated chassis. For a DC PowerSupply, this is the maximum current drawn when operating at the lowest permissible value of its input	String	Not present in M.3100 or the CIM.



Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
	voltage range. For an AC PowerSupply, the value of this attribute is the same as the value of the InputCurrentMin attribute. This is an optional attribute.		
inputCurrentMin	This is a string that defines the input current rating in amperes for a fully populated chassis. For a DC PowerSupply, this is the minimum current drawn when operating at the lowest permissible value of its input voltage range. For an AC PowerSupply, the value of this attribute is the same as the value of the InputCurrentMax attribute. This is an optional attribute.	String	Not present in M.3100 or the CIM.
dcOutputPower	This is the maximum value of the output power of the DC PowerSupply. If this is an AC PowerSupply, then this attribute must be set to NULL. This is an optional attribute.	String	Not present in M.3100. The CIM calls this TotalOutputPower.
isRedundant	This is a Boolean attribute that, if TRUE, signifies this PowerSupply as a redundant PowerSupply. This is a required attribute.	Boolean	Not present in M.3100 or the CIM.



Business Entity Model – PowerSupply

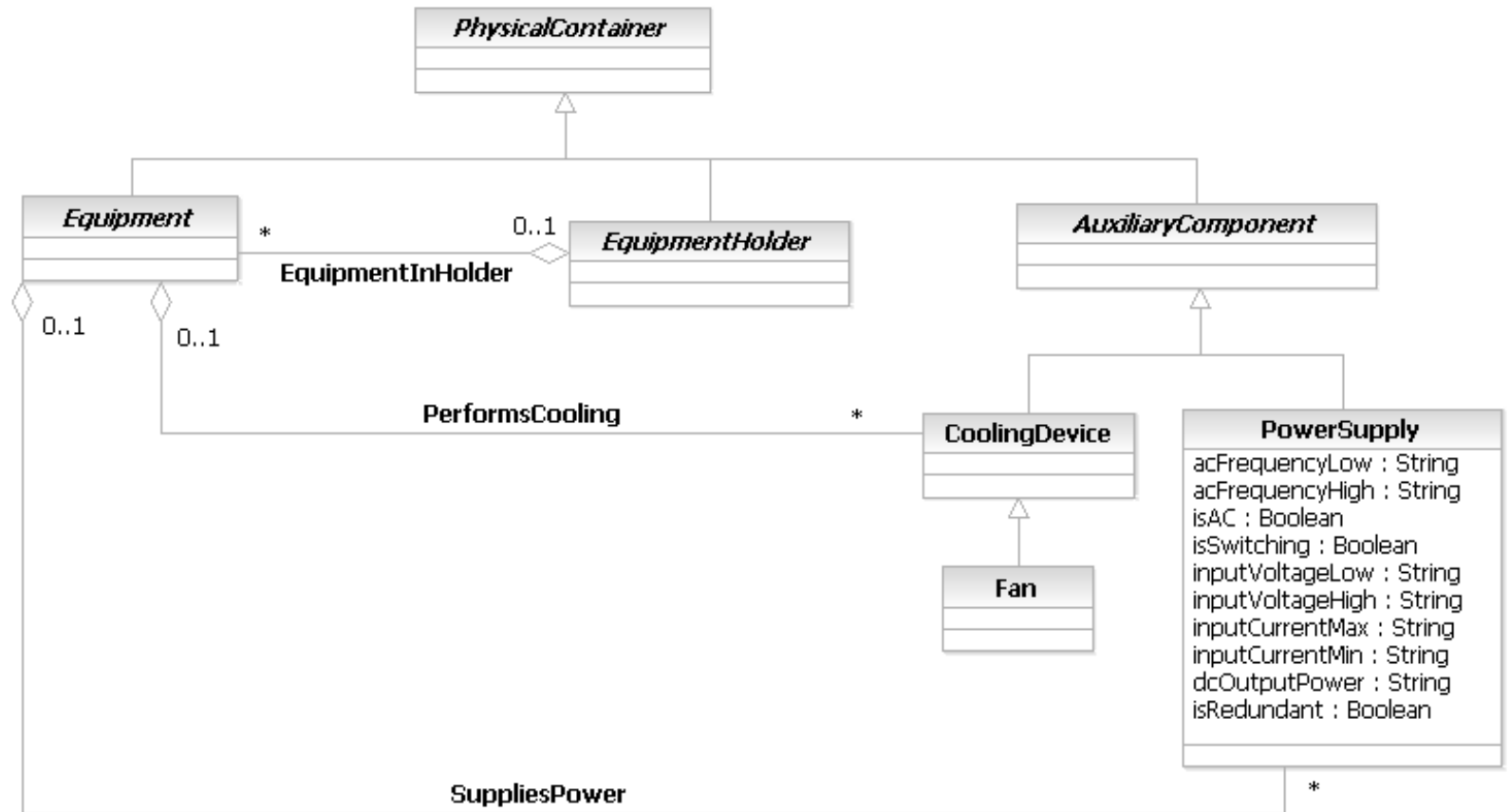


Figure PR- 55. PowerSupply Business Entity Model



PhysicalComponent Business Entity Definition

Business Entity Name	PhysicalComponent
Description	This is the base class for different types of PhysicalComponents that can reside either in an Equipment or an EquipmentHolder object. They can NOT be used as a stand-alone object. From a management point-of-view, this object either cannot or does not need to be split into its constituent parts. For example, an ASIC (or Chip) cannot, and a tape for data storage does not, need to be split up into their constituent parts. Any piece of hardware that is not a PhysicalLink, PhysicalConnector, Equipment, or EquipmentHolder, is a subclass of this class.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	No direct analog in M.3100. (Note that the CIM has a similarly named class, but its function is different. The DEN-ng PhysicalContainer class contains those attribute. There is no configuration notion in the current version of the CIM, and hence there is no counterpart in the CIM to this class.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer (superclass), Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, AuxiliaryComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



PhysicalComponent Business Entity Attributes Definition

Business Entity Name	PhysicalComponent		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
isConfigurable	This is a Boolean attribute that, if TRUE, means that this PhysicalComponent is configurable by the end-user. This is an optional attribute.	Boolean	No analog in either the M.3100 or the DMTF CIM.

Business Entity Model – PhysicalComponent

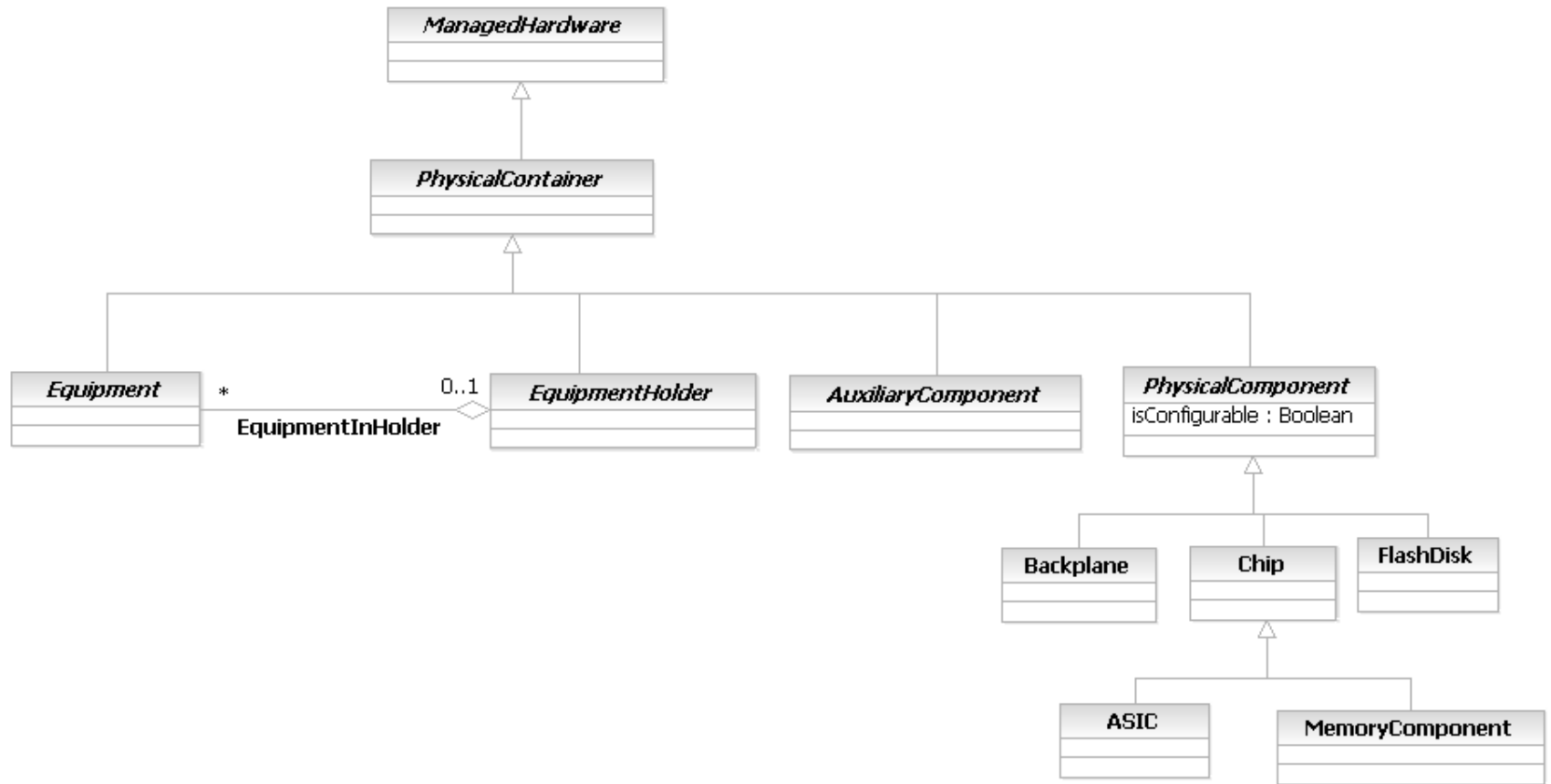


Figure PR- 56. PhysicalComponent Business Entity Model



Backplane Business Entity Definition

Business Entity Name	Backplane
Description	This class models the backplane that is present in devices. The current version of this class is serving as a placeholder to illustrate different types of PhysicalComponents. It will be further developed in the next version of the SID.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	This class models the backplane that is present in devices. The current version of this class is serving as a placeholder to illustrate different types of PhysicalComponents. It will be further developed in the next version of the SID.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, AuxiliaryComponent, PhysicalComponent (superclass), PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Backplane Business Entity Attributes Definition

Business Entity Name	Backplane		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)



Business Entity Model – Backplane

The Backplane Business Entity Model is the same as that for PhysicalComponent, and is shown in Figure PR.56 above.



Chip Business Entity Definition

Business Entity Name	Chip
Description	This is a concrete class that models different types of chips that are either directly configurable by the end-user (e.g., a programmable device) and/or represent FRUs (e.g., a special ASIC that can be upgraded by replacing it with a new version of that same ASIC). Note that capacity characteristics (e.g., how much memory can a Flash chip hold) are defined through the <u>PhysicalCapacity</u> association.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	No direct analog in M.3100. The CIM has a similarly named class, but its position in the CIM class hierarchy is different than the position of this class in the DEN-ng class hierarchy. Therefore, the semantics are different.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, AuxiliaryComponent, PhysicalComponent (superclass), ASIC, MemoryComponent, Backplane, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



Chip Business Entity Attributes Definition

Business Entity Name	Chip		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
formFactor	This is an enumerated integer that defines the form factor of this PhysicalComponent. Values include: 0: Unknown 1: Other 2: Custom 3: SIP 4: DIP 5: ZIP 6: SOJ 7: SIMM 8: DIMM 9: PGA 10: RIMM 11: SRIMM 12: SODIMM 13: SOIC 14: LCC 15: PLCC This is an optional attribute.	Integer	No analog in M.3100; this is a subset of the DMTF CIM attributes.



Business Entity Model – Chip

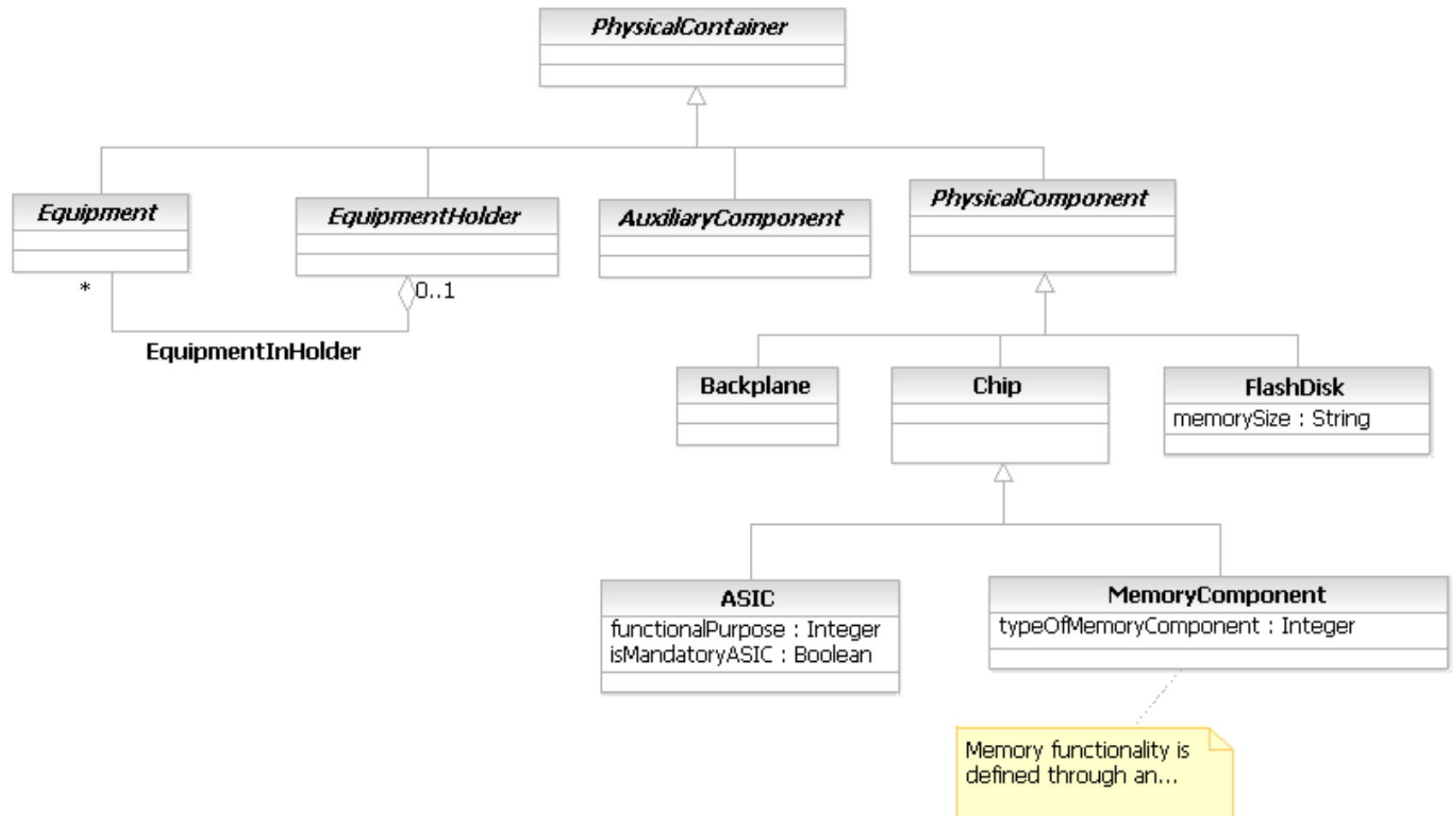


Figure PR-57 - Chip, ASIC, and MemoryComponent Business Entity Model



ASIC Business Entity Definition

Business Entity Name	ASIC
Description	An ASIC is an Application Specific Integrated Circuit. This class represents a special type of Chip that is used to upgrade the functionality of a networking device. Examples include upgrading the modem function of a device, increasing the processing capability of a Card or Device, and implementing some type of networking functionality, such as fast switching, directly in hardware (as opposed to emulating it in software).
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	No direct analog in M.3100 or the CIM.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, PhysicalComponent, Chip (superclass), MemoryComponent, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, AuxiliaryComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



ASIC Business Entity Attributes Definition

Business Entity Name	ASIC		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
functionalPurpose	This is an enumerated integer that defines the functional purpose of this ASIC. Values include: 0: Unknown 1: Routing Traffic 2: Forwarding Traffic 3: Encrypting Traffic 4: Firewalling 5: Encoding 6: General Computational Functions Additional values will be added in the next release of DEN-ng.	Integer	No analog in either the M.3100 or the DMTF CIM.
isMandatoryASIC	This is a Boolean attribute that, if TRUE, signifies that this ASIC is mandatory for the correct operation of the PhysicalResource that contains it.	Boolean	No analog in either the M.3100 or the DMTF CIM.



Business Entity Model – ASIC

The ASIC Business Entity Model is the same as that for Chip, and is shown in Figure PR.57 above.



MemoryComponent Business Entity Definition

Business Entity Name	MemoryComponent
Description	This class represents memory devices that are implemented as Chips. Note that this only represents the physical packaging of the Memory Chip – the logical functionality is defined in a separate class, called Memory, that is part of the LogicalResource model. The current version of this class is serving as a placeholder to illustrate different types of PhysicalComponents. It will be further developed in the next version of the SID.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	No direct analog in M.3100 or the CIM. (Note that the CIM has a class named Memory, but its (class) derivation is different. Hence, there is no counterpart in the CIM to this class.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, PhysicalComponent, Chip (superclass), ASIC, Backplane, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, AuxiliaryComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



MemoryComponent Business Entity Attributes Definition

Business Entity Name	MemoryComponent		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
typeOfMemoryComponent	This is an enumerated integer that defines the type of memory that this Chip is. Values include: 0: Unknown 1: Proprietary 2: RAM 3: DRAM 4: Synchronous DRAM 5: Cache DRAM 6: EDRAM 7: VRAM 8: SRAM 9: EDO 10: ROM 11: PROM 12: EPROM 13: EEPROM 14: FEPRAM 15: Flash 16: CDRAM 17: 3DRAM 18: SDRAM 19: SGRAM 20: RDRAM 21: DDR This is an optional attribute.	Integer	No analog in either the M.3100 or the DMTF CIM.



Business Entity Model – MemoryComponent

The MemoryComponent Business Entity Model is the same as that for Chip, and is shown in Figure PR.57 above.



FlashDisk Business Entity Definition

Business Entity Name	FlashDisk
Description	FlashDisk are memory-based devices that conform to the PC Card (formerly PCMCIA) standard, and that present an ATA (AT Attachment) interface to the device. The Flash Disk has controller circuitry that allows it to emulate a hard disk and that automatically maps out bad blocks and performs automatic block erasure. Furthermore, the Flash Disk provides the capability to allocate noncontiguous sectors, which a Flash memory device can't.
Sources	DEN-ng
Cross-References	
Synonyms / Aliases	No direct analog in M.3100 or the CIM.
Related Business Entities	Resource, PhysicalResource, PhysicalDevice, Hardware, ManagedHardware, PhysicalContainer, PhysicalComponent (superclass), Chip, ASIC, MemoryComponent, Backplane, Equipment, Card, EquipmentHolder, Chassis, Rack, Slot, Shelf, Bay, AuxiliaryComponent, PhysicalResourceRole, PhysicalDeviceRole, HardwareRole
Business Rules	<i>To be determined</i>



FlashDisk Business Entity Attributes Definition

Business Entity Name	FlashDisk		
Attribute Name	Description	Data Type	Notes (used to map to other models; blank is other models don't have it)
memorySize	This is the size of the Flash disk in MBytes.	String	No analog in either the M.3100 or the DMTF CIM.



Business Entity Model – FlashDisk

The FlashDisk Business Entity Model is the same as that for Chip, and is shown in Figure PR.57 above.



2. Administrative Appendix

This Appendix provides additional background material about the TM Forum and this document. In general, sections may be included or omitted as desired; however, a Document History must always be included.

2.1. About this document

This is a TM Forum Guidebook. The guidebook format is used when:

- The document lays out a 'core' part of TM Forum's approach to automating business processes. Such guidebooks would include the Telecom Operations Map and the Technology Integration Map, but not the detailed specifications that are developed in support of the approach.
- Information about TM Forum policy, or goals or programs is provided, such as the Strategic Plan or Operating Plan.
- Information about the marketplace is provided, as in the report on the size of the OSS market.

2.2. Document History

Version History

This section records the changes between this and the previous Official document release

Version Number	Date Modified	Modified by	Description of changes
0.1	January 2002	J. Strassner	GB915 Addendum Template created from TMF407 v1.1 Guidebook template
0.2	February 2002	J. Strassner	Initial population using DEN-ng physical model, which contains mined information from DEN, CIM, and M.3100
0.3	March 2002	J. Strassner	Revisions and publication to Resource team
0.4	April 2002	J. Strassner	Revisions from JohnR and Wayne incorporated
0.45	April 2002	J. Strassner	Revisions to build closer link to Location model; additional business entity definitions
0.5	May 2002	J. Strassner	Final revisions per SID conference call
1.0	June 2002	TMF	Formatted for Member Evaluation release



1.1	August 2002	J. Strassner	Incorporation of review comments from Phase 2 work
1.5	September 2002	J. Strassner	Incorporated final review comments from Phase 2 work and implementation experience comments
1.6	October 2002	J. Strassner	Minor revisions incorporated from final review
2.0	October 2002	TMF	Formatted for Member Evaluation release
2.1	May 2003	J. Strassner	Revisions to sync this model to other Addenda that are part of NGOSS release 3.5
2.2	June 2003	J. Strassner	Final submission to team
2.3	June 2003	J. Strassner	Minor corrections to PartyRole interaction
3.0	July 2003	TMF	Formatted for Member Review with NGOSS R3.5
3.1	Aug 2004	TMF	To reflect TMF Approved status
9.0	1 Mar 2010	John Wilmes	Generated business entity section
9.1	28 Apr 2010	Alicja Kawecki	Minor cosmetic corrections for web posting and ME
9.2	31 Dec 2010	John Reilly	Updated Notice
9.3	23 Mar 2011	Alicja Kawecki	Minor formatting corrections for web posting and ME
9.4	30 Sep 2011	John Wilmes	Updates to document
9.5	4 Oct 2011	Alicja Kawecki	Updated cover page and footer to reflect TM Forum Approved status
9.6	11 Jun 2013	John Reilly	Removed duplicate manufactureDate from EquipmentHolder.
9.7	30 July 2013	Alicja Kawecki	Rebranding; updated header & corrected footer prior to posting
9.8	18 Sep 2013	Alicja Kawecki	Added IPR Mode to cover page, updated notice to reflect TM Forum Approved status
9.8.1	11 Nov 2013	Alicja Kawecki	Updated cover, header, footer & Notice
14.5.0	10 Oct 2014	John Reilly	Remove references to value network role.
15.0.0	30 April 2015	Cécile Ludwichowski	Takes into account CR artf3576 (https://collab.tmforum.org/sf/go/artf3576/) related to improper naming of relationships.
15.0.1	19 May 2015	Alicja Kawecki	Updated cover; minor cosmetic corrections
15.0.2	27 May 2015	Cécile Ludwichowski	Updated Figure numbering based on the document submitted for TMF approval on confluence
16.0.0	02 May 2016	Cécile Ludwichowski	Added ResourceSpecRelationship and ResourceRelationship composite/Atomic pattern removed from ResourceSpec subclasses Figures updated
16.0.1	25 May 2016	Alicja Kawecki	Minor cosmetic edits prior to publication for Fx16
16.0.2	8 Sep 2016	Alicja Kawecki	Updated cover, header and Notice to reflect TM Forum Approved status



Release History

Release Number	Date Modified	Modified by:	Description of changes
9.0	28 Dec 2009	Keith Dorking	Addition of TIP model and various adjustments/corrections
9.0	1 Mar 2010	John Wilmes	Generated business entity section
9.5	31 Dec 2010	John Reilly	Updated Notice
13.0	11 Jun 2013	John Reilly	Removed duplicate manufactureDate from EquipmentHolder.
14.5.0	10 Oct 2014	John Reilly	Remove references to value network role.
15.0.0	30 April 2015	Cécile Ludwichowski	Takes into account CR artf3576 (https://collab.tmforum.org/sf/go/artf3576/) related to improper naming of relationships.
16.0.0	02 May 2016	Cécile Ludwichowski	- Added ResourceSpecRelationship and ResourceRelationship - composite/Atomic pattern removed from ResourceSpec subclasses - Figures updated

2.3. Acknowledgments

This document was prepared by the members of the TM Forum Information Framework (SID) team.

The Shared Information/Data Model is a genuinely collaborative effort. The TM Forum would like to thank the following people for contributing their time and expertise to the production of this document. It is just not possible to recognize all the organizations and individuals that have contributed or influenced the introduction. We apologize to any person or organization we inadvertently missed in these acknowledgments.

Key individuals that reviewed, provided input, managed, and determined how to utilize inputs coming from all over the world, and really made this document happen were:



Name

Cliff Faurer
Ram Garg
Chris Hartley
Helen Hepburn
Jenny Huang
Matt Izzo
Mike McLoughlin
John Reilly
Wayne Sigley
John Strassner

Affiliation

TM Forum
MetaSolv Software
Telstra
British Telecom
AT&T
Agilent
British Telecom
MetaSolv Software
Telstra
Intelliden (editor)